Federated Learning on Resource-Constrained IoT and TinyML Devices: A Systematic Review of Constraints, Optimization, and Deployment Evidence

ARTICLE INFO

Keywords:
Artificial Intelligence of Things
Federated Learning
TinyML
Intelligent Edge
Resource Constraints
Model Compression
Privacy-Preserving IoT

ABSTRACT

Federated learning (FL) is proposed for Internet of Things (IoT) devices because it trains models where the data are generated, yet how far it has been demonstrated on microcontroller-class devices is unclear. Following PRISMA 2020, this review searched Scopus and six publisher libraries for peer-reviewed studies published from 2019 onward and included 93 studies (96 reports) that evaluated FL for microcontroller-class (MCU) or single-board-computer (SBC) clients against an explicitly identified baseline. Targeted and evaluated hardware were classified separately by device tier, and each evaluation as physical, hybrid or simulated. Of the 93 studies, 62 trained on physical devices, 38 of them SBCs, and 26 trained on MCU-class hardware, typically on three or four boards and fully specified in six. Evaluation matched the targeted tier in 76 studies; all 15 mismatches used more capable hardware. The resources that bind MCUs were the least measured: peak training memory was reported by 26 studies, optimizer state by one and activation memory by none, and on MCUs memory changes were measured in four studies, energy changes in four and communication changes in three. Reported resource changes favored the proposed technique in 84.3% of cases but rest on different baselines and scopes and cannot be pooled. Of the 29 studies with a security or privacy mechanism, 8 were fully reported (property, threat model and client-side cost), one evaluated on an MCU. Only 8 of 4,278 study pairs share a dataset, architecture, evaluation setting, baseline and metric, and public training code was found for 18 studies. These figures describe the best-documented part of the eligible literature: among the 276 eligible reports excluded for insufficient data, physical MCU-class training was rarer (5.1% against 25.8%), so the field as a whole is likely weaker on these measures. We propose research directions and a minimal benchmark and reproducibility package for FL on constrained devices.

1. Introduction

1.1. Key issues

The ability of federated learning (FL) to train on the devices that hold the raw data makes it attractive for the Internet of Things (IoT), where centralized data collection may be infeasible given network scale and privacy concerns [1, 2]. However, FL methods assume that every client has the resources to train, whereas an IoT device may be short of compute, memory, network bandwidth or energy [3]. Tiny machine learning (TinyML) brings ML models onto such low-resource devices [4], particularly microcontrollers (MCUs) [5], whose memory is two to three orders of magnitude smaller than a mobile phone's [6], by keeping models within the device's compute and latency budget [7]. Yet TinyML has largely supported only inference, with models trained on a powerful machine and flashed to the device [8], whereas FL requires on-device training, whose memory consumption is prohibitive for such devices [9]. Whether FL is feasible on a given device must therefore be demonstrated against that device's actual capacity.

Furthermore, claimed feasibility is not demonstrated feasibility. How practical FL is on resource-constrained IoT devices was already named as an open question by early end-to-end evaluations [10], and real devices differ from idealized clients: because hardware and device state vary, a client may be unable to train when called or may fail to deliver its update [11]. Optimizations bring trade-offs of their own; on microcontrollers, shortening the interval between federated rounds brought the training loss down sooner at the price of more bandwidth and a longer training run [12]. A reported gain is therefore interpretable only against an identified baseline on an identified device tier.

Moreover, keeping raw data local does not by itself make FL private, since private training data can be recovered from shared gradients [13]. Defenses consume client resources: a trusted-execution design for mobile FL, for instance, increased client CPU time, memory use and energy consumption [14], so whether such protection fits within microcontroller budgets must also be established. Finally, the evidence is difficult to compare and reproduce. TinyML has long lacked a benchmark that is both widely accepted and easy to reproduce [15], and none of 400 surveyed AI conference papers documented every variable needed to reproduce their results [16]. Without comparable and reproducible evidence, it remains unclear which FL approaches are viable on which constrained devices, the question this review sets out to answer.

1.2. Research gap

Existing reviews have surveyed FL for resource-constrained IoT devices [3, 17], where studies rely on lightweight architectures and optimization techniques to cope with limited memory, processing power, and energy [18]. However, these reviews largely catalog what has been proposed, not where it has been demonstrated. None of the reviews we audited (Section 1.8) compares the device class a study targets with the one it is evaluated on, stratifies reported effects

by device tier, or audits the reproducibility artifacts of its included studies, and the four that discuss the cost of security mechanisms on constrained devices do so only qualitatively. A method validated only in simulation or on a single-board computer can therefore pass as evidence of microcontroller feasibility.

1.3. Significance of the study

Closing these gaps matters because TinyML targets billions of IoT devices and microcontrollers [5], where on-device training lets models adapt to new sensor data without sending it to the cloud [9]. Choosing an FL method for such hardware requires evidence that it fits and runs there, not only that it was designed for it. Systematic reviews can identify such problems in primary research so that future studies rectify them [19]; by separating targeted from evaluated hardware, stratifying effects by device tier, and auditing reproducibility artifacts, this review reports which FL approaches have been demonstrated on which device tiers.

1.4. Primary contribution of this study

This study contributes an evidence-centric review of where FL methods for resource-constrained IoT and TinyML devices have been demonstrated, not only what has been proposed:

- A rule-based device-tier classification (T1–T4) of both the targeted and the evaluated hardware of every included study, alongside its evaluation mode.
- A comparison of targeted with evaluated tiers, showing how often methods are validated on the device tier they target.
- Optimization techniques mapped to the constraints they address, with reported effects tied to an identified baseline and stratified by device tier, not pooled.
- Reported memory, model-size, and computation figures read against device capacity, with the MCU-native runtimes, libraries, and frameworks used for FL on MCUs.
- Security and privacy mechanisms mapped to their property, threat model, client-side cost, and feasibility on microcontrollers.
- An audit of cross-study comparability and of the reproducibility artifacts primary studies release, from code and data to random seeds, with this review's materials archived on Zenodo.

1.5. Research objectives

This review aims to identify the device tiers and evaluation modes on which FL methods for resource-constrained IoT and TinyML devices are validated, and how often each occurs; the optimization techniques proposed and the constraint each addresses; the MCU-native runtimes, libraries, and frameworks used for FL on MCUs; and the security and privacy mechanisms employed, with their properties and threat models. Its purpose is to establish where these methods have been demonstrated and whether reported training memory, optimizer state, and model and update size fit the target device. The review evaluates changes in task performance, model size, memory, computation, communication, latency, and energy against an explicitly identified baseline under stated conditions, reporting trade-offs alongside gains, and investigates the client-side overheads of security mechanisms and their feasibility on MCUs. Comparisons are made between targeted and evaluated tiers, across device tiers, and across studies by dataset, model, metric, hardware, communication protocol, and baseline, without pooling incomparable results. Finally, it audits which reproducibility artifacts primary studies release and derives research directions from the remaining gaps.

1.6. Research questions

The questions were formulated with the PICOC framework (Population, Intervention, Comparison, Outcome, Context; Section 2.3). Resource-constrained IoT and TinyML devices denotes the review population, device tiers T1 and T2; MCU-class denotes T1 and SBC-class T2, so questions naming MCU-class devices are deliberately narrower than the population. Each question is answered from the included studies, RQ4 from those reporting a security or privacy mechanism with its property, threat model, and client-side cost, and the frequency of insufficiently specified hardware (RQ1a) also from the full-text exclusion counts.

RQ1 — Deployment evidence

RQ1a. What device tiers and evaluation modes are used to validate FL methods targeting resource-constrained IoT and TinyML devices, and how is the evidence distributed across physical deployment on MCU-class devices, evaluation on SBC-class devices, hybrid hardware-in-the-loop evaluation, simulation or emulation, and cases where hardware is insufficiently specified?

RQ1b. To what extent does the hardware on which FL methods are actually evaluated correspond to the device tier those methods claim or are designed to target?

RQ2 — Resource optimization

RQ2a. What techniques are proposed to reduce the computation, memory, communication, and energy requirements of FL on resource-constrained IoT and TinyML devices, and which resource constraints does each technique explicitly address?

RQ2b. Relative to an explicitly identified baseline, what changes are reported in task performance, model size, memory consumption, computation, communication, latency, and energy for resource-optimization techniques in FL, and under what experimental conditions are those changes measured?

RQ2c. How do the reported effects of FL resource-optimization techniques vary across device tiers?

RQ3 — Resource feasibility and runtime support

RQ3a. What reported measurements establish whether FL workloads fit their target devices, particularly with respect to peak training memory, optimizer state, model and update size, and other execution-resource requirements?

RQ3b. Which MCU-native training and inference runtimes, libraries, and deployment frameworks are used for FL on MCU-class devices, and what role do they play in enabling resource-constrained execution?

RQ4 — Security and privacy

RQ4a. What security and privacy mechanisms are used in FL for resource-constrained IoT and TinyML devices, what security or privacy properties do they provide, and against which threat models or attacker capabilities are they evaluated?

RQ4b. What computational, memory, communication, latency, and energy overheads are introduced by security and privacy mechanisms in FL, and to what extent are these mechanisms demonstrated to be feasible on MCU-class devices?

RQ5 — Benchmarking and reproducibility

RQ5a. To what extent are empirical results in FL for resource-constrained IoT and TinyML devices comparable across studies with respect to datasets, models, evaluation metrics, hardware, communication protocols, and experimental baselines?

RQ5b. What reproducibility artifacts are publicly released by primary FL studies for resource-constrained IoT and TinyML devices?

1.7. The structure of this review

Background on federated aggregation, architectures, model compression and privacy mechanisms is provided as supplementary material. Section 2 describes the systematic review methodology, including the PICOC scope, the device-tier and evaluation-mode classification, the search strategy, and the screening and extraction procedures. Section 3 presents the results organized by research question. Section 4 discusses open challenges and future research directions, Section 5 states the limitations of the review, and Section 6 summarizes the implications.

1.8. Study comparison

Table 1 compares this review with the five secondary studies closest to it [18, 17, 3, 20, 21], and Table 2 extends the comparison to 13 reviews, adding eight of narrower or broader scope, among them surveys of FL for the IoT and a review of machine learning on microcontroller-class hardware.

Both tables rest on a full-text audit of the original 12 reviews against 34 criteria fixed in advance, 11 of them restating this review's research questions; the two reviews added later were audited on the questions of Table 2. Verdicts were LLM-assisted; each Yes or Partial verdict was recorded with a supporting quote, a criterion was recorded as absent only after a logged search of the full text for named terms, and the first author verified 43 of the 408 verdicts for the original 12 reviews, including all of those for the closest [18].

No audited review reports how evaluation hardware is distributed across its included studies, compares the device class a study targets with the class it is evaluated on, or stratifies reported effects by device tier, and none audits the reproducibility artifacts of its included studies; one records whether seven FL frameworks are open source [20]. Two further questions are covered only in part: one review counted the studies evaluated only in simulation, without resolving their device class [21], and four discuss the cost of security mechanisms on constrained devices only qualitatively [22, 23, 7, 20]. The last column of Table 1 gives the evidence this review adds on each.

2. Materials and methods

2.1. Systematic literature review methodology

This review followed a protocol written before the search was run; its version 1.4, archived on Zenodo, records every later amendment and deviation with its date and rationale. The aim was to identify, classify, and synthesize primary research on FL for resource-constrained IoT and TinyML devices, with emphasis on deployment evidence, resource optimization and feasibility, security and privacy overhead, benchmarking, and reproducibility, and to locate the gaps that remain.

The search covered one bibliographic database and six publisher digital libraries:

- Scopus
- IEEE Xplore
- ACM Digital Library
- ScienceDirect
- SpringerLink
- Wiley Online Library
- Taylor & Francis Online

Because Scopus also indexes records hosted on these platforms, duplicates across sources were removed before screening (Section 2.2). The search sought primary studies of FL with a defined model-aggregation step whose target clients are MCU-class (T1) or SBC-class (T2) devices, and that evaluate FL methods, resource optimizations, runtime or deployment approaches, or security and privacy mechanisms they propose or apply for those clients.

2.2. PRISMA guidelines

This review follows the PRISMA 2020 guidelines [19]; Fig. 1 shows the selection process. The seven sources (Section 2.1) were searched on 26–27 August 2026 for studies published from 2019 onward and yielded 6,582 exported records. Duplicates were removed in Rayyan [30] and checked against an independent script, leaving 5,028 unique records.

Table 1
Comparison of the five secondary reviews most closely related to this study and the evidence added by the present review.

	Prior review (full-text audit)			This review
Study	Focus	Key features	Limitations relative to this review's questions	Evidence added (research question)
[18]	Tiny FL (TinyFL) for constrained sensors; systematic review of 30 studies (2020–2024).	Reports its databases, eligibility criteria and study counts; names example MCU and SBC devices (ESP32, Raspberry Pi 4) and cites TensorFlow Lite Micro.	No MCU–SBC boundary; no distribution of evaluation hardware or mode; targeted and evaluated hardware not compared; results not stratified; no quality assessment.	Targeted and evaluated tier and evaluation mode of each of 93 studies: 24 (25.8%) trained on physical MCU-class and 38 (40.9%) on physical SBC-class devices, 12 (12.9%) were evaluated in hybrid hardware-in-the-loop setups and 19 (20.4%) in simulation or emulation (RQ1a). Evaluation matched the targeted tier in 76 (81.7%), and each of the 15 mismatches moved to a more capable tier (RQ1b).
[17]	FL and TinyML on IoT edge devices; review.	Compares FL–TinyML frameworks by supported hardware, memory use and performance; cites MCU runtimes (MCUNet, X-CUBE-AI); proposes and evaluates its own LoRa-based FL framework with Raspberry Pi aggregators and microcontroller clients.	Search and selection not reported; frameworks tabulated, not stratified by device class; security not linked to client-side cost.	Reported changes from 89 studies kept apart by tier and mode, not pooled: evaluated physically or in hybrid setups on MCU-class (24) and SBC-class devices (47), and simulated with an MCU target (7) or an SBC target (11) (RQ2c).
[3]	FL for resource-constrained IoT devices; survey.	Survey of how FL can be trained on constrained IoT devices, with device examples down to 2 kB of RAM; cites CMSIS-NN.	No device classes or thresholds; resource figures quoted per example, not read against device capacity; no hardware or evaluation-mode distribution.	Peak training memory was reported by 26 of 93 studies (15 measured physically or in hybrid setups) and optimizer state by 1; read against the memory of the same device, MCU-class devices had at most 1.60 times the requirement in 4 of 10 studies (RQ3a).
[20]	Decentralized FL and TinyML, introducing swarm learning; survey of 41 studies.	Catalogs MCU and SBC boards with their SRAM and flash, and MCU runtimes such as μTVM and CMSIS-NN; notes that end devices lack memory for security measures.	Boards listed without counts or evaluation mode; effects quoted per study, not by tier; security cost discussed only qualitatively.	Of 29 studies with a security or privacy mechanism, 8 are fully reported (property, threat model and client-side cost), one of them evaluated on an MCU; 4 of the 5 studies that trained on MCUs and reported a mechanism state no cost of it (RQ4).
[21]	Edge machine learning for sustainable IoT networks; PRISMA systematic review of 10 studies (2013–2023).	Reports its PRISMA flow diagram, search strings, search date and quality assessment; admits only hardware-evaluated studies and counts the otherwise eligible studies evaluated only in simulation (96).	Not specific to FL; no MCU–SBC boundary; targeted and evaluated hardware not compared; artifacts not audited.	For FL, the physical-versus-simulated split resolved by device tier (RQ1a), and an audit of artifacts: public code for 18 of 93 studies, random seeds reported by 5 (RQ5b).

Evidence added: results of Section 3.

Title and abstract screening applied the eligibility criteria (Section 2.6) in two parts: the first author screened 1,519 records in Rayyan, and an LLM tool screened the remaining 3,509 with two differently worded prompts that encode the criteria and a blind third that settled their disagreements. The prompts were calibrated on the first author's decisions: wherever the tool disagreed with her, her decision was kept and the prompts were revised to follow it. Code checked every answer for a valid output, a decision consistent with its criterion-level judgments and verbatim quotations from the record, and she approved all automated decisions. Title and abstract screening excluded 3,397 records. Of the 1,631 reports sought, 1,364 were retrieved from open-access sources, through institutional access or as arXiv preprints; the rest were recorded as not retrieved rather than excluded and are listed in the supplementary material, and three included studies were assessed from their preprints.

At full text, an LLM screener extracted the evidence on every criterion from each retrieved report, with a verbatim quotation for each judgment, and gave a verdict; code re-derived the decision from the judgments and verified the quotations, and low-confidence or undetermined decisions went to a blind second screener and, on disagreement, a blind third. The screener was calibrated in the same way on ten full texts screened by the first author, who made the final decisions from the tool's evidence and verdicts.

Both stages were validated by two authors on stratified random samples with the LLM decisions withheld: agreement with the tool was 94.5% (Cohen's $\kappa = 0.85$) on 200 title-and-abstract records and, at full text, 92.5% ($\kappa = 0.73$)

Table 2
Extended comparison of 13 secondary reviews against the coverage addressed by this review.

Reference	Scope	Years covered	Studies	Coverage of the questions this review addresses						
				MCU and SBC classes separated	Physical vs. simulated evaluation	Hardware or mode distribution across studies	Targeted vs. evaluated device class	Effects stratified by device tier	Security cost and MCU feasibility	Artifacts of included studies audited
[18]	Tiny FL for constrained sensors; systematic review	2020–2024	30	Partial	Partial	No	No	No	No	No
[17]	FL and TinyML on IoT edge devices; review	–	–	Partial	No	No	No	No	No	No
[3]	FL for resource-constrained IoT devices; survey	–	–	No	No	No	No	No	No	No
[20]	Decentralized FL and TinyML, with swarm learning; survey	Last five years (relative)	41	Partial	No	No	No	No	Partial	Partial
[21]	Edge machine learning for sustainable IoT networks; systematic review	2013–2023	10	Partial	Yes	Partial	No	No	No	No
[24]	Federated split learning with TinyML in the Internet of Vehicles; review	–	–	No	Partial	No	No	No	No	No
[22]	Blockchain-based secure and private FL in resource-constrained computing; survey	–	–	No	No	No	No	No	Partial	No
[7]	Machine learning on microcontroller-class hardware (not FL-specific); review	–	–	Yes	No	No	No	No	Partial	No
[25]	FL on the IoT; systematic review	–	92	Partial	Partial	No	No	No	No	No
[23]	FL for the IoT: techniques, challenges and applications; survey	–	–	No	No	No	No	No	Partial	No
[26]	FL for the IoT: applications, taxonomy and challenges; review	–	–	No	No	No	No	No	No	No
[27]	The IoT under FL: advances and applications; review	–	–	No	No	No	No	No	No	No
[28]	FL and the Industrial IoT; survey	–	–	No	No	No	No	No	No	No
This review	**FL on resource-constrained IoT and TinyML devices, by device tier; systematic review**	**2019–2026**	**93**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

Verdicts are from the full-text audit; –: not stated in the review; Partial: the topic is present without the analysis, for example discussed qualitatively or counted by evaluation mode but not by device class. The first five rows are the reviews of Table 1. No audited review reports a count or percentage of evaluation hardware across its studies. One further audited survey is not listed because its venue could not be verified; it covers none of the last five questions. A systematic review of FL privacy and security in resource-constrained IoT [29] could not be obtained in full text and was not audited. The two reviews added after the original audit [20, 24] were audited on the questions shown.

on whether a deployment target was identified and 86.7% ($\kappa = 0.70$) on inclusion; after adjudication the screening decision on inclusion held for all 37 reports checked, and the first author confirmed the 15 studies included after the samples were drawn. Appendix Tables A.2 and A.3 give the contingency tables and confidence intervals.

In total, 1,268 reports were excluded, most often because the deployment target was not identified (T4, Section 2.5; $n = 808$); the other reasons are given in Fig. 1. The ten grouped there as other reasons were on-device inference or compression alone (5), a record that was not a peer-reviewed primary study (2), no FL method, optimization or mechanism evaluated by the study itself (1), and a target whose device tier could not be determined together with a missing contribution test (2). After reports of the same study were combined, the 96 remaining reports yielded 93 studies, which were included in the synthesis.

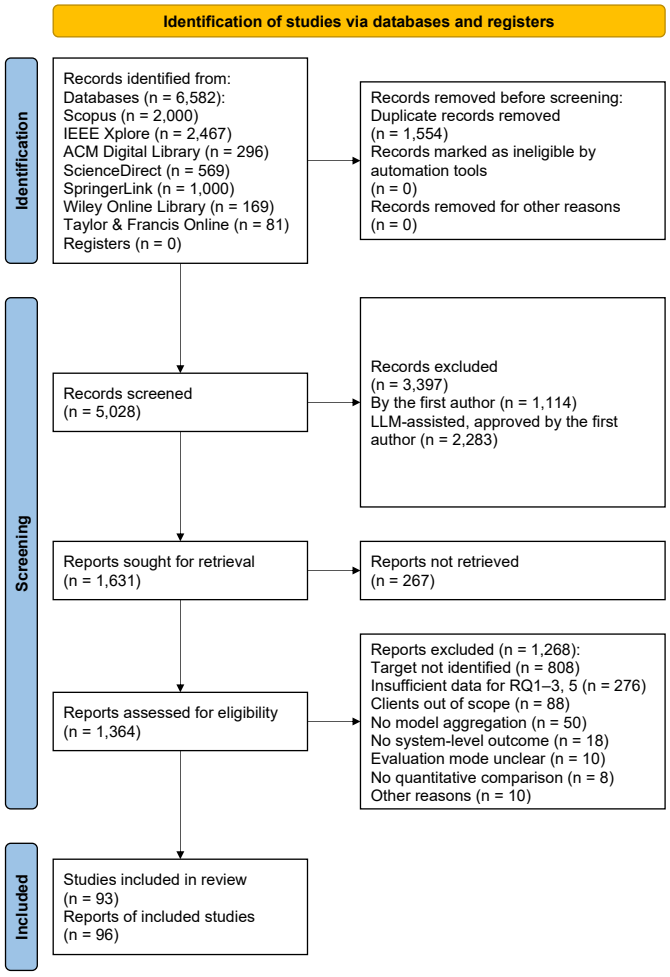


Figure 1: PRISMA 2020 flow of records through identification, screening, full-text eligibility assessment and inclusion.

2.3. Review scope (PICOC)

The review scope was defined with the PICOC framework (Population, Intervention, Comparison, Outcome, Context), summarized in Table 3, and it shaped the research questions (Section 1.6), the search (Section 2.4) and the eligibility criteria (Section 2.6). PICOC was preferred to PICO because its Context element holds the deployment setting, the device tier and evaluation mode, against which every result in this review is read. Intervention, Population and Context formed the search blocks, whereas Comparison and Outcome, which titles and abstracts often omit, were applied at full text.

2.4. Sources searched and search strings

The seven sources (Section 2.1) were searched on 26–27 August 2026. The search expression follows the PICOC scope (Section 2.3) and combines four term blocks: federated-learning terms (FL) with either device classes that are resource-constrained by definition (DEV) or general edge and embedded terms (EDGE) qualified by resource-constraint terms (RES). Table 4 summarizes how each platform applied the blocks and the records exported from it, and Appendix Table A.1 gives the complete strings. The five platforms searched through their search forms used shorter term lists; ScienceDirect, which limits the number of Boolean operators per query, was searched with five queries. Publication years from 2019 onward were applied as a filter in Scopus and at screening for all sources. Federated learning was introduced in 2016 [1], and the earliest peer-reviewed studies running federated training on T1 hardware appeared from 2021 [31, 12], so the 2019 start precedes the earliest physically deployed T1 evidence by two years. The Scopus and SpringerLink exports were capped at 2,000 and 1,000 records, respectively; export details and deviations from the planned supplementary searches are documented on Zenodo.

2.5. Device-tier and evaluation-mode classification

The hardware each study targets and the hardware on which it was evaluated were classified separately, into the four tiers of Table 5. Architecture decides the tier, not terms such as TinyML, IoT or edge: a memory management unit (MMU), external DRAM or a general-purpose operating system places a platform above T1, and among such platforms a desktop or server architecture or operating system gives T3, otherwise T2. The numeric ranges in Table 5, compiled from device specifications reported in peer-reviewed benchmarks and reviews [32, 33, 34, 35, 36, 37, 38], only corroborate this test and never override it; for a named device, the manufacturer's datasheet was used. T4 records that the reported evidence cannot support a tier; it is not a hardware class and never stands for T1. A T4 target excludes a study at full text, whereas T4 evaluation hardware does not. A study's evaluated tier is that of the lowest-resource platform on which FL training actually ran, and server-side aggregation does not raise a client's tier. Evaluation mode was recorded separately as E1 (physical deployment), E2 (hybrid or hardware-in-the-loop), E3 (emulation), E4 (simulation) or E5 (unspecified), so MCU-class physical validation is T1 with E1. The full decision rules and boundary cases are given in the review protocol.

2.6. Inclusion and exclusion criteria

2.6.1. Inclusion criteria

A study was included only if it met all of the following:

- **IC1.** A peer-reviewed primary study (journal or conference paper, early access included) in English, published from January 2019 to the search date.
- **IC2.** Federated learning across at least two clients with a defined model-aggregation step, including split, hierarchical, clustered, decentralized, swarm and gossip variants.
- **IC3.** Clients fielded as IoT endpoints or gateways, with an identified T1 or T2 target (Section 2.5) or a client mix including at least one T1 or T2 class; without a T1 class, the study must state a memory, energy, computation or bandwidth constraint and propose a mechanism for it.
- **IC4.** An FL method, resource optimization, runtime or deployment approach, or security or privacy mechanism

Table 3
PICOC scope used to define the review population, interventions, comparisons, outcomes and deployment context.

PICOC element	Definition in this review
Population (P)	Primary FL studies (at least two clients and a defined model-aggregation step, variants included) whose identified target clients are MCU-class (T1) or SBC-class (T2) devices fielded as IoT endpoints or gateways; the evaluation hardware may be of any tier.
Intervention (I)	FL methods, resource-optimization techniques, runtime and deployment approaches, and security or privacy mechanisms for T1 or T2 clients, proposed or applied and evaluated by the study itself.
Comparison (C)	A quantitative comparison with an explicitly identified baseline: a named FL method or alternative mechanism, centralized or local training, or the study's own unoptimized or ablated variant.
Outcome (O)	System-level outcomes (model or update size, memory, computation, latency, communication and energy) and resource fit, together with task performance, security and privacy properties, deployment evidence, benchmark comparability and reproducibility.
Context (C)	T1 and T2 deployments, with each result read against the device tier, evaluation mode, dataset, model and experimental protocol.

Table 4
Databases and publisher libraries searched, search logic and records retrieved (searched 26–27 August 2026; publications from 2019 onward).

Source	Fields	Search logic	Records
Scopus	Title, abstract, keywords	FL AND (DEV OR (EDGE AND RES)); articles and conference papers in English, 2019 onward	2,000*
IEEE Xplore	All metadata	FL AND (DEV OR (EDGE AND RES))	2,467
ACM Digital Library	FL′ in abstract; others anywhere	FL′ AND (DEV′ OR EDGE′) AND RES′	296
ScienceDirect	Not field-restricted	Five queries, each FL′ AND three DEV, EDGE or RES terms	569
SpringerLink	Not field-restricted	FL′ AND (DEV′ OR EDGE′)	1,000*
Wiley Online Library	FL′ in abstract; others anywhere	FL′ AND (DEV′ OR EDGE′) AND RES′	169
Taylor & Francis Online	Anywhere	FL′ AND (DEV′ OR EDGE′) AND RES′	81
Total			**6,582**

FL: federated learning; federated machine, deep, transfer, edge or meta-learning; embedded or on-device federated learning; TinyFL; tiny federated learning; swarm learning; gossip learning. DEV: microcontroller*, MCU, TinyML, tiny machine learning, sensor node*, wireless sensor network*, WSN, ESP32, STM32, Cortex-M*, nRF52*, nRF53*. EDGE: IoT, Internet of Things, edge device*, edge computing, edge AI, edge intelligence, embedded system* or device*, embedded AI, single-board computer*, Raspberry Pi, Jetson, on-device (machine) learning. RES: resource-constrained or resource-limited; low-resource; low-power or ultra-low-power; memory-, energy-, compute- or computation-constrained; embedded machine learning; on-device training. A prime (′) marks a subset of the block. *Export limit of the platform reached; Scopus returned 2,002 records, 2,000 of which could be exported.

for those clients, proposed or applied and evaluated by the study itself.

- **IC5.** An identifiable evaluation mode (E1–E4).
- **IC6.** At least one system-level outcome of the study's own: model or update size, memory, computation, latency, communication or energy.
- **IC7.** A quantitative comparison with an explicitly identified baseline: a named method, centralized or local training, or the study's own unoptimized or ablated variant.
- **IC8.** Sufficient data for RQ1, RQ2, RQ3 and RQ5; RQ4 evidence was recorded but not required.

2.6.2. Exclusion criteria

Studies were excluded for any of the following:

- **EC1.** Reviews, surveys, books, edited-volume chapters, preprints and other non-peer-reviewed work, theses, technical reports, abstract-only records and retracted work; conference papers in series such as LNCS remain eligible.
- **EC2.** Centralized or single-device learning, and distributed learning without a defined aggregation step, such as message-only gossip, split learning without federated aggregation, or datacenter data-parallel SGD.
- **EC3.** A deployment target that is not identified (T4); server, workstation or cloud clients (T3), including cross-silo FL between institutions; smartphones or tablets as the only clients (excluded by role, not tier; mixed client populations with SBC- or MCU-class devices remain eligible); or a T2 target without a stated and addressed resource constraint.
- **EC4.** FL only as motivation or future work, on-device inference or compression alone, no evaluation of the study's own, or a security or privacy mechanism as the only intervention without a reported client-side cost.
- **EC5.** Failing IC5–IC8: an evaluation mode that cannot be established (E5), task-performance or privacy outcomes only, no quantitative comparison with an explicitly identified baseline, or insufficient data for RQ1, RQ2, RQ3 or RQ5.

Reports of the same study were counted once. Title and abstract screening applied the criteria as far as abstracts allow, and IC5–IC8 were applied at full text. Excluding T4

Table 5
Rule-based device-tier classification used throughout the review: the decisive architecture test determines the tier, and the typical resource envelope only corroborates it.

Tier	Class	Decisive test	Typical envelope	Examples
T1	MCU-class	MCU architecture with no MMU, external DRAM or general-purpose OS; bare-metal or RTOS	≤2 MB on-chip SRAM; ≤2 MB integrated flash; ≤600 MHz; ≤0.5 W active	STM32, nRF52, nRF53, ESP32, Cortex-M
T2	Application processor or single-board computer (SBC)	An MMU, external DRAM or a general-purpose OS, without a desktop or server architecture or OS	External DRAM, typically 512 MB–8 GB (T2-L) and up to 64 GB (T2-H); typically 1.0–2.0 GHz	Raspberry Pi, BeagleBone, Jetson
T3	Server, workstation or cloud	A desktop or server CPU architecture, or a desktop or server OS	≥4 GB DRAM; mains-powered	x86 workstations and servers, GPU servers, cloud instances
T4	Insufficiently reported	Reported evidence cannot support T1–T3; an evidence status, never T1	—	"IoT device" or "edge device" without specifications

T2 sub-tiers: T2-H carries an ML accelerator of at least 0.5 TOPS (INT8) or 100 GFLOPS (FP16) with a vendor runtime, or, where no throughput is published, an integrated ML accelerator with a vendor inference runtime; T2-L otherwise; T2-U when this cannot be determined. Board-level external flash, PSRAM attached to an MCU, core count, MCU-integrated NPUs and power never change the tier. A cloud instance is classified by the compute it provides.

Table 6
Quality-assessment domains and scoring criteria; Q2–Q4 also define the risk-of-bias judgments (Yes: low; Partial: some concerns; No: high).

Domain	Question	Yes (1)	Partial (0.5)	No (0)
Q1 Reporting of the FL setup	Is the federated experiment reported fully enough to interpret and repeat? Items: clients (and clients per round), rounds and local epochs, learning rate and batch size, and the data partition.	All four items reported.	One item missing.	Two or more items missing.
Q2 Hardware and evaluation validity	Does the evaluation support the study's claims about its target? Judged by where the clients' local training physically runs.	A client trains alone on its own physical device of the target tier, or on a T1 device for a T2 target.	A T2 device for a T1 target; several clients on one T1 or T2 device; or T3 hosts with client behavior measured on, parameterized from, or capped to an identified T1 or T2 device.	Training only on T3 or unclassifiable hosts, without such calibration.
Q3 Comparator validity	Does the comparison isolate the effect of the intervention?	The reference is run by the study with the same dataset, model, settings and client configuration.	Same dataset and model, but another setting differs or is not stated; or figures quoted for the same dataset, model and metric.	A different dataset, model or metric.
Q4 Measurement and outcome reporting	Are the system-level outcomes behind the claim measured and reported reliably, without selective reporting?	Each outcome is measured on the hardware that ran it, with the instrument or procedure and, where values vary, repetitions; or computed exactly, with the computation stated.	At least one outcome measured or computed exactly, but not all meet Yes; also the maximum when task performance for the same configuration is not reported.	Only analytical or proxy estimates, or figures of unstated basis.
Q5 Reproducibility	Can the principal results be reproduced from what is released and reported?	R3 or R4: public FL training code, an obtainable dataset, and complete hardware and configuration details.	R2; or R0 or R1 with public FL training code.	R0 or R1 without public FL training code.

targets and E5 modes, and requiring IC7 and IC8, are stricter than the original protocol and are reported as deviations.

2.7. Record extraction

Each included study has two records: the full-text screening record, which decided eligibility, and the extraction record, which feeds the synthesis. The screening record gives, with a supporting quote for each item, the study's target tier (the lowest-resource tier among the client classes it names as its deployment target), its evaluated tier and its

evaluation mode (Section 2.5), together with five contribution tests, C1–C5, one per research question, each recorded as DIRECT, MISSING or UNCLEAR. C1 (RQ1) is DIRECT when the study fits an RQ1a category and the correspondence between its target and evaluated tiers can be determined; C2 (RQ2) when a technique aimed at a named constraint (computation, memory, communication or energy) shows a quantitative change against an explicitly identified baseline under stated conditions; C3 (RQ3) when a measured or exactly computed resource figure is read against the capacity or stated limit of the same device, or a named MCU-native runtime ran FL training or inference of the federated model on an identified T1 device; C4 (RQ4) when a named security or privacy mechanism is reported with its property, its threat model and at least one client-side cost, measured against FL without it or reported as negligible; and C5 (RQ5) when the study reports its dataset, model, evaluation metrics, hardware and baseline. IC8 required C1, C2, C3 and C5 to be DIRECT; C4 was recorded but not required, and RQ4 rests on the included studies whose C4 is DIRECT (Section 3.4.1). During the quality assessment (Section 2.8), each included study was re-screened blind against the same criteria to check these records.

Data were extracted per study, with linked records for each device, intervention, security mechanism and reproducibility item. Field groups follow the research questions: target and evaluated hardware and evaluation mode (RQ1); optimization techniques, the constraints they address and their effects against an explicitly identified baseline (RQ2); resource-fit figures and MCU-native runtimes (RQ3); security and privacy mechanisms with their property, threat model and client-side overhead (RQ4); and datasets, models, metrics, hardware, baselines, communication protocols and released artifacts (RQ5), together with non-IID partitioning, client dropout and stragglers. A study can inform several RQs, and one using several optimization techniques was coded under each, with a primary family for counts that include each study once; the categories therefore overlap, and per-RQ counts sum to more than the number of included studies.

Values were recorded as reported, with units and conditions, never converted or derived, and each resource figure was labeled as measured, estimated, exact, configured or of unstated basis; model and update size, training and inference memory, per-round and total communication, and energy by component and phase were kept separate, and missing items were coded as not reported, not applicable, unclear or not determinable. Extraction was LLM-assisted (Claude, Anthropic): a fixed, protocol-derived prompt recorded each value with a supporting quote, which code checked against the source, and a second LLM extraction of the 47 key fields, run blind to the first for every included study, agreed with it on 93.5% of values; a third LLM run adjudicated the disagreements. The authors approved the LLM's final answers; verification details and supporting quotes are archived on Zenodo.

2.8. Quality assessment

In this review, quality assessment does not decide inclusion, which IC1–IC8 already settle (Section 2.6); it grades how far each included study's evidence can be trusted, so that every synthesized statement can be read against the reliability of the studies behind it and checked for whether it holds without the weaker ones. Five domains, Q1–Q5 (Table 6), were each rated Yes (1), Partial (0.5) or No (0), giving a score from 0 to 5; each domain was judged on the experiment behind the study's principal resource claim, and unreported information counted as absent. Venue rank, which reflects where a study was published rather than how it was conducted, was not scored. Table 7 shows the assessment of three included studies.

Q2–Q4 also give each study's risk of bias on the RoB 2 judgment levels [42], covering deviations from the intended intervention (Q2), confounding (Q3), and outcome measurement and selective reporting (Q4): Yes is low risk, Partial some concerns and No high risk, and the overall risk is low when all three are low, high when any is high, and some concerns otherwise. Q1 and Q5 concern reporting and reproducibility; the reproducibility level behind Q5 (R0–R4, defined in the protocol and glossed in Section 3.5.2) was assigned from the reproducibility items as coded by the assessment, with code counted as public only when the link printed in the report held the training code the paper describes, and a dataset counted as obtainable when it is a named public benchmark, its printed link held it, or it is offered on restricted terms.

Assessment was LLM-assisted (Claude, Anthropic): blind to the screening records and to other assessments, an agent applied the rubric, revised after a five-study pilot and frozen before the main run, to every report of a study, citing a verbatim quote for each judgment; quotes and scores were checked in code, and the first author decided failed quotes and Q2 judgments conflicting with the recorded tier or mode. Quality therefore neither excludes nor weights studies. Because conclusions drawn from quality scores can depend on the scale used [43], each score is reported with its domain profile (Appendix Table A.4), and the principal quantitative statement of RQ2b, RQ2c, RQ3a and RQ4b is reported with how many contributing studies were rated Yes on Q2 and how many on Q4, and a sensitivity analysis fixed in advance recomputes it on the studies scoring at least 3.5 of 5 (Section 3.6). The full assessment records are archived on Zenodo.

Table 7
Quality assessment of three included studies chosen to span the scores; each cell shows the domain judgment and its basis in the assessment record.

Study	Target	Evaluation	Q1 FL setup	Q2 Hardware	Q3 Comparator	Q4 Measurement	Q5 Reproducibility	Score
[39]	T1, Arduino Portenta H7	E1: three boards; a PC aggregates	**P** learning rate not reported	**Y** each client trains alone on its own T1 board	**Y** own 32-bit float model on the same boards, data and rounds	**Y** transfer size computed exactly from the parameter count	**Y** R3: public code and dataset	**4.5**
[40]	T2-L, Raspberry Pi Zero W +2	E1: ten clients on Raspberry Pi and Jetson boards	**N** rounds, learning rate and batch size not reported	**Y** each client trains on its own SBC	**Y** five reference methods run under identical settings	**Y** energy measured with a power monitor; ten runs	**P** R2: no code; public benchmark datasets	**3.5**
[41]	T1, no device named	E4: Flower simulation on a desktop host	**P** data partition not reported	**N** training only on the host, without calibration to a T1 device	**P** baseline figures quoted from other papers	**P** size computed exactly; energy estimated	**N** R1: the printed repository holds only a dataset script	**1.5**

The second study lies at the sensitivity threshold. The overall risk of bias is low for the first two studies and high for the third; the second loses its points only on reporting (Q1) and reproducibility (Q5). Appendix Table A.4 gives all 93 studies. Y: Yes (1); P: Partial (0.5); N: No (0). E1: physical deployment; E4: simulation; +2: two further devices. R1–R3: reproducibility levels (Section 3.5.2).

3. Results and discussion

3.1. RQ1: Deployment evidence

3.1.1. *RQ1a. What device tiers and evaluation modes are used to validate FL methods targeting resource-constrained IoT and TinyML devices, and how is the evidence distributed across physical deployment on MCU-class devices, evaluation on SBC-class devices, hybrid hardware-in-the-loop evaluation, simulation or emulation, and cases where hardware is insufficiently specified?*

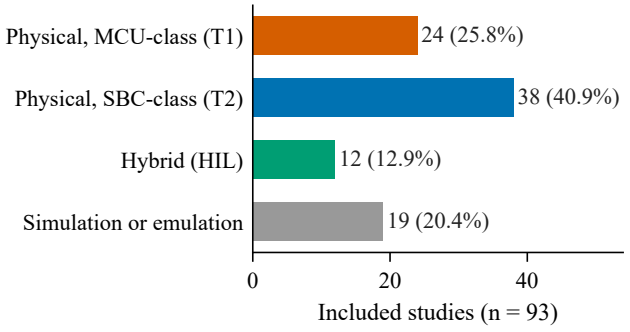


Figure 2: Distribution of the 93 included studies across the RQ1a evaluation categories, by evaluated tier and evaluation mode.

Of the 93 included studies, 35 (37.6%) target MCU-class (T1) devices and 58 (62.4%) SBC-class (T2) devices. Fig. 2 places each study by the platform on which its FL training actually ran, so a study whose physical ESP32 sensor nodes only ran inference while a Raspberry Pi 5 gateway trained their models counts as SBC-class [44]. Most studies (62, 66.7%) trained on physical devices, 24 (25.8%) on MCU-class and 38 (40.9%) on SBC-class devices; 12 (12.9%) evaluated physical devices with some components simulated or executed elsewhere (hybrid hardware-in-the-loop), and 19 (20.4%) relied on simulation or emulation alone. No included study trained physically on server-class hardware or left its evaluation mode undetermined (E5), and one, a simulation whose host is not described [45], has an evaluated tier too vaguely reported to classify (T4); how often unclassifiable hardware and an undetermined mode occur in the screened literature is discussed below.

Fig. 3 resolves these categories by tier and mode. Physical SBC-class evaluation is concentrated on lower-resource boards (T2-L, 26 of the 38 studies), with 11 on accelerator-equipped T2-H platforms such as the NVIDIA Jetson Nano [46]; a federation that mixed Raspberry Pi and Jetson devices is counted as T2-L because its lowest-resource clients were Raspberry Pi 4 boards [47]. Of the 19 studies without physical clients, 13 simulated the federation on a server or PC, in some cases timing inference alone on a Raspberry Pi 4 [48, 49]; only 2 used emulation. Hybrid evaluation occurred mainly on SBCs (10 studies); on MCU-class hardware, one study streamed pre-recorded audio from a host computer to five ESP32 nodes that updated their classifiers on-device [50], and the other trained on 18 Raspberry Pi Pico boards and extended the network in a calibrated ns-3 simulation [51].

Deployments used few devices. Of the 74 studies evaluated physically or in hybrid setups (modes E1 and E2), 63 reported how many they used: a median of 4 (interquartile range 3–10; range 1–100). Physical MCU-class deployments (E1) had a median of 3.5 boards: 11 of the 22 that reported

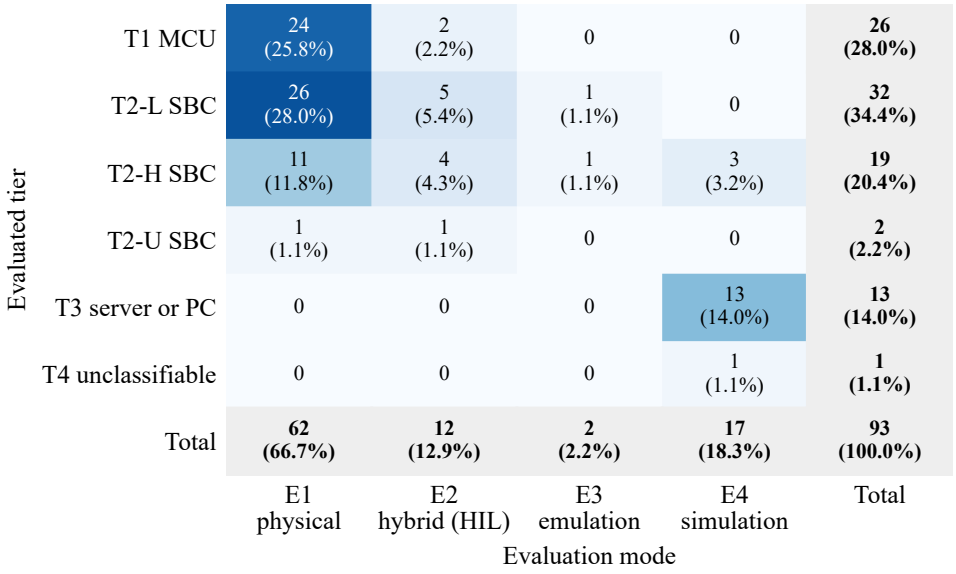


Figure 3: Distribution of the 93 included studies by evaluated tier and evaluation mode; each cell gives the study count and the percentage of all included studies.

a count used three or fewer [12, 52], and only 6 used ten or more, the largest being 100 ESP32-S3 nodes [53] and 50 nRF52840 boards [54]. SBC-class deployments were somewhat larger (median 6; up to 80 devices). In addition, 40 studies were evaluated on more than one tier, most often an SBC alongside a server or PC (15); 19 of these combined MCU-class devices with another tier, such as ESP32 clients training in the same federation as a Raspberry Pi 4 client [55], and 13 included a secondary platform whose hardware could not be classified, such as GPU clients and CPU-only clients whose device model is not given [56], or simulated clients whose host machine is not described [57, 51].

Hardware was seldom specified completely. Only 14 of the 93 studies (15.1%) give enough detail to identify their devices and configuration. Thirteen of these 14 were evaluated physically or in hybrid setups (E1 or E2), 6 of them training on MCU-class hardware, such as ten ESP32-CAM boards [58], and 7 on SBC-class hardware, such as four Raspberry Pi 4 boards [59]; the remaining one was a simulation.

Table 8 lists the 26 studies whose FL training ran on MCU-class hardware: the 24 physical deployments and the two hybrid studies above. ESP32-family modules were the most common platform (11), followed by Arduino boards (6) and STM32 microcontrollers (4); 6 of the 26 specified their hardware completely.

Discussion of RQ1a Ordered by how directly they demonstrate deployment, from physical training on the targeted class of device to simulation, the evidence thins toward the devices the field aims to serve: physical evaluation is mostly on SBCs, and FL training on MCU-class devices is less frequent, uses few boards and is seldom fully specified.

Screening removed insufficiently specified hardware. Of the 1,268 full-text exclusions, 808 (63.7%), the most frequent reason, did not identify their deployment target (T4) and 10 their evaluation mode (E5); reports evaluated on unclassifiable hardware were eligible (Section 2.6), but none reported enough for every research question. Requiring a quantitative comparison removed little deployment evidence: 8 reports, 6 of them physical deployments and 2 on MCU-class hardware.

The included studies are therefore not representative of the eligible literature (Fig. 4). In the 276 eligible reports excluded for insufficient data, physical MCU-class training appears in 5.1% against 25.8% of the included studies and simulation or emulation in 44.2% against 20.4%, and 72 (26.1%) ran on unclassifiable hardware, 65 of them simulations. The shares in Fig. 2 thus describe the best-documented part of the field, not the whole.

In answer to RQ1a, most included studies (62 of 93) trained on physical devices, 38 of them on SBCs; FL training on MCU-class devices was demonstrated in 26 studies, typically on three or four boards and fully specified in 6, and evaluation hardware too vaguely specified to classify appears in one included study, a simulation, against 72 of the eligible reports excluded for insufficient data. Claims that an FL method suits MCU-class devices should therefore rest on training on such devices, which a minority of studies provide; a few boards show feasibility but not behavior at scale, for which the 6 physical MCU-class studies with ten or more boards, and a hybrid study that extended its Raspberry Pi Pico testbed through a calibrated network simulation [51], are the main evidence; and incomplete hardware reporting limits both how evidence can be classified and whether it can be reproduced (RQ5b). RQ1b examines whether the evaluated hardware matches the targeted hardware.

Table 8
Included studies with FL training on MCU-class hardware (evaluated tier T1; $n = 26$); entries are abridged, with the full extraction values retained in the review's data file.

Study	Device	Nodes	Link	Dataset	Aggregation	Mode
[12]	Arduino Nano 33 BLE Sense	3	USB serial port	Authors' spoken-keyword dataset...	FedAvg	E1
[60]	Arduino Portenta H7	3	UART to an external TTGO LORA32...	Keyword spotting speech dataset...	Weighted average of own and peer...	E1
[61]	Arduino Nano 33 BLE Sense	–	–	Sine-wave example	Reptile-style server update	E1
[52]	STM32F446	3	I^2C +1	Gesture recognition dataset	FedAvg	E1
[54]	nRF52840	50	Bluetooth	Heart attack +12	Server aggregation of client...	E1
[62]	Arduino MKR1010	3	Wi-Fi	ECG heartbeat dataset +1	Unweighted averaging of node...	E1
[39]	Arduino Portenta H7	3	Serial port to the PC for FL... +1	Custom keyword spotting speech...	FedAvg	E1
[63]	nRF52840	5	–	Iris +3	Weighted averaging by local...	E1
[64]	ESP32-CAM module	10	NB-IoT via cellular module +1	Custom dial-meter digit image...	Standard parameter averaging	E1
[65]	Thunderboard Sense 2	9	IPv6 over Bluetooth Low Energy	WISDM	Weighted averaging of client...	E1
[66]	ESP32	4	Wi-Fi	TON IoT	FedAvg	E1
[57]	ESP32	5	–	CSE-CIC-IDS2018	Sample-weighted server-side...	E1
[67]	STM32F746G-DISCO	30	–	PhysioNet Challenge 2016	SQI-adaptive FedProx	E1
[53]	ESP32-S3	100	–	PecanStreet +4	DP-FedYogi	E1
[68]	ESP32	–	–	Unnamed physiological sensor...	FedAvg	E1
[69]	Arduino Nano BLE 33	1	–	Sine-wave	Reptile-style server update	E1
[58]	ESP32-CAM	10	Wi-Fi	MNIST +2	FedAvg	E1
[70]	STM32F303	10	UART +1	UCI HAR	fBind bound updates aggregator	E1
[71]	STM32H750IBK6	2	Serial port	PLAID +4	Analytical closed-form aggregation	E1
[72]	ESP32-S3	2	Wi-Fi +1	–	Modality-aware aggregation	E1
[55]	ESP32 DevKitC	4	Wi-Fi	Own testbed sensor time series	FedAvg	E1
[73]	ESP32-S3	3	Wi-Fi	CIFAR-10	Layer-wise dynamic aggregation	E1
[74]	ESP32-S3	3	LoRaWAN	M1 Manitou multifunction vehicle... +2	FedAvg	E1
[75]	Raspberry Pi Pico W	2	Wi-Fi +1	Temperature values in Celsius...	Centralized aggregation of...	E1
[51]	RP2040	18	LoRa RFM95 +1	Edge-IIoTset +1	Reputation-weighted aggregation...	E2
[50]	ESP32	5	Wi-Fi +1	SONYC +1	Weighted averaging based on FedAvg	E2

Physical deployments (E1) first, then the hybrid studies (E2). Entries give the first listed item, +k further items. Nodes: physical devices used; –: not reported.

3.1.2. RQ1b. To what extent does the hardware on which FL methods are actually evaluated correspond to the device tier those methods claim or are designed to target?

Of the 93 included studies, 76 (81.7%) were evaluated on the device tier they target, 15 (16.1%) on a different tier, in 1 (1.1%) the sub-tier of the evaluated T2 device, a smartphone, could not be confirmed, and in 1 (1.1%) the evaluated tier could not be classified because the simulation host is not described [45] (Fig. 5). Every mismatch moved to a more capable tier: 13 of the 15 simulated the federation on a server or PC, and two studies whose targets included ESP32 microcontrollers were evaluated on Raspberry Pi boards, one in a hybrid setup [76] and one physically, with training moved from the ESP32 sensor nodes to a Raspberry Pi 5 gateway [44]. No study targeting SBC-class devices was evaluated on MCU-class hardware.

What a match rests on also differs by target (Fig. 6). All 26 matched MCU-targeting studies (74.3% of 35) trained on MCU-class hardware (24 physical deployments, such as

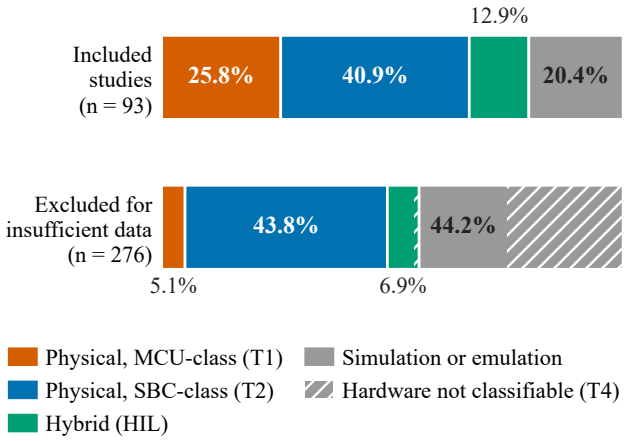


Figure 4: Distribution of the RQ1a evaluation categories among the 93 included studies and the 276 eligible reports excluded for insufficient data; hatching marks evaluation hardware that could not be classified (T4) among the excluded reports; the one included T4 study (1.1%, a simulation) is not marked.

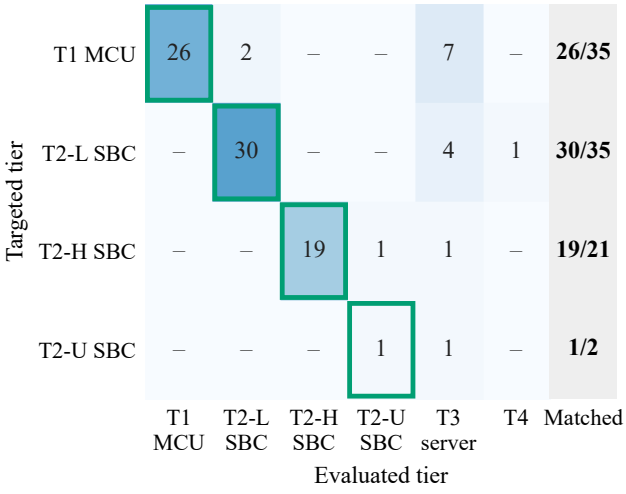


Figure 5: Cross-tabulation of targeted and evaluated tiers for the 93 included studies; outlined cells are matches, and the final column gives the matched studies of each targeted tier.

Thunderboard Sense 2 clients [65], and 2 hybrid setups, one pairing Raspberry Pi Pico nodes with a calibrated ns-3 simulation [51]), whereas 5 of the 50 matched SBC-targeting studies ran a simulated or emulated federation on a device of the targeted tier, so that 45 of 58 (77.6%) trained on hardware of the targeted tier, physically (36) or in hybrid setups (9). Among the 35 MCU-targeting studies, 7 (20.0%) were evaluated only in simulation on a server or PC and 2 on an SBC; among the 58 SBC-targeting studies, 6 (10.3%) were simulated on a server or PC and one on a host that is not described.

The correspondence determines which claims the evidence can carry (Fig. 7). In 71 of the 76 matched studies, the reported resource figures establish that the FL workload

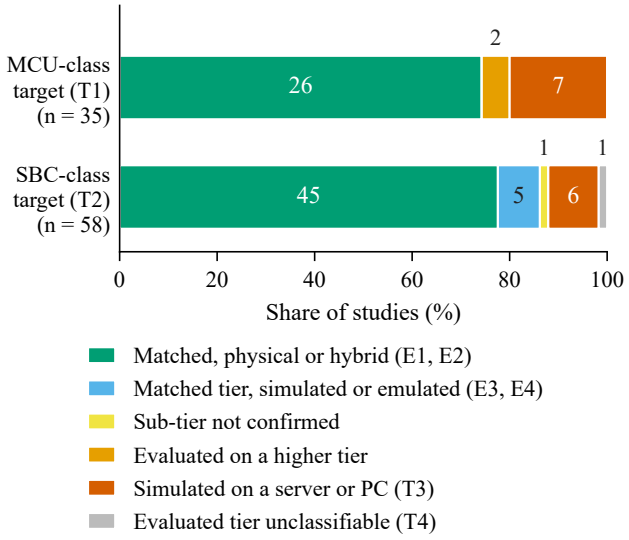


Figure 6: Correspondence between targeted and evaluated tiers for the 93 included studies, separating matches evaluated physically or in hybrid setups (E1, E2) from matches simulated or emulated on a device of the targeted tier (E3, E4), from evaluation on another tier, from an evaluated sub-tier that could not be confirmed and from an evaluated tier that could not be classified (T4); share of studies, with counts in the bars.

fits the target device, with or without optimization, against 6 of the 17 that were not matched; for 10 of these 17 the fit could not be determined, and in 1 the reported figures exceed the capacity of the targeted Cortex-M4 node [77]. Energy was measured, alone or with an estimate, in 23 of the matched studies (30.3%) and in 4 of the 17 others; 4 of those 17 relied on energy estimates alone. Among the matched MCU-class studies, for example, one measured energy with a current sensor and extrapolated battery life from the per-round figure [51]. Another explains that it could not measure energy on its boards in practice and instead used communication and computation costs as a stand-in [65].

Table 9 lists the 17 studies not matched to their targeted tier: the 15 mismatches, the study whose sub-tier could not be confirmed and the study whose evaluated tier could not be classified. Nine of them target MCU-class devices, 6 naming a specific device such as an ESP32-S3 [78] or an STM32L4 [77], yet none of the 9 ran its federated training on one.

Discussion of RQ1b Correspondence could be assessed for all but one included study because the eligibility rules required an identified target and, as RQ1a showed, only one included study, a simulation, was evaluated on hardware too vaguely reported to classify. The 808 reports excluded because their deployment target could not be identified (63.7% of the 1,268 full-text exclusions) cannot be assessed at all, which makes an unnamed target the main obstacle to answering RQ1b for this literature.

The included set also overstates the correspondence found in the eligible literature. Among the 276 eligible

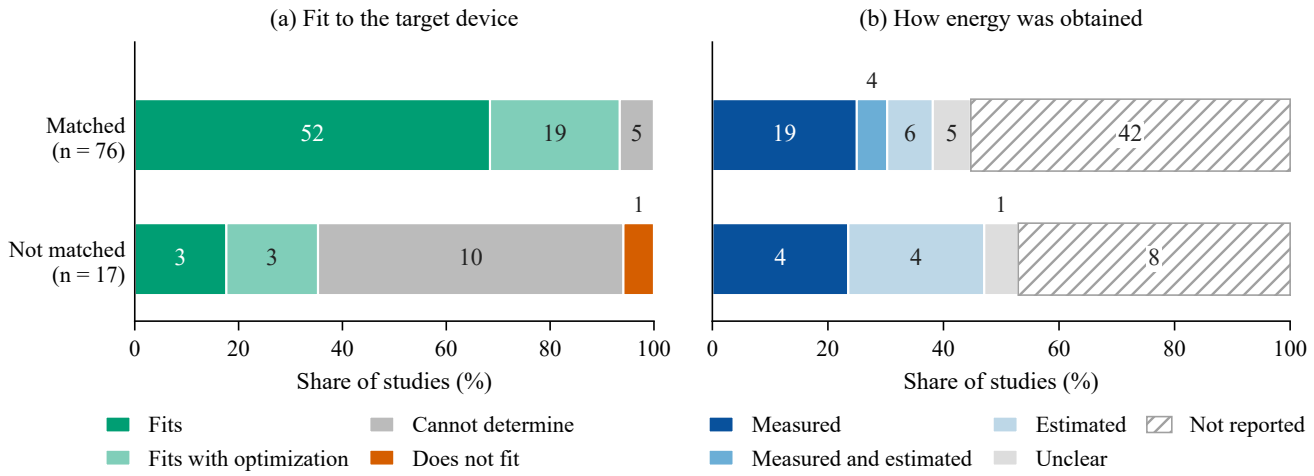


Figure 7: Fit and energy evidence for studies evaluated on the targeted tier (matched) and on another tier (not matched). (a) Fit: whether the reported resource figures establish that the workload fits the device; (b) energy: how the energy evidence was obtained. Share of studies, with counts in the bars.

Table 9
Included studies not matched to their targeted tier (15 mismatches, 1 unconfirmed sub-tier and 1 unclassifiable evaluated tier; n = 17), with the extracted fit and energy evidence.

	Target		Evaluation			Evidence	
Study	Tier	Device	Tier	Platform	Mode	Fit	Energy
[48]	T1	ESP32	T3	Intel Xeon E5-2620 v4 server	E4	Cannot determine	Estimated
[78]	T1	ESP32-S3	T3	Simulation host PC: 11th Gen Intel...	E4	Fits	–
[77]	T1	ARM Cortex-M4 node +1	T3	NVIDIA A100 GPU server	E4	Does not fit	Estimated
[76]	T1	ESP32 +1	T2-L	Raspberry Pi 4B	E2	Fits with optimization	–
[41]	T1	–	T3	Simulation host	E4	Cannot determine	Estimated
[49]	T1	ESP32 +1	T3	FL simulation host: system with...	E4	Cannot determine	Unclear
[79]	T1	Microcontroller-class IoRT robot...	T3	Intel Core i7 system	E4	Cannot determine	–
[80]	T1	–	T3	Kaggle workstation equipped with a...	E4	Cannot determine	Estimated
[44]	T1	ESP32-WROOM-32	T2-L	Raspberry Pi 5 fog gateway with Hailo...	E1	Fits with optimization	Measured
[81]	T2-H	Samsung A32	T2-U	Samsung A32 smartphone	E1	Fits	Measured
[82]	T2-H	Jetson Nano +1	T3	Virtual machine instance	E4	Cannot determine	–
[83]	T2-L	Raspberry Pi 4 Model B	T3	Simulation host with an NVIDIA...	E4	Cannot determine	–
[84]	T2-L	Raspberry Pi 5	T3	Argonne Swing cluster node	E4	Cannot determine	–
[85]	T2-L	Raspberry Pi 4 +1	T3	Raspberry Pi 4	E4	Fits	Measured
[45]	T2-L	Raspberry Pi 4 Model B	T4	Raspberry Pi 4 Model B	E4	Cannot determine	Measured
[86]	T2-L	Raspberry Pi 4B	T3	High-performance computing node with...	E4	Fits with optimization	–
[87]	T2-U	KV260	T3	Supermicro server with 8 NVIDIA RTX...	E4	Cannot determine	–

Target device as named by the study (+k further devices); entries are abridged, and the full values are in the review's data file. Fit and energy as in Fig. 7; –: none named or not reported; E1: physical deployment; E2: hybrid; E4: simulation; T4: evaluated tier not classifiable.

reports excluded for insufficient data for RQ1, RQ2, RQ3 or RQ5, classified from the full-text screening record, the evaluated hardware could not be classified in 72 (26.1%); of the 204 that could be assessed, 145 (71.1%) matched their target, 7 differed only in SBC sub-tier, and 47 of the 52 mismatches were evaluated in simulation or emulation on a server or PC. The gap is widest for MCU-class targets: 16 of the 66 such reports (24.2%) were evaluated on MCU-class hardware, against 26 of 35 (74.3%) included studies; 33 of the 66 had hardware that could not be classified, 32 of them simulations, so 16 of the 33 that could be classified (48.5%) trained on MCU-class hardware.

A mismatch limits the claims a study can support rather than invalidating it. A federation simulated on a server can establish convergence, accuracy and communication volume under the simulated conditions, but not whether

training fits within the memory, time or energy budget of the targeted device; the 3 server-simulated studies coded as fitting drew that evidence from separate runs on target-class hardware, whereas one MCU-targeting study ran its inference prototype on a Raspberry Pi 4 and notes that the resulting inference-time and energy figures may not translate to MCU-class devices [48]. Read this way, claims of MCU-class suitability rest on MCU-class training in 26 of the 35 included studies that make them, and in about a quarter of the eligible studies that make them but were excluded for insufficient data. A match at tier level can still rest on part of the workload or of the evidence: one matched study benchmarked a single iteration of its smallest model, a sine-wave regression, on an Arduino Nano BLE 33 and its two convolutional workloads only on a Raspberry Pi 4 [69], and another drew its quantitative results from a Python simulation of the federation, describing the run on its ESP32 testbed only briefly, with per-client results in a figure [57]. Both are among the studies whose MCU feasibility is coded as only partly demonstrated (RQ3b). Studies should therefore state which claims their evaluation platform supports, and a claim of MCU-class suitability should be backed by training on MCU-class hardware or by resource figures read against the target device's capacity (RQ3a), as some matched MCU-class studies do by reporting the on-device memory footprint as a share of the board's capacity [65, 51].

3.2. RQ2: Resource optimization

3.2.1. *RQ2a. What techniques are proposed to reduce the computation, memory, communication, and energy requirements of FL on resource-constrained IoT and TinyML devices, and which resource constraints does each technique explicitly address?*

The 93 included studies evaluated 402 techniques or variants (median 4 per study; range 1–9), and 90 combined techniques from more than one optimization family, not counting the residual family Other. Table 10 summarizes them by the families of the extraction codebook. Local-training and aggregation optimization was the most widespread family (63 studies), followed by communication-efficient FL (51), quantization (33), model compression or reduction (32) and personalization or transfer learning (31). Counted once, by its primary family, a study fell most often under model compression or reduction (17 studies), quantization (14), local-training and aggregation optimization (13) and communication-efficient FL (12).

The families differ in the constraints they explicitly address (Table 10). Communication-efficient FL targets communication in 91.9% of its techniques, and system or network optimization in 72.1%. Model compression targets computation (80.0%) and memory (61.7%) more often than communication (40.0%); quantization targets memory (53.2%) and communication (54.8%), and pruning communication (67.3%) and computation (63.3%). Runtime and deployment techniques target memory (56.1%) and latency (43.9%) but hardly energy (4.9%). Energy is the explicit target of almost every energy-aware technique (97.1%), but of at most 41.9% of the techniques in any other family, and of under a fifth in model compression, quantization and knowledge distillation; pruning reaches 26.5%.

Counted once, by the resources its main intervention targets (study-level coding), communication was the constraint most often targeted (67 studies, 72.0%), followed by computation (61, 65.6%), latency (55, 59.1%), memory (56, 60.2%) and energy (40, 43.0%); these are the four resources named in RQ2a and latency, and other targets, such as privacy or deployment, are not counted. Constraints were rarely addressed alone: 7 studies targeted a single constraint, 25 two, and 61 (65.6%) three or more; one of the single-constraint studies, for instance, addressed communication alone through adaptive Top-K gradient sparsification [57]. The most frequent combination was all four resources named in RQ2a together with latency (10 studies), followed by communication, computation and latency and by computation, latency and memory (7 each), and 13 studies targeted all four resources named in RQ2a, 10 of them also latency.

Targeting a constraint did not mean quantifying it (Fig. 8). For the primary configuration, the change against the baseline was quantified in 36 of the 67 studies targeting communication (53.7%), 34 of 61 for computation (55.7%), 23 of 56 for memory (41.1%), 31 of 40 for energy (77.5%) and 46 of 55 for latency (83.6%). Measurement, physically, in hybrid setups or on a simulation host, was rarer still: 9 of the communication-targeting studies (13.4%) measured the change, against 19 (31.1%) for computation, 12 (21.4%) for memory, 13 (32.5%) for energy and 30 (54.5%) for latency. A further 16 memory-targeting studies reported a change in model size but not in memory.

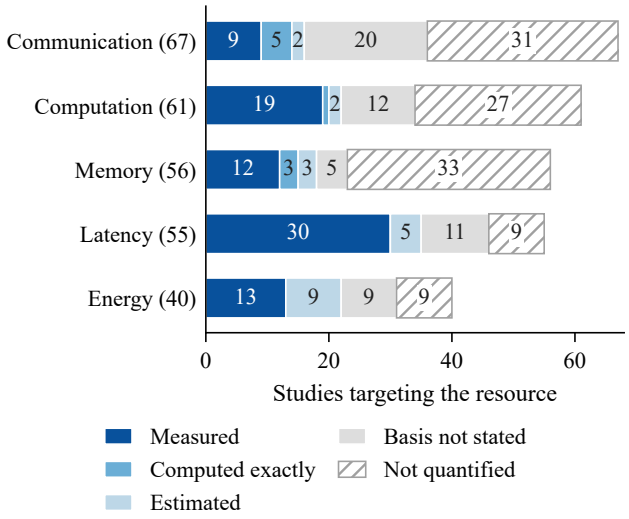


Figure 8: Evidence basis for quantifying the change in each targeted resource, among the studies whose primary technique targets it (primary configuration); segments show whether the change was measured, computed exactly, estimated, reported without a stated basis or left unquantified.

Table 10
Optimization families in the extraction codebook and the resources they explicitly target, among the 402 techniques evaluated by the 93 included studies.

	Counts			Techniques targeting each resource (%)				
Family	Primary	Studies	Techniques	Communication	Computation	Latency	Memory	Energy
Model compression or reduction	17	32	60	40	**80**	43	**62**	18
Quantization	14	33	62	**55**	27	37	**53**	16
Local-training and aggregation optimization	13	63	138	44	**52**	42	28	22
Communication-efficient FL	12	51	111	**92**	32	47	24	21
Personalization or transfer learning	6	31	62	44	**50**	27	37	21
System or network optimization	6	25	43	**72**	28	**53**	28	30
Energy-aware optimization	5	17	35	29	17	31	9	**97**
Hardware-aware optimization	4	28	62	39	42	**56**	42	27
Pruning or sparsification	4	24	49	**67**	**63**	37	49	27
Runtime and deployment optimization	4	24	41	32	37	44	**56**	5
Other	3	28	47	17	28	23	19	17
Client selection or participation optimization	3	19	31	39	26	**61**	13	42
Privacy or security mechanism	1	23	40	28	15	18	13	23
Knowledge distillation	1	9	18	**61**	**56**	28	6	11

Primary: studies whose primary family it is; Studies: studies with at least one technique in the family; Techniques: the family's techniques, the denominator of the percentages on the right. Shaded and bold: at least half of the family's techniques. A technique can belong to several families and target several resources. Families are ordered by primary studies, as in Fig. 9.

Fig. 9 places each study's primary technique against the evaluation categories of RQ1a. Studies whose primary technique belongs to a compression family were the most common (36 studies), yet only 6 of them were demonstrated by training on physical MCU-class devices, 5 of these quantization [39, 68, 60, 55, 65]. Among the 24 physical MCU-class deployments, the most frequent primary family was quantization (5 studies), followed by hardware-aware [73, 54, 67], runtime or deployment [12, 75, 62], system or network [52, 72, 63], local-training and aggregation optimization [61, 71, 58] and communication-efficient FL [70, 69, 57] (3 studies each). Of the 15 studies whose primary technique is communication-oriented (communication-efficient FL or client selection), 6 were evaluated on physical SBC-class devices and 4 on MCU-class devices [70, 69, 57, 74]; a further MCU-class federation, of Raspberry Pi Pico boards whose network was extended in a calibrated simulation, counts as hybrid [51].

Discussion of RQ2a The constraint most often targeted is the one least often measured. Communication cost leads the field's targets, but its reductions are mostly reported without a stated basis (20 of 36) or computed from message sizes (5), and only 9 studies measured the traffic they saved, 8 of them physically or in hybrid setups, for instance by counting incoming and outgoing bytes at the server of a Thunderboard Sense 2 federation [65]. Energy is targeted least, and about a third of the quantified energy changes are estimates (9 of 31). Memory, the binding limit of MCU-class devices, is quantified in fewer than half of the studies that target it, and often only through model size; where a RAM figure is given, it can be the static footprint of the client executable rather than the memory used during training [65]. Claims of resource efficiency therefore rest unevenly on evidence: they are best supported for latency and weakest for communication and memory.

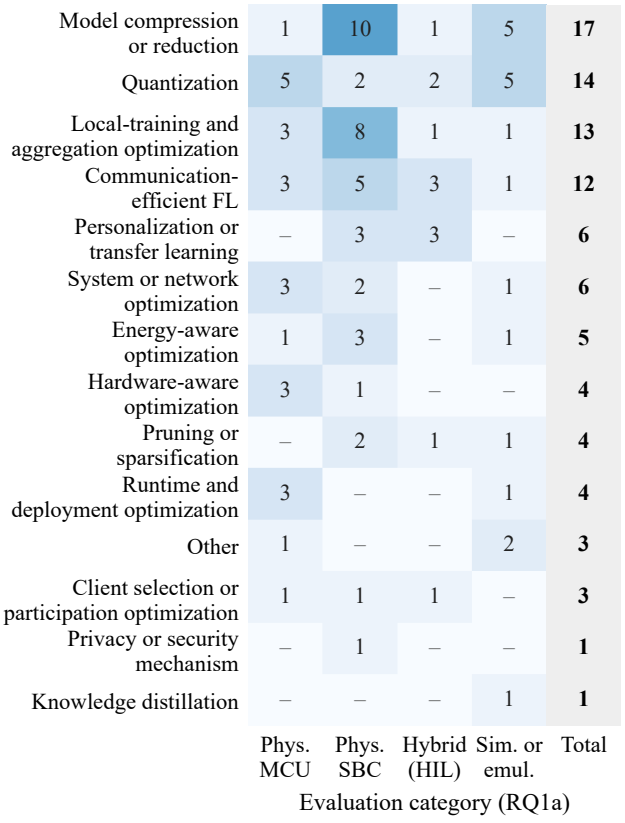


Figure 9: Primary optimization technique family by evaluation category (RQ1a); each included study contributes once, by its primary family.

Most studies address constraints jointly (65.6% target three or more), which reflects that savings in one resource can cost another; a technique that cuts transmitted bytes, for instance, can add local computation. Compression, the most common primary strategy, mostly targets computation or

Table 11
Reported changes from the explicitly identified baseline for the primary configuration of the 93 included studies, by measure.

	Reporting and direction				Measured on	Single-% baseline change	
Measure	Studies	Favorable	No change	Unfavorable	hardware, *n* (%)	Studies	Range (%)
Task performance	75	49	3	23	49 (65.3%)	–	–
Model size	27	26	0	1	1 (3.7%)	8	−96 to −53
Memory	23	19	1	2	11 (47.8%)	8	−70 to +56
Computation	43	32	0	10	23 (53.5%)	16	−75 to +65
Communication	39	36	1	2	9 (23.1%)	21	−99.9 to −17
Latency	64	50	2	12	35 (54.7%)	23	−92 to +65
Energy	34	31	1	2	15 (44.1%)	18	−89 to +15

Unit of analysis: study (one change per study and measure). Direction: favorable to the proposed technique, no change or unfavorable; directions that could not be determined are omitted, so the direction columns can sum to less than Studies. Measured on hardware: changes measured in studies evaluated physically or in hybrid setups, *n* (% of the studies reporting the measure). Single-% baseline change: studies stating the change as a single percentage of the baseline, and the reported range; negative values are decreases. The energy increase is a change in power.

memory, the limits that matter most on MCU-class devices, yet it was demonstrated on such devices in only 6 of 36 studies, and the MCU-class deployments drew on hardware-aware and runtime techniques as often as on compression (6 studies each). The optimization literature thus says more about what FL techniques could save than about what they save on the devices that need them most; RQ2b and RQ2c examine the reported savings, their trade-offs and their dependence on device tier.

3.2.2. *RQ2b. Relative to an explicitly identified baseline, what changes are reported in task performance, model size, memory consumption, computation, communication, latency, and energy for resource-optimization techniques in FL, and under what experimental conditions are those changes measured?*

All 93 included studies name an explicit baseline, as IC7 requires, most often another FL algorithm (63 studies), another reference such as centralized, local or non-FL training (51), or a different or unoptimized model (40). Table 11 summarizes the changes reported against that baseline for the primary configuration; 89 studies report such a change, and the other four compare only other configurations, such as successively smaller fractions of the transmitted parameters [57], or a simulation. A task-performance change was reported in 75 studies, latency in 64, computation in 43, communication in 39, energy in 34, model size in 27 and memory in 23.

The reported changes overwhelmingly favored the proposed techniques: 194 of the 230 resource changes (84.3%) used less of the resource than the baseline (quality of this evidence: Table 20). This counts the direction of the reported observations against each study's chosen baseline, not the probability that a technique saves the resource, because the observations differ in baseline, scope, measurement basis and resource. Task performance improved in 49 studies, was unchanged in 3 and declined in 23. Resource costs concentrated in computation and latency (10 and 12 unfavorable changes, from 15 studies, 7 of which reported both, often as the same training-time figure), whereas model size had a single unfavorable change and communication two.

How the changes were obtained differs by measure (Fig. 10). Of the 305 reported changes, 143 were measured in studies evaluated physically or in hybrid setups, 26 on a simulation host, 13 computed exactly, 24 estimated and 99 reported without a stated basis. More than half of the latency (35 of 64) and computation changes (23 of 43) were measured on hardware, against 9 of the 39 communication changes and 1 of the 27 model-size changes; 9 of the 34 energy changes were estimates.

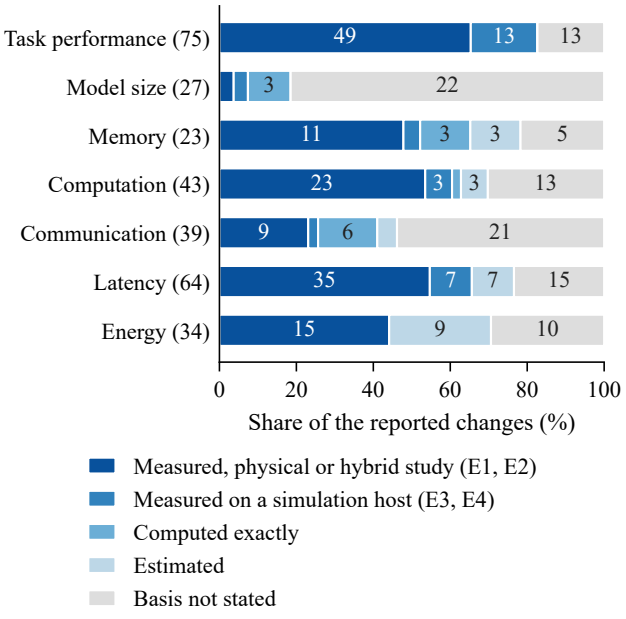


Figure 10: Evidence basis for the changes reported against the explicitly identified baseline (primary configuration), by measure: measured in studies evaluated physically or in hybrid setups (E1, E2), measured on simulation hosts (E3, E4), computed exactly, estimated, or reported without a stated basis. Share of each measure's changes, with counts in the bars; in parentheses, the studies reporting a change in the measure.

Where a study states its change as a single percentage, the reported changes span wide ranges (Table 11): communication reductions of 17% to 99.87% (21 studies) and model-size reductions of 52.95% to 96% (8), and changes ranging from a 92% reduction to a 65% increase in latency (23), from a 75% reduction to a 65% increase in computation (16), from a 69.9% reduction to a 55.92% increase in memory (8) and from an 89.41% reduction to a 15% increase in energy or power (18; the increase is a power figure). These are reported ranges, not effect estimates: the percentages refer to different baselines and scopes, such as a single round or the whole training, and are therefore not pooled. Even within one measure the units differ: of the 43 computation changes, 31 were expressed as time and 5 as operation counts (FLOPs or MACs).

The changes were obtained most often on physical SBC-class hardware (Table 13). On physical MCU-class deployments (E1), changes were measured for task performance in 12 studies and for latency in 12, but for computation in 6, memory in 4 [73, 61, 65, 69], energy in 4 [73, 72, 55, 69], communication in 3, one of which found no change [61], one an increase in fetch requests per round [63] and the third a reduction from quantizing the differences between successive model versions, counted at the server [65], and model size in none. A hybrid MCU-class study, outside these E1 counts, reported a change in every measure of Table 11, partly from its simulated extension of the network [51]. Of the 89 studies reporting a change, 58 evaluated non-IID data; a partition scheme was named in 61 and the data split described in 85, the number of rounds was given in 67, and the dataset, model, partition scheme and rounds together in 44; the energy-measurement method was stated in 35.

Gains in one dimension often came with costs in another (Table 12). Of the 74 studies reporting a resource saving, 21 also reported lower task performance, 40 higher, 2 unchanged and 11 did not report it, and in 14 studies one resource improved while another worsened, most often latency or computation (7 each). The direction of the performance change depended on the baseline (Table 12, lower block): in studies whose baselines include another FL algorithm, performance improved in 37 of 50, whereas in studies compared only with other references, such as centralized training, the uncompressed model or training without FL, it declined in 11 of 25 and improved in 12. By technique group, compression reported a performance loss in 12 of its 32 studies with a performance change, and the hardware, runtime, system and energy group carried the most resource costs (14 of its 35 resource changes were unfavorable).

Discussion of RQ2b The evidence supports the direction of these changes more firmly than their size. Most reported resource changes favor the technique against its chosen baseline, but the stated savings span tens of percentage points, refer to different scopes and baselines, and cannot be pooled into a general effect; in some studies, the abstract or text even states a larger change than the paper's own tables or detailed results support [55, 56, 47]. The measures the field targets most are not the best measured: communication savings, the most frequent target (RQ2a), were measured on hardware in 9 of 39 studies and on MCU-class hardware in only 3, one of which found no change, whereas latency and computation, which carry most of the reported costs, are the best measured.

The baseline shapes the conclusion. The same technique can appear beneficial against another FL algorithm and costly against centralized training or an uncompressed model, so studies should report both comparisons and state the scope of each change. Even among baselines of one kind the verdict can flip: compared with its fixed-precision baselines, an adaptive-quantization framework reported a smaller model size than FP16 but a larger one than INT8 and INT4 [47]. Where published baselines did not fit the device, one MCU-class study adapted blockchain-FL frameworks to the memory of its RP2040 boards before comparing against them, so the comparison is with the adaptation rather than with the published method [51]. What cannot yet be concluded is whether savings measured on SBCs or in simulation carry over to MCU-class devices: MCU-class evidence covers latency and task performance but barely memory, energy and communication, and only 44 of the 89 studies name the dataset, model, partition scheme and number of rounds together. RQ2c examines how the reported effects vary across device tiers.

3.2.3. *RQ2c. How do the reported effects of FL resource-optimization techniques vary across device tiers?*

RQ2c compares the changes reported under RQ2b across device tiers. Each change is placed where the technique was evaluated: on MCU-class (T1) or SBC-class (T2) devices, physically or in hybrid setups (modes E1 and E2), or, for simulated or emulated evaluations, by the targeted tier, because the evaluated tier of a simulation is its host. Of the 89 studies reporting a change, 24 were evaluated on T1 devices and 47 on T2 devices (22 and 37 of them in physical deployments, the rest hybrid; two physical MCU-class deployments report no change against their baselines [70, 57]), and 7 and 11 in simulation targeting T1 and T2, respectively (Table 13); which techniques were evaluated in which evaluation category is shown in Fig. 9 (RQ2a).

The evidence thins where the constraints are tightest (Table 13). Task performance and latency changes were reported by 16 and 17 of the 24 MCU-class studies evaluated physically or in hybrid setups, but memory and computation changes by 10 each, communication changes by 11, energy by 8 and model size by 4, and fewer were measured on the devices: memory changes in 4 studies, communication in 3 [61, 63, 65] and model size in none, although energy was measured in 4 of 8. On SBC-class devices, energy changes were reported by 19 of 47 studies and measured in 11.

Resource savings were reported on every tier, but costs were reported more often on MCU-class hardware (Table 13). Of the 60 resource changes reported on T1 devices,

Table 12
Reported trade-offs by primary technique group and by baseline type; resource changes are counted separately from task-performance decline.

			Resource changes			
Group	Studies	Perf. decline, n/N (%)	Fav.	Unfav.	Costs in	HW
Primary technique group						
Compression	36	12/32 (37.5%)	92	7	Latency (3); Computation (2); Memory (1); Model size (1)	42
Communication-oriented	15	2/10 (20.0%)	33	4	Computation (3); Energy (1)	23
Local training and aggregation	13	3/11 (27.3%)	31	4	Latency (3); Computation (1)	32
Hardware, runtime, system and energy	19	3/13 (23.1%)	20	14	Latency (6); Computation (4); Communication (2); Memory (1); Energy (1)	33
Personalization, security and other	10	3/9 (33.3%)	18	0	none	13
Type of baseline						
Baselines include another FL algorithm	59	12/50 (24.0%)	143	16	Latency (7); Computation (6); Communication (2); Memory (1)	103
Other references only	30	11/25 (44.0%)	51	13	Latency (5); Computation (4); Energy (2); Memory (1); Model size (1)	40

Studies counts all 93 studies in the upper block and the 89 that reported a change in the lower block. Studies and Perf. decline count studies (decline: studies whose task performance declined, of those reporting a performance change); Fav. (favorable), Unfav. (unfavorable) and Costs in count resource changes; HW counts changes of all measures, including task performance, measured in studies evaluated physically or in hybrid setups.

Table 13
Reported changes and unfavorable outcomes by tier stratum; physical or hybrid strata are grouped by evaluated tier and simulated strata by targeted tier.

	Physical or hybrid (by evaluated tier)				Simulated (by targeted tier)	
	T1 ($n = 24$)		T2 ($n = 47$)		T1 ($n = 7$)	T2 ($n = 11$)
Measure	Studies	Unfavorable	Studies	Unfavorable	Studies (unfav.)	Studies (unfav.)
Task performance	16	4 (25.0%)	43	13 (30.2%)	7 (3)	9 (3)
Model size	4	0 (0.0%)	15	1 (6.7%)	3 (0)	5 (0)
Memory	10	1 (10.0%)	12	1 (8.3%)	0 (0)	1 (0)
Computation	10	4 (40.0%)	25	4 (16.0%)	4 (1)	4 (1)
Communication	11	2 (18.2%)	21	0 (0.0%)	3 (0)	4 (0)
Latency	17	7 (41.2%)	34	3 (8.8%)	6 (1)	7 (1)
Energy	8	1 (12.5%)	19	1 (5.3%)	5 (0)	2 (0)

Unit of analysis: study (one change per study and measure, primary configuration). Physical or hybrid strata (E1, E2): studies, and unfavorable changes as n (% of the studies); simulated strata (E3, E4): studies, with unfavorable changes in parentheses. The stated percentage changes are shown in Fig. 11.

15 (25.0%), from 9 studies, were unfavorable, against 10 of 126 (7.9%) on T2 devices (Table 20), 2 of 21 in simulations targeting T1 and 2 of 23 in simulations targeting T2; 9 of the 22 T1 studies reporting a resource change reported at least one such cost, against 8 of 44 on T2. The difference held within baseline type, though on few studies: in studies whose baselines include another FL algorithm, 7 of 38 changes on T1 and 6 of 97 on T2 were unfavorable, and in the others 8 of 22 and 4 of 29. The costs on T1 lay in latency (7) and computation (4), with two in communication and one each in memory and energy, and 12 of the 15 were measured on the devices.

Task performance declined in 4 of the 16 physical or hybrid MCU-class studies that reported it, but in 13 of 43 on SBC-class devices and in 3 of 7 and 3 of 9 simulations targeting T1 and T2. The stated percentage changes overlap across tiers (Fig. 11): communication reductions on SBC-class devices evaluated physically or in hybrid setups ranged from 17% to 88% (14 studies) and latency reductions there from 10% to 92% (18), whereas the 4 latency percentages stated on MCU-class devices ranged from a 78% reduction in consensus latency against a flat peer-to-peer baseline [51] to a 65% increase. Only 14 of the stated percentages come from physical or hybrid MCU-class evaluations, too few to compare magnitudes across tiers.

Within-study comparisons could separate the tier from the technique. Of the included studies, 40 were evaluated on more than one tier (30 of them on at least two classifiable tiers), most often an SBC alongside a server or PC (15), and only 11 combined an MCU with an SBC; the changes analyzed here are those of the primary configuration, so these evaluations are not used here to show whether the same technique keeps its benefit when moved from SBC-class to MCU-class hardware. Two of these 11 studies show what such comparisons can and cannot settle. INT8 quantization sped up inference by about 2.5 times over the FP32 baseline on both an ESP32 and a Raspberry Pi 4B, although latency

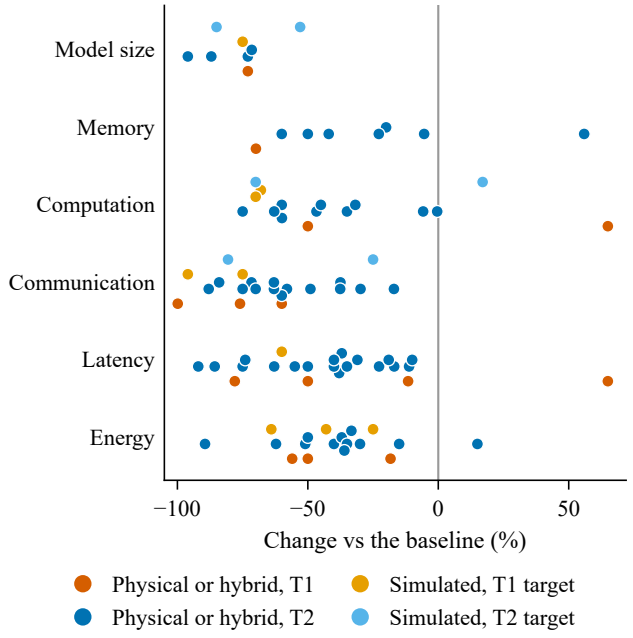


Figure 11: Reported single-percentage changes from the baseline by resource and tier stratum; each point is one study's reported change, not a pooled effect estimate. Points within 5 percentage points of each other are offset vertically so that all remain visible.

remained higher on the MCU [55]. Communication and energy savings against TinyReptile were larger on a Raspberry Pi 4 than on an Arduino Nano BLE 33, but each device ran a different task and model, so the tier cannot be separated from the workload [69].

Discussion of RQ2c Resource savings are reported on every tier, but the evidence does not support a comparison of their sizes across tiers. The clearest tier-dependent pattern is evidential: physical or hybrid MCU-class evaluations report costs, which appear in a quarter of the resource changes on MCU-class devices (25.0%), mostly as latency and computation measured on the devices themselves, partly because some MCU-class studies compare against the same workload on a more capable platform [75, 58], while they measure memory and communication far less often. The principal gap is therefore not evidence that optimization is less effective on MCUs, but too little controlled on-device evidence to establish whether savings demonstrated in simulation or on SBCs transfer to MCU-class devices.

The resources that bind MCU-class devices are also the least measured on them, as the results above show, and whether the optimized workloads fit their devices is examined under RQ3a. One MCU-class study names the obstacle: on its Thunderboard Sense 2 testbed, energy could not be measured on the devices in practice, so communication and computation costs stood in for it [65]. Studies should evaluate the same technique and workload on more than one tier and measure memory, energy and communication on the MCU itself; until then, the stated savings cannot be compared across tiers, and results from SBCs and simulations should not be transferred to MCU-class devices without measurement.

Table 14
Resource figures reported by the 93 included studies and their evidence basis; hardware (HW) counts refer to measurements in mode E1 or E2.

		Basis				
Figure	Rep.	Meas.	Exact	Est.	N.s.	HW
Peak training memory	26	15	1	4	6	15
Optimizer state	1	0	1	0	0	0
Activation memory	0	0	0	0	0	0
Static or runtime overhead	8	0	1	0	7	0
Peak inference memory	11	3	2	0	6	2
Model size	50	3	6	1	40	2
Parameter count	41	0	12	1	28	0
Update payload	33	6	7	0	20	6
Computation	64	45	3	2	14	41

Rep.: reported; basis (the strongest among the evidence quotes): Meas.: measured; Exact: computed exactly; Est.: estimated; N.s.: not stated. HW: measured physically or in hybrid setups.

3.3. RQ3: Resource feasibility and runtime support

3.3.1. *RQ3a. What reported measurements establish whether FL workloads fit their target devices, particularly with respect to peak training memory, optimizer state, model and update size, and other execution-resource requirements?*

Of the figures that bound on-device training, peak training memory was reported by 26 of the 93 included studies, 15 of them measured physically or in hybrid setups, whereas optimizer state was reported by 1 and activation memory by none (Table 14). Model size (50 studies), parameter count (41) and update payload (33) were reported more often but measured in 3, none and 6 studies; most were reported without a stated basis (40, 28 and 20) or computed from the architecture (6, 12 and 7). Computation, reported mostly as training or inference time (48 of the 64 studies), was the best-measured figure (45 of 64, 41 physically or in hybrid setups).

Every included study met IC8 for RQ3 at full-text screening, by setting a resource figure against its target's capacity or by running federated training or inference with a named MCU-native runtime on an identified T1 device, and the screening record names the figure that established fit (Fig. 12). It was a model or update size in 59 studies, peak training memory in 24, other runtime or static memory in 12, storage in 8 and computation in 2; 47 studies rested on a model or update size alone, and 6 passed only through a named MCU-native runtime (RQ3b).

For 15 studies, all evaluated on physical devices (one in a hybrid setup [51]), the extraction derived a fit ratio, the memory of a device divided by the peak training memory on that device (Table 15); 8 of these requirements were measured,

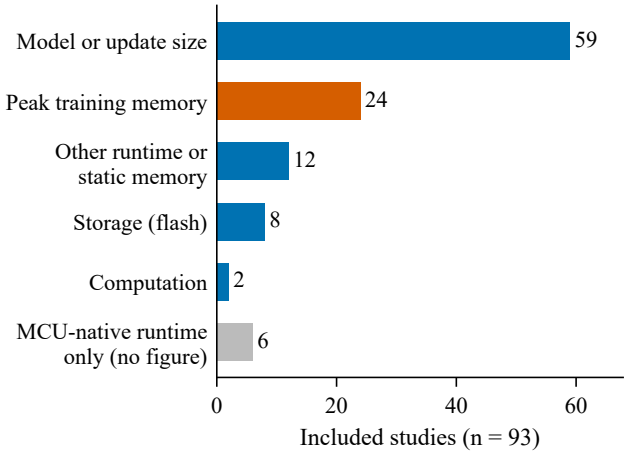


Figure 12: Evidence type used to establish resource fit at full-text screening among the 93 included studies; a study may rely on more than one resource figure.

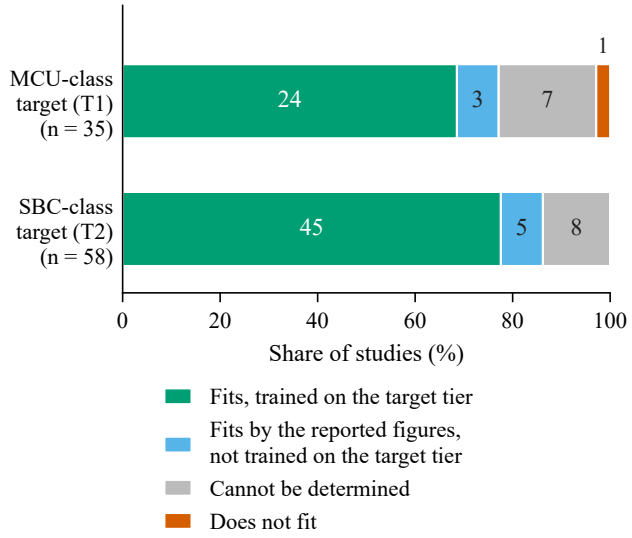


Figure 13: Resource-fit verdicts by targeted tier, with the studies that fit split by whether FL training ran on hardware of the target tier; share of studies, with counts in the bars. Same-device fit ratios are given in Table 15.

and a few further studies report both figures in forms the extraction did not convert, such as training memory given as a percentage of a Raspberry Pi 4's memory [59]. On MCU-class devices the headroom was small in fewer than half: device memory was at most 1.60 times the requirement in 4 of the 10 (2 of them measured), including 256 kB of 256 kB SRAM on an nRF52840 [54] and 78.67 kB of 80 kB on an STM32F303 [70], and the device memory exceeded the requirement more than tenfold only for workloads of a few kilobytes [52, 61]. What the requirement covers also varies: on the RP2040 it is the runtime allocation of the complete implementation, in which the FL client state sits alongside the inference model, a blockchain ledger, network buffers and the system stack [51], and on the Thunderboard Sense 2 it is the RAM footprint of the client executable reported by the Linux size command [65]. The ESP32-CAM requirement in the table is that of the smallest of the three autoencoders trained in the federation. The largest, which performed best after federated training, needed an estimated 398.86 kB, more than the roughly 300 kB of on-chip RAM left for training, and relied on external pseudostatic RAM (PSRAM) [58]. On SBC-class devices, the ratios ranged from 1.14, on a Jetson Nano with 2 GB [88], to about 1860.

Read together with where training ran, fit was established on hardware of the targeted tier for 24 of the 35 MCU-targeting studies and 45 of the 58 SBC-targeting studies (Fig. 13; these sets differ slightly from those of RQ1b, which do not depend on fit); a further 3 and 5 fit by their reported figures without training physically or in hybrid setups on hardware of the targeted tier, fit could not be determined for 7 and 8, and one MCU-targeting study reported figures that exceed the capacity of its target. Fit could not be determined for 9 of the 13 studies that simulated the federation on a server or PC.

Discussion of RQ3a What the literature most often offers as evidence of fit is the size of the model or update. That bounds storage and the bytes sent in each round, but not the memory of training, which must also hold the gradients and activations of the backward pass and any optimizer state. Peak training memory, the figure that does bound it, was reported in 26 studies, and its components were almost never reported separately (optimizer state in 1 study, activation memory in none, and gradients pooled with the optimizer state in another [51]); for 47 studies, fit at screening rests on a size figure alone.

Where training memory was read against the memory of an MCU, the workload used most of it in 4 of the 10 cases, 2 of them measured (Table 20), which suggests that the margin by which an FL workload fits an MCU can be thin and lost to a larger model, batch or optimizer. A claim of MCU-class suitability should therefore report peak training memory measured on the device, state what it includes, and read it against the device's capacity; RQ3b examines the runtimes that make such measurements possible.

3.3.2. *RQ3b. Which MCU-native training and inference runtimes, libraries, and deployment frameworks are used for FL on MCU-class devices, and what role do they play in enabling resource-constrained execution?*

Of the 93 included studies, 38 used or targeted an MCU-class device (the 35 MCU-targeting studies of RQ1a and 3 SBC-targeting studies with an ESP32 or an Arduino Portenta H7), and they differ in what the MCU ran (Fig. 14a). In 26, the FL clients trained on MCU-class hardware (24 physical deployments and the two hybrid studies of RQ1a), with MCU feasibility coded in the extraction as demonstrated in 17 of them, partly demonstrated in 8 and not applicable in one [63]. In 5 studies, FL training ran off the MCU: on SBC-class clients in four, where an MCU ran inference of the federated model in three [93, 94, 44] and the fourth

Table 15
Peak training memory relative to device memory in studies providing same-device fit evidence; the fit ratio is device memory divided by peak training memory.

Study	Tier	Device	Peak training memory	Device memory	Fit ratio	Basis	Fit evidence
[54]	T1	nRF52840	256 kB SRAM	256 kB	1.00	Not stated	Fits with optimization
[70]	T1	STM32F303	78.67 kB average SRAM usage during on-device FL training	80 kB	1.02	Measured	Fits with optimization
[12]	T1	Arduino Nano 33 BLE Sense	210 kB of the 262 kB available	256 kB SRAM	1.22	Estimated	Fits
[73]	T1	ESP32-S3	320 kB peak SRAM	512 kB SRAM	1.60	Measured	Fits with optimization
[51]	T1	RP2040	140 kB SRAM	264 kB	1.89	Not stated	Fits
[67]	T1	STM32F746G-DISCO	124 kB	320 kB SRAM	2.58	Measured	Fits with optimization
[65]	T1	Thunderboard Sense 2	74.76 kB RAM	256 kB	3.42	Measured	Fits
[58]	T1	ESP32-CAM	103.73 kB	520 kB internal SRAM	5.01	Estimated	Fits
[52]	T1	STM32F446	6,608 bytes	128 kB SRAM	19.37	Not stated	Fits
[61]	T1	Arduino Nano 33 BLE Sense	4.8 kB	256 kB	53.33	Measured	Fits
[88]	T2-H	Jetson Nano 2GB	1747 MB	2 GB RAM	1.14	Measured	Fits with optimization
[89]	T2-L	Raspberry Pi Model 3B	599.43 MB	1 GB RAM	1.67	Measured	Fits
[90]	T2-L	Raspberry Pi 4 Model B	110 MB	8 GB SDRAM	72.73	Measured	Fits
[91]	T2-L	Raspberry Pi 4B	36.9 MB	4 GB	108.40	Not stated	Fits
[92]	T2-L	Raspberry Pi 4	4.3 MB	8 GB LPDDR4 SDRAM	1860.47	Not stated	Fits

Fit ratios as extracted (values near 1 leave little headroom), in decimal units (1 kB = 1,000 bytes), so ratios for memories sized in binary units are about 2% low. Device memory as reported in the papers; entries are abridged, and some requirements are averages, budgets or estimates rather than measured peaks (see Basis). Fit evidence: whether the reported figures show that training fits the device, or fits only with the study's optimization. The fit verdicts of all included studies are shown in Fig. 13.

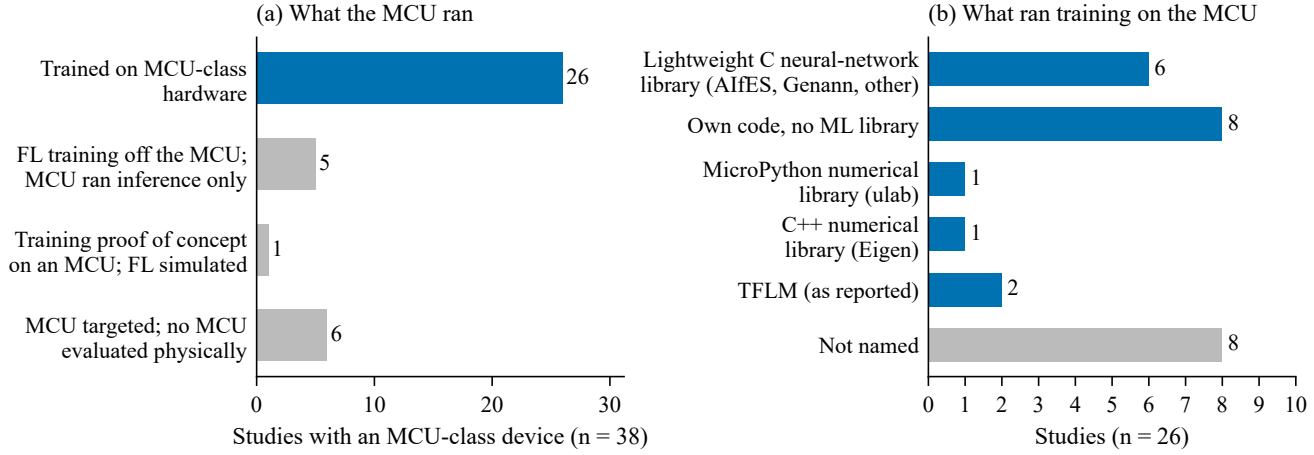


Figure 14: MCU-related deployment evidence: (a) what the MCU ran in the 38 studies that used or targeted an MCU-class device; (b) the software used for local training in the 26 studies that trained on MCU-class hardware (other library: public Arduino neural-network code).

does not state what its ESP32 ran [76], and in a simulation on a workstation in one, with ESP32 nodes used only to evaluate model compression [49]; one trained a model on an ESP32 only as a proof of concept beside a simulated FL evaluation [78]; and 6 named an MCU-class device but evaluated none physically. Only the first group shows federated training running on MCUs, and the rest of this subsection concerns these 26 studies unless stated otherwise.

For these studies, the extraction coded the MCU runtime or library as TFLM in 3, as another runtime or library in 13 and as none in 1, and recorded it as not reported in 9. Read together with the deployment toolchain and the supporting quotes, 13 studies named an ML library, runtime or optimized kernel code that ran on the MCU, two of them only kernel code [70, 51] (Table 16): TFLM in 4 (once as a comparator), AIfES in 3, Genann and Edge Impulse code

Table 16
Software used on or for MCU-class hardware in FL, grouped by ML runtime, kernel, host-side preparation and FL-framework layer; detailed extraction values are retained in the review's data file.

Layer	Software	n	Studies, MCU devices and role
ML library or runtime	TensorFlow Lite Micro (TFLM)	6	[50] ESP32: inference (frozen audio embedding); [44] ESP32-WROOM-32: inference only (sensor tier); [94] ESP32 Devkit V1: inference of the FL-trained model; [53] ESP32-S3: training and inference (as reported); [55] ESP32 DevKitC: training and inference (as reported); [62] ESP32: inference of a remotely trained model (comparator)
	AIfES	3	[52] STM32F446: training and inference; [66] ESP32: training and on-MCU compression (extended library); [72] ESP32-S3: training
	Genann	3	[74] ESP32-S3: training and inference (extended library); [78] ESP32: training (proof of concept, extended library); [62] Arduino MKR1010, ESP8266 and ESP32: training and inference (modified: float instead of double, activation lookup table)
	Edge Impulse	2	[50] ESP32: inference (continuous inference API); [12] Arduino Nano 33 BLE Sense: speech processing and MFCC features
	Eigen	1	[63] nrf52840 development kit, Raspberry Pi Pico and STM32F746 discovery: linear algebra for on-device SGD training
	Public Arduino neural-network code	1	[12] Arduino Nano 33 BLE Sense: training and inference
	ulab	1	[75] Raspberry Pi Pico W: numerical operations (NumPy subset) for training
Kernel library	ARM Thumb assembly	1	[51] Raspberry Pi Pico: hand-optimized SHA-256 critical sections
	CMSIS-NN	1	[70] STM32F303, Arduino Nano BLE Sense, Arduino Nano ESP32 and Adafruit M4 Express: optimized kernels; unavailable on the Nano ESP32, which fell back to portable C kernels
	ESP-NN	1	[50] ESP32: accelerated INT8 inference kernels
Host-side preparation	TensorFlow Lite converter	7	[93] ESP32: quantization; [94] ESP32 Devkit V1: model conversion; [68] ESP32: post-training quantization; [53] ESP32-S3: INT8 quantization to .tflite flatbuffers; [70] Arduino Nano BLE Sense, Arduino Nano ESP32 and Adafruit M4 Express: INT8 quantization (named as TensorFlow Lite Micro); [76] ESP32: conversion and compression for deployment; [57] ESP32: translation of the Keras models into optimized C code for MCU deployment
	PlatformIO	2	[50] ESP32: build; [78] ESP32: build and deployment
	TensorFlow Model Optimization Toolkit	2	[93] ESP32: pruning; [94] ESP32 Devkit V1: pruning
	Edge Impulse	1	[50] ESP32: INT8 post-training quantization and TFLM export
	STM32Cube.AI	1	[70] STM32F303: INT8 quantization
	U-TOE	1	[63] nrf52840 development kit, Raspberry Pi Pico and STM32F746 discovery: model transpilation and deployment
	GCC	1	[51] Raspberry Pi Pico: build of the C implementation (-O2)
FL framework	Flower	5	[93] ESP32: FL orchestration (training off the MCU); [94] ESP32 Devkit V1: FL orchestration (training off the MCU); [76] ESP32: FL orchestration (training off the MCU); [78] ESP32: FL simulation; [49] ESP32: FL simulation
	CoAPerator	1	[63] nrf52840 development kit, Raspberry Pi Pico and STM32F746 discovery: FL orchestrator (custom, Python; off the MCU)
	MPT-FLA	1	[75] Raspberry Pi Pico W: FL framework for MicroPython devices

Studies in which an MCU trained, in which training ran off the MCU, or with an MCU training proof of concept; operating systems and communication or storage libraries are in Fig. 15. Each entry gives the study, its MCU devices and what the software did; n: number of studies.

in 2 each, and the public Arduino neural-network code, ulab under MicroPython, the Eigen C++ library, ESP-NN kernels, CMSIS-NN kernels and hand-optimized ARM Thumb assembly for hashing in one each. Of the remaining 13, 5 ran their own code, one stating that it used no ML library [64] and four naming none [58, 67, 71, 65], two named only the converter that prepared their models [68, 57], and 6 named no ML library or runtime [73, 61, 54, 39, 60, 69]. For 6 studies the runtime was itself the evidence of resource fit: they met the fit requirement at screening only by running FL with a named MCU-native runtime (RQ3a), AIfES [66], TFLM [53, 55], ulab under MicroPython [75], RIOT with U-TOE [63] or TensorFlow Lite [57].

Training on the MCU seldom used the established inference tools (Fig. 14b). Training ran on lightweight C neural-network libraries in 6 studies (AIfES in 3, Genann in 2 and the public Arduino code in 1), on ulab under MicroPython in 1, on the Eigen C++ library in 1 [63] and on the studies' own code in 8, such as DNN training and inference implemented in C [65]; two studies report training with TFLM but do not describe how the backward pass was implemented [53, 55], and 8 did not name what ran training. These training libraries trained in every study that used them, whereas TFLM ran only inference in 4 of the 6 studies in Table 16 that used it on an MCU, and the TensorFlow Lite converter (7 studies in the table), STM32Cube.AI and the Edge Impulse toolchain quantized and converted models on the host (Table 16). Two

studies that prepared or ran inference with these tools wrote their own C code for training [50, 70], one because TFLM does not implement automatic differentiation [70]. In all 4 studies whose FL training ran on SBC-class clients, the model for the MCU was prepared or run with TensorFlow Lite tools.

Where the software was adapted to fit the device, the adaptation was the study's own. One study reordered the backward pass with checkpoint-recompute and in-place gradient updates, lowering peak training RAM from a theoretical 412 kB for standard TFLite backpropagation to 124 kB on an STM32F746 board with 320 kB of SRAM [67]; a modified Genann library (float instead of double, a smaller activation lookup table) let an Arduino MKR1010 with 32 kB of RAM train a small regression network, though not the ECG network [62]; an extended Genann added a lookup-table tanh and SD-card streaming [74]; a float32 reimplementation in C kept what it could in internal RAM and moved part of the model and of the training buffers to external PSRAM once internal RAM ran out [58]; one study hand-wrote the inner loop of its SHA-256 hashing in ARM Thumb assembly so that the hash state stayed in registers on the Cortex-M0+ instead of spilling to memory [51]; one study avoided external libraries altogether to minimize overhead and memory footprint [64], and another made minor adjustments to the TFLM runtime [50]. On the inference side, INT8 quantization cut the TFLM tensor arena from 51,635 to 20,147 bytes and ESP-NN kernels cut inference from about 1,260 to 250 ms per window [50], storing the quantized model in flash reduced its RAM requirement [68], and two ESP32 boards could load only the quantized model [93, 94]. The remaining layers carried communication (MQTT, SSL and LoRa mesh routing libraries, and the OpenThread network stack over which one study exchanged CoAP messages [65]), storage (a flash file system) and task scheduling (FreeRTOS), and each item was named by at most two studies (Fig. 15).

FL frameworks were rarely used on the MCU itself. Of the 26 studies that trained on MCUs, 23 implemented FL themselves, one used MPT-FLA, an FL framework designed for MicroPython devices [75], and two did not report it; Flower appeared where training ran off the MCU (6 of the other 12 studies) and once as an emulated comparator [74].

Discussion of RQ3b In these studies, the common MCU deployment tools (TFLM, the TensorFlow Lite converter, STM32Cube.AI and Edge Impulse) mostly served inference, quantizing, converting and executing trained models, while training depended on small backpropagation libraries such as AIfES and Genann or on the authors' own code, often adapted to fit memory. The training libraries were also narrow; Genann natively supports only sigmoid and linear activations, so tanh had to be added [74, 78]. Portability across MCUs was limited: CMSIS-NN kernels were unavailable on one board, which fell back to portable C [70], and in another study TFLM was compatible only with the ESP32 among its boards, while Genann had to be modified for compatibility with Arduino, Raspberry and Espressif microcontrollers [62].

MCU-native FL therefore rests on software assembled study by study rather than on a shared stack: 23 of the 26 studies wrote their own FL client logic, and 8 did not name what ran training on the device. Because the runtime determines which models can be trained and how much memory training takes (RQ3a), a claim of MCU-native FL should name the runtime or library, its version and its role, and state whether it was modified.

3.4. RQ4: Security and privacy

3.4.1. RQ4a. What security and privacy mechanisms are used in FL for resource-constrained IoT and TinyML devices, what security or privacy properties do they provide, and against which threat models or attacker capabilities are they evaluated?

RQ4 rests on the fully reported studies defined below; the 29 included studies that reported any security or privacy mechanism are described first for context. The 28 of them with mechanism records list 56 mechanisms (Fig. 16); a further mechanism record belongs to a study whose study-level field reports no mechanism and is not counted. Differential privacy was the most frequent (17 mechanisms in 17 studies), followed by secure aggregation (13 in 12), robust aggregation or reputation (10), encrypted channels and blockchain ledgers (4 each), and signatures, attestation or watermarking (3). Differential privacy and secure aggregation together account for 30 of the 56 mechanisms; 16 studies combined mechanisms of different kinds, 10 of them pairing a privacy mechanism with an integrity, authenticity or robustness mechanism.

Few of these studies reported what is needed to judge a mechanism (Fig. 17a). Taken one element at a time, of the 29, 28 stated the property a mechanism provides, 21 a threat model or attacker capability, 18 an evaluation of the protection and 10 a cost of the mechanism; counted cumulatively, as in Fig. 17a, these fall to 28, 21, 17 and 9. Only 9 reported a mechanism with its property, threat model and client-side cost, the evidence RQ4 requires (Section 1.6); full-text screening confirmed this for 8, the *fully reported* studies, and rated the ninth unclear [95]. The protection was evaluated empirically in 10 studies, formally in 4 and both ways in 4, was not evaluated in 9, and could not be determined in 2. Mechanisms left unevaluated and without a defined attacker included MQTT broker-native TLS, client authentication and access control lists [44]; encrypted model updates with no named scheme [45]; and homomorphic encryption of the parameters exchanged between client and cloud [46].

Mechanisms were rarely run under MCU constraints. Of the 29, 5 trained on MCU-class devices (4 physically, 1 in a hybrid setup), 14 on SBC-class devices (12 physically, 2 in hybrid setups) and 10 in emulation or simulation; of the 8 fully reported studies, 4 were evaluated on SBC-class hardware (3 physically, 1 in a hybrid setup), 3 in simulation and one on MCU-class hardware, in a hybrid setup of 18

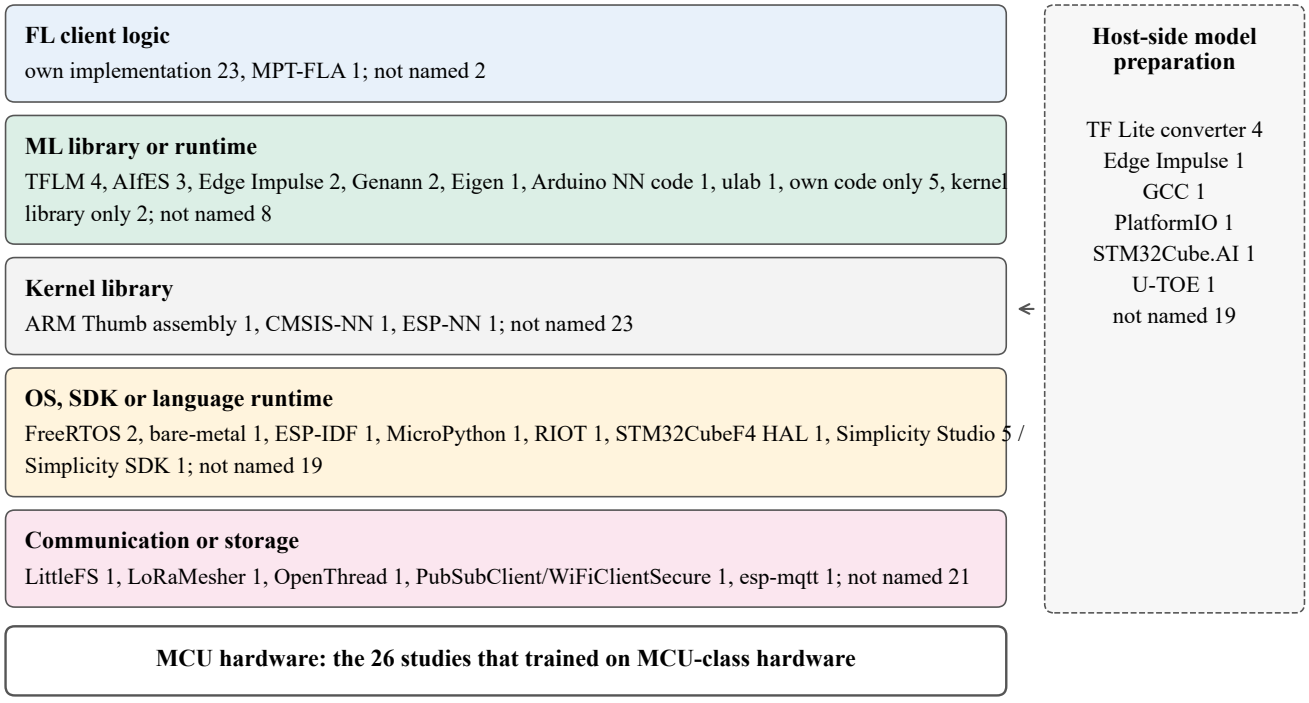


Figure 15: Software named at each layer of the MCU-side FL stack in the 26 studies that trained on MCU-class hardware; counts are studies, and a study may name several items in a layer. TFLM includes one use as a comparator; own code only: own code without an ML library; not named: no software named in that layer.

Raspberry Pi Pico boards and a calibrated network simulation [51] (Fig. 17b). Of the five studies that trained on MCUs, four [67, 53, 72, 71] reported differential privacy (3), secure aggregation (2), a blockchain ledger (1) and an encrypted channel (1); the mechanism ran on the MCU clients only in part in three of them and only in a PC simulation in the fourth [71], none reported a cost of its mechanism and only one stated a threat model, so none was fully reported. The fifth [51] ran a reputation-weighted consensus and a cryptographic identity layer, and in part its anti-centralization mechanisms, on its RP2040 clients, with threat models and reported costs, and is the one fully reported study evaluated on an MCU.

In the 8 fully reported studies, the 16 mechanisms provided robustness (6 mechanisms), integrity and authenticity (5 each), differential privacy (4), confidentiality (3) and access control (2) (Table 17). Differential privacy came from DP-SGD or local noise addition with stated (ε, δ) budgets [96, 97, 98] and, in one study, from a single reported result at $\varepsilon = 1$ [51], and confidentiality from secure aggregation by pairwise masking with secret sharing [97, 99, 77], in one study backed by a TrustZone trusted execution environment [99]. Integrity and authenticity came from blockchain ledgers [80, 41], signed configuration manifests [97], the TEE-backed protocol [99] and ECDSA identities with cryptographic commitments and hash-anchored state [51], and robustness from reputation-based isolation [80], a Byzantine-robust aggregation rule [96], cross-modal quorum verification [97], the update checks of a blockchain layer [41] and reputation-weighted consensus with anti-centralization safeguards [51].

The threat most often modeled was inference: 6 of the 8 studies protected against inference from their updates (individual updates, membership, gradient inversion or re-identification), in 4 of them by a curious (honest-but-curious or semi-honest) server or aggregator. Poisoning and tampering or forgery were each considered in 4 studies, collusion in 3, and eavesdropping and sensor spoofing in 1 each. The protection was evaluated empirically for 6 of the 16 mechanisms, formally for 2 and both ways for 5; 3 were described with a threat model but not evaluated, among them a blockchain ledger against a man-in-the-middle attacker [80] and a secure aggregation protocol [97].

Discussion of RQ4a These results suggest that security for constrained FL is framed mostly as privacy of the model updates: differential privacy and secure aggregation make up 30 of the 56 mechanisms, and inference was the threat in 6 of the 8 fully reported studies, whereas poisoning was considered in 4. Most studies report their mechanisms without the evidence needed to judge them: 20 of the 29 lacked at least one of the three elements, and even among the fully reported studies the protection of 3 mechanisms was asserted rather than evaluated.

The gap with the deployment target is wider still. One fully reported study was evaluated on an MCU [51], and the other four studies that trained on MCU-class hardware reported no cost of their mechanisms. Because MCU clients are where memory, energy and computation are scarcest, the

Table 17
Security and privacy mechanisms in the 8 fully reported studies, with the extracted property, threat model and evaluation, and the evaluated tier and mode.

ID	Study	Mechanism	Category	Property	Attacker	Attack capability	Evaluation	Tier, mode
M1	[80]	Permissioned Proof-of-Authority blockchain ledger	Blockchain or ledger	Integrity; Authenticity	External	Tampering or forgery	None	T3, E4
M2	[80]	Cosine-similarity reputation mechanism with penalties and permanent banning	Robust aggregation or reputation	Robustness	Clients	Poisoning	Empirical	T3, E4
M3	[96]	Local differential privacy	Differential privacy	Differential privacy	Server or aggregator	Inference	Formal	T2-L, E1
M4	[96]	Byzantine-robust aggregation rule rejecting poisoned client updates	Robust aggregation or reputation	Robustness	Clients	Poisoning	None	T2-L, E1
M5	[97]	DP-SGD	Differential privacy	Differential privacy	Server or aggregator; Clients	Inference	Both	T2-L, E1
M6	[97]	Bonawitz-style secure aggregation	Secure aggregation	Confidentiality	Server or aggregator; Clients	Inference	None	T2-L, E1
M7	[97]	Policy-attested execution with Ed25519-signed operator graph manifests	Signatures, attestation or watermarking	Integrity; Authenticity	Not specified	Tampering or forgery	Empirical	T2-L, E1
M8	[97]	Cross-modal quorum verification of security-critical events	Other	Robustness	External	Sensor spoofing	Empirical	T2-L, E1
M9	[98]	DP-SGD	Differential privacy	Differential privacy	Not specified	Inference	Both	T2-H, E2
M10	[41]	PureChain blockchain validation layer	Blockchain or ledger	Integrity; Authenticity; Robustness; Access control	Clients; Not specified	Poisoning; Tampering or forgery	Empirical	T3, E4
M11	[51]	Multi-Factor Proof-of-Performance	Robust aggregation or reputation	Robustness	Clients; External	Poisoning; Tampering or forgery; Collusion	Both	T1, E2
M12	[51]	Anti-centralization mechanisms	Robust aggregation or reputation	Robustness; Access control	Clients	Tampering or forgery; Collusion	Both	T1, E2
M13	[51]	Cryptographic identity and integrity	Blockchain or ledger	Authenticity; Integrity	External; Clients	Tampering or forgery	Empirical	T1, E2
M14	[51]	Differential privacy against gradient inversion	Differential privacy	Differential privacy	Not specified	Inference	Empirical	T1, E2
M15	[99]	TrustZone/OP-TEE-backed secure aggregation	Secure aggregation	Confidentiality; Integrity; Authenticity	Server or aggregator; External; Clients	Inference; Eavesdropping or traffic manipulation; Collusion	Both	T2-L, E1
M16	[77]	Secure aggregation by pairwise additive masking	Secure aggregation	Confidentiality	Server or aggregator; Clients	Inference; Collusion	Formal	T3, E4

Property as extracted; attacker and attack capability coded from the extracted threat model. Mechanism names are abridged; ID: mechanism identifier, also used in Table 18 and Fig. 18.

near-absence of costed, threat-modeled mechanisms evaluated on them is the main gap RQ4a identifies; RQ4b examines the reported overheads.

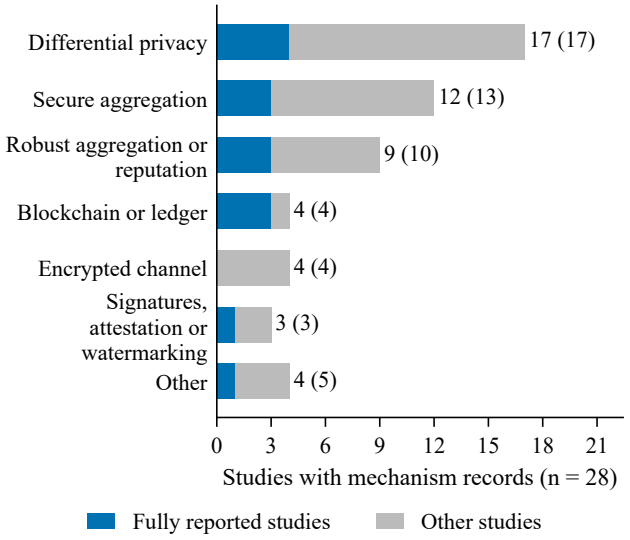


Figure 16: Security and privacy mechanism categories among the 28 studies with mechanism records; bars show studies, and labels in parentheses give the number of mechanisms. Dark: fully reported studies (a mechanism reported with its property, threat model and client-side cost, confirmed at full-text screening). A study can report mechanisms of several categories; one further study described its mechanism only at the study level.

3.4.2. *RQ4b. What computational, memory, communication, latency, and energy overheads are introduced by security and privacy mechanisms in FL, and to what extent are these mechanisms demonstrated to be feasible on MCU-class devices?*

RQ4b is synthesized from the 8 fully reported studies (Section 3.4.1); a ninth study that reported the three elements was rated unclear at screening and is not synthesized. Their 16 mechanisms are the unit here: 10 reported at least one overhead, one per study except for the three costed mechanisms of EdgeTrust-Shard [51], giving 24 overhead observations (Table 18, Fig. 18), and the other 6, reputation-based isolation [80], a Byzantine-robust rule [96], DP-SGD, signed manifests and quorum verification [97], and a differential-privacy result reported in one sentence [51], reported none.

Latency was the overhead reported most often (7 mechanisms), followed by communication (6), computation and energy (4 each) and memory (3); only the TrustZone-backed secure aggregation reported all five [99]. Of the 24 observations, 7 compared FL with and without the mechanism, 3 gave a relative increase without naming the reference, and 14 stated the cost of the mechanism as an absolute amount, such as gas per on-chain transaction [41], bytes of trusted storage [99] or the SRAM held by a blockchain ledger on an RP2040 [51].

Communication overheads ranged from none to fixed or relative additions per round. Differential privacy left the upload per round unchanged (234 kB with and without it [96]; 80 MB for a DP-FedAvg baseline against 80.2 MB for FedAvg [98]), the TrustZone-backed protocol added no bytes in its active phase and 2,784 to 5,600 bytes for dropout recovery [99], blockchain layers added a few kilobytes [80] or 180 to 220 bytes per round [41] in simulation, and secure aggregation added 10 to 15% of airtime per round [97]. Latency rose by 4.2% with local differential privacy [96] and by about 15% (12 ms) per round with secure aggregation [97], the active-phase latency of the TrustZone protocol by 44 and 139 ms for a small and a medium CNN [99], and each on-chain validation took about 0.11 s in simulation [41]. The active-phase energy per round rose by 257.9 mJ with the TrustZone protocol (576.5 against 318.6 mJ, measured) [99], a DP-FedAvg baseline used 328 J per round against 312 J for FedAvg [98], and an analytical estimate in simulation gave 2.1 J against 1.95 J for pairwise masking [77]. Computation and memory were reported only as absolute costs: 65,618 gas per on-chain transaction [41], 44,477 μs per mask-generation call in the trusted execution environment for the small model, and 1,016 B of persistent Secure World storage, 18 kB of runtime memory and a 48 kB trusted-application binary [99]. On RP2040 microcontrollers, the three costed mechanisms of EdgeTrust-Shard were reported as absolute amounts: a 24 kB SRAM ledger, 8 kB for software ECDSA, and under 2 ms per round for the anti-centralization checks [51]. Its optimized hashing took 100 ms per round for 1,000 operations against 338 ms for the baseline C implementation and used 22 mJ of cryptographic energy per round. Its consensus latency of 187 ms per round is a total rather than an increment over FL without the mechanism [51].

How the overheads were obtained differs (Fig. 18): 6 observations were measured, one of them on a simulated chain [41], 1 computed exactly and 1 estimated, and 16 were reported without a stated basis (Table 20). Of the 24, 19 come from the 5 studies evaluated, by each study's own setting, physically or in hybrid setups and 5 from the 3 simulations, and the 5 measured on physical hardware come from two studies, on Rock Pi 4 boards [99] and on RP2040 microcontrollers [51].

Of the 16 mechanisms, 3 were evaluated on an MCU, two fully and one in part, all in EdgeTrust-Shard on RP2040 boards [51], the one study of the 8 that demonstrated MCU feasibility; the other 13 were not, among them the differential-privacy result of the same study, which did not report where it ran [51]. For two studies, the record gives a reason: the blockchain side was run only in simulation [41], and porting the TrustZone protocol to Cortex-M devices is left to future work because of MMU-to-MPU constraints [99]. Outside this set, 3 mechanisms were partly evaluated on MCU clients, Gaussian noise on STM32 nodes [67], and DP-FedYogi [53] and TLS [72] on ESP32-S3 boards, but none reported a cost of the mechanism or a threat model, so none was fully reported (Section 3.4.1).

Discussion of RQ4b The overheads the fully reported studies report are mostly communication and latency, and more than half carry no stated basis. Computation, memory

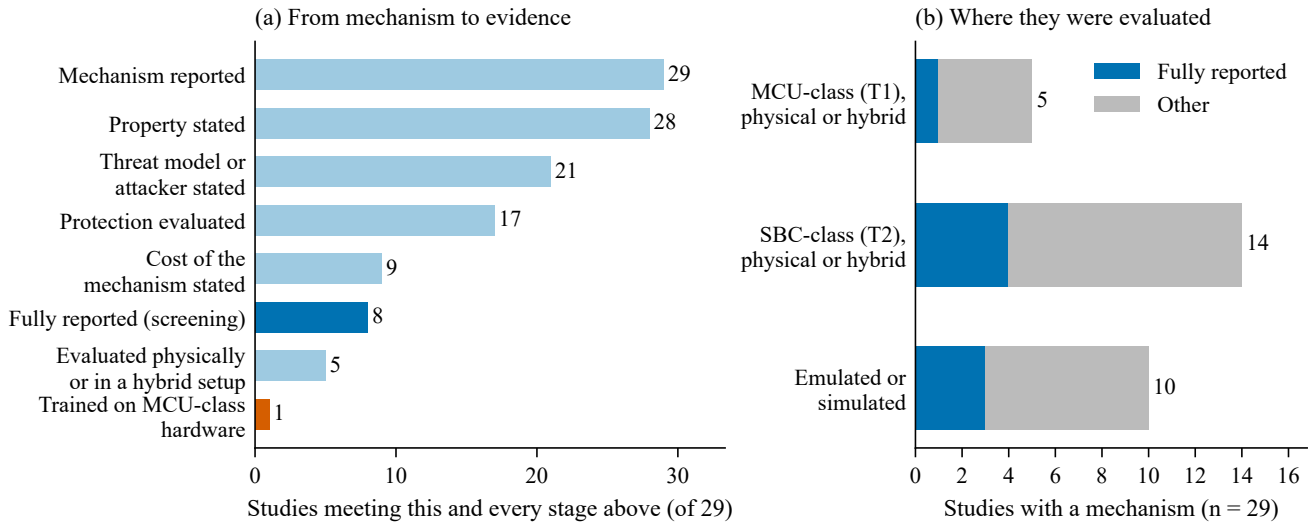


Figure 17: Security and privacy evidence completeness: (a) cumulative evidence stages among the 29 studies reporting a mechanism, each bar counting the studies that meet that stage and every stage above it; (b) evaluation setting of those studies, split into fully reported and other studies.

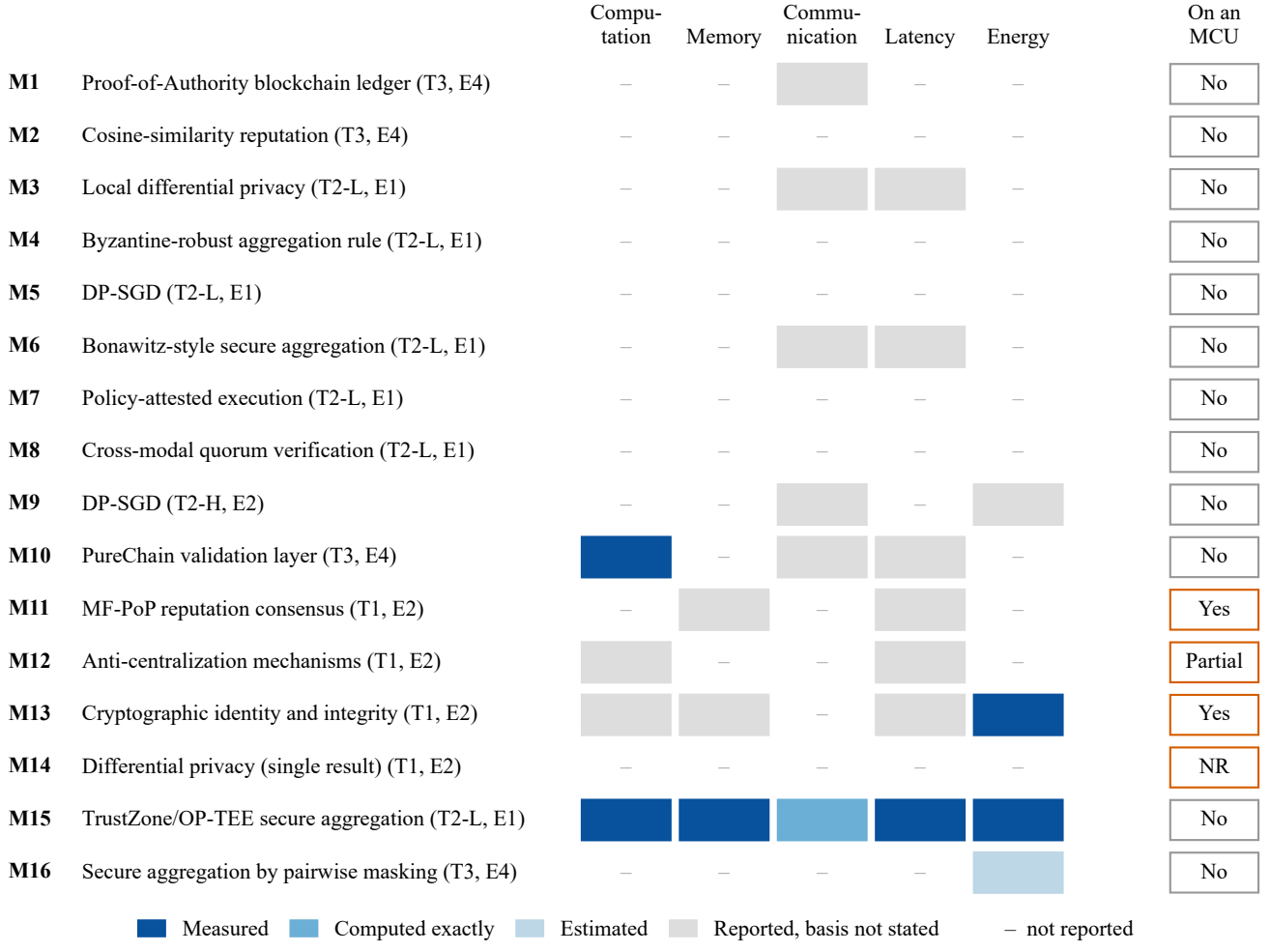


Figure 18: Reported overheads for the 16 mechanisms of the 8 fully reported studies, by resource and evidence basis; the final column indicates whether the mechanism was evaluated on an MCU. Mechanism IDs as in Tables 17 and 18; in parentheses, the study's evaluated tier and mode. Measured includes one value measured on a simulated chain.

Table 18
Client-side overheads reported for the costed security and privacy mechanisms of the 8 fully reported studies; values are shown as reported and are not converted.

ID	Study	Mechanism	Setting	Computation	Memory	Communication	Latency	Energy
M1	[80]	Permissioned Proof-of-Authority blockchain ledger	T3, E4	–	–	a few kilobytes per round (SHA-256 hashes and digital signatures)	–	–
M3	[96]	Local differential privacy	T2-L, E1	–	–	234 kB vs 234 kB upload per round (w/o DP)	4.2% latency overhead	–
M6	[97]	Bonawitz-style secure aggregation	T2-L, E1	–	–	+10-15% WiFi 6 airtime per update round	+15% (approximately 12 ms) per update round	–
M9	[98]	DP-SGD	T2-H, E2	–	–	80 MB vs 80.2 MB upload per round (DP-FedAvg vs FedAvg + YOLOv10)	–	328 J vs 312 J per round (DP-FedAvg vs FedAvg + YOLOv10)
M10	[41]	PureChain blockchain validation layer	T3, E4	65,618 gas per transaction (SetModel, measured)	–	180-220 bytes per round (metadata packet)	≈0.11 s per on-chain validation transaction (PoA2)	–
M11	[51]	Multi-Factor Proof-of-Performance	T1, E2	–	24 kB SRAM blockchain ledger (headers and reputation scores); 12 kB flash validation set (500 INT8 samples)	–	187 ms consensus latency per round (200-node hybrid; total, not an increment over a non-secure baseline)	–
M12	[51]	Anti-centralization mechanisms	T1, E2	under 2 ms per round (adaptive decay 0.3 ms per update, VRF 1.2 ms per rotation check, capping 0.5 ms per round); proof-of-work onboarding approximately 15 s on RP2040, one-time per new node	–	–	under 2 ms per round (1.1% of 187 ms consensus latency)	–
M13	[51]	Cryptographic identity and integrity	T1, E2	SHA-256 99.2 μs per block optimized (338.5 μs baseline C)	8 kB software ECDSA footprint	–	100 ms per round for 1000 hash operations (optimized; 338 ms baseline)	22 mJ per round cryptographic energy (optimized; 50 mJ baseline); 22 μJ per hash measured
M15	[99]	TrustZone/OP-TEE-backed secure aggregation	T2-L, E1	44477 μs per GENERATE_MASK call (small CNN), 137730 μs (medium CNN); idle phase 31 DH operations ≈280 ms per epoch	1016 B persistent Secure World storage (N = 32); 18 kB runtime; 48 kB TA binary	No extra active-phase bytes vs plaintext FedAvg; recovery 2,784 bytes (k = 3), 5,600 bytes (k = 7)	+44 ms (small CNN) and +139 ms (medium CNN) vs B1 Plaintext FedAvg	+257.9 mJ per round active phase vs B1 (576.5 vs 318.6 mJ), measured
M16	[77]	Secure aggregation by pairwise additive masking	T3, E4	–	–	–	–	2.1 J vs 1.95 J per FL round (with vs without privacy masking; analytical estimate)

Values verbatim from the extraction. Unit of analysis: mechanism; ID as in Table 17. Setting: the study's evaluated tier and mode; –: not reported. Fig. 18 summarizes the evidence basis of each overhead.

and energy, which decide whether a mechanism fits a constrained client, were reported for few mechanisms and measured on physical hardware in two studies. For differential privacy, the reported cost lay in task performance, latency and energy rather than in transmitted bytes: accuracy fell from 98.612% to 98.347% and latency rose by 4.2% [96], F1 fell by 1.9 points [97] and mAP@0.5 by 1.1% in one study's ablation, where a DP-FedAvg baseline also used 328 J per round against 312 J [98], whereas, where it was reported, the upload per round did not grow.

Whether security and privacy mechanisms are feasible on MCU-class devices is therefore demonstrated by one study. The one protocol whose costs were reported for all five resources, four of them measured, ran on an SBC with a TrustZone trusted execution environment, and its port to Cortex-M devices is left open [99]; EdgeTrust-Shard ran its reputation consensus and cryptographic layer on RP2040 clients within 140 kB of the 264 kB SRAM and reported their costs, some as totals rather than increments over FL without them [51]; the other mechanisms that ran partly on MCU clients were not costed. Demonstrating feasibility requires running the mechanism on the MCU and measuring the memory, computation, latency and energy it adds against the same FL without it.

3.5. RQ5: Benchmarking and reproducibility

3.5.1. *RQ5a. To what extent are empirical results in FL for resource-constrained IoT and TinyML devices comparable across studies with respect to datasets, models, evaluation metrics, hardware, communication protocols, and experimental baselines?*

Eligibility required each included study to report the dataset, model, evaluation metrics, hardware and baseline (IC8, Section 2.6), and the extraction records all five for 89 of the 93 studies; in the other 4, the dataset and metric (1), the model (2) or the evaluated hardware (1) of the primary experiment is not reported or unclear. The question is therefore whether these dimensions coincide across studies (Table 19). Sixteen public datasets were used by two or more studies, led by CIFAR-10 (22 studies), MNIST (8) and Fashion-MNIST and UCI HAR (7 each), and 43 studies used at least one of them; the other 50 shared none. A named standard architecture appeared in 25 studies, and 18 shared one with another study, most often AlexNet or VGG-16 (6 studies each). Of the 74 studies evaluated physically or in hybrid setups, 70 identify their device model, most often the Raspberry Pi 4 (22 studies), the Jetson Nano (16), the ESP32 (12) and the Jetson TX2 (8), and 66 share a device model with another study. FedAvg was among the named baselines in 38 studies, the other baselines ranging from other FL algorithms to centralized training and unoptimized models (Section 3.2.2). The protocol carrying the FL traffic was reported by 53 studies, most often Wi-Fi as the link (21) and MQTT as the application protocol (12). CoAP, the application protocol standardized for constrained RESTful environments, was rarely used; one MCU study exposes its models as CoAP resources over an OpenThread IPv6 network [65].

Comparability narrows quickly when the dimensions are combined (Fig. 19). Of the 4,278 pairs of included studies, 297 share a dataset, 39 also share a named architecture, 30 also the evaluation setting, and 8 also name FedAvg as a baseline and report accuracy. These 8 pairs involve 5 studies, all evaluated physically on SBC-class devices [100, 101, 81, 40, 102], and because datasets and architectures are matched at the study level, only 3 of the pairs train the same model on the same dataset: AlexNet or VGG-16 on CIFAR-10, and ResNet-18 on CIFAR-100. Of the 26 studies that trained on MCU-class hardware (Section 3.1.1), only two pairs share a public dataset: MNIST [58, 63], with different models and tasks (anomaly detection and classification), and TON_IoT [66, 51], with different models (two feedforward networks with two hidden layers, one for ten classes and one a smaller INT8-quantized network for two classes); otherwise the closest are three studies by one group that use overlapping sets of the group's own keyword recordings [12, 39, 60], and two by another group that pose a synthetic sine-wave regression task of the same form with fully connected networks of different size [61, 69].

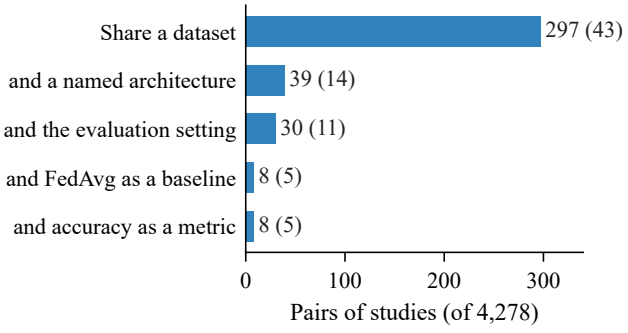


Figure 19: Pairwise cross-study comparability as successive dimensions are matched; bars show comparable study pairs (of 4,278; each stage requires those above), and labels in parentheses give the studies involved. Evaluation setting: the same evaluated tier (T1, T2 or T3), with both studies evaluated physically or in hybrid setups (E1, E2) or both in simulation or emulation (E3, E4). Datasets and architectures are compared at the study level, so the counts are upper bounds; Table 19 details the dimensions.

Where the same metric is reported, its scope often differs. Communication was reported per round by 28 studies and as total volume by 9, all 9 of which also report it per round; computation changes were expressed as time in 31 of 43 studies and as operation counts in 5; and memory was reported as peak training memory by 26 studies and as peak inference memory by 11, only 3 reporting both. Even the training figure differs in scope: one MCU study reports the static RAM footprint of its client executable [65], another the total runtime SRAM allocation of the node, including its ledger and network buffers [51]. In some studies, energy was reported as power draw [56, 45] or estimated from CPU thermal design power [48], and in one MCU study it was

Table 19
Cross-study comparability among the 93 included studies across datasets, architectures, baselines, hardware, communication protocols, metrics and experimental configuration: what the studies report and how far it is shared.

Dimension	What the included studies report	How far it is shared
Dataset and task	16 public datasets used by two or more studies, led by CIFAR-10 (22), MNIST (8), Fashion-MNIST (7) and UCI HAR (7)	**43** studies use a shared dataset
Model architecture	Named standard architecture in 25 studies; shared: AlexNet (6), VGG-16 (6), ResNet-18 (5), MobileNetV2 (4), LeNet (2), LeNet-5 (2), LightGBM (2), MobileNetV3 (2), ResNet-50 (2)	**18** studies use a shared named architecture
FL algorithm and baseline	FedAvg among the baselines in 38; baseline types: another FL algorithm (63), another reference such as centralized or local training (51), a different or unoptimized model (40)	Baselines differ in kind across studies
Hardware, tier and mode	Evaluated physically or in hybrid setups in 74, device model identified in 70: Raspberry Pi 4 (22), Jetson Nano (16), ESP32 (12), Jetson TX2 (8)	**66** studies share a device model
Communication protocol	Reported in 53; link: Wi-Fi (21), LoRa or LoRaWAN (6); application protocol: MQTT (12), HTTP(S) (9), TCP or UDP sockets (7), serial or UART (6)	Not reported in **40**
Task metric	Accuracy in 75; F1 (30), precision and recall (19)	**75** studies report accuracy
Resource metrics and scope	Communication per round in 28, total in 9 (all 9 also per round); peak training memory in 26, peak inference memory in 11 (both in 3)	Same names, different scopes
Experimental configuration	Rounds in 68, partition scheme in 63, client population in 93; complete configuration (the four Q1 items, from the paper or its repository) in 27	Q1 Yes 26, Partial 35, No 32

Unit of analysis: study. Dataset, architecture and device names are normalized across studies (e.g., HAR as UCI HAR); datasets used only for pre-training, and networks built from blocks of a standard architecture, are not counted. Q1: reporting of the FL setup (Table 6). Fig. 19 combines the dimensions.

measured with a current sensor and reported as energy per round and a projected battery life [51]. Task performance was mostly measured as accuracy (75 studies), with F1 in 30. The configuration needed to rerun an experiment was reported unevenly: the number of rounds in 68 of the 93 studies and the partition scheme in 63 (Section 3.2.2 gives these counts for the 89 studies that report a change), whereas every study stated the client population; the quality assessment judged the reporting of the FL setup (Q1) Yes in 26 studies, Partial in 35 and No in 32, and comparator validity (Q3) Yes in 40, Partial in 48 and No in 5.

Discussion of RQ5a These results suggest that the included studies share a few benchmarks but rarely experiments: 43 studies use a shared dataset, but once the model, setting, baseline and metric are also required, comparable results reduce to 8 pairs, all on SBC-class hardware and only 3 with the same model on the same dataset, and only two pairs of studies that trained on MCU-class hardware share a public dataset, each with different models. Completeness of these five dimensions is a condition of inclusion (IC8) rather than a finding, and the 276 eligible reports excluded for insufficient data (Section 3.1.1) indicate that missing reporting is common in the wider literature; among the studies that report all five, comparability is limited instead by the absence of shared choices and of a common scope for resource metrics.

A benchmark for FL on constrained devices would therefore need to fix more than the dataset: a reference model, a baseline such as FedAvg, the client population and partition, and resource metrics with a stated scope (bytes per round and in total, peak training memory, energy per round with its measurement method), run on named MCU- and SBC-class boards. The authors of one included MCU study make the same point: their evaluation did not compare directly with existing methods because, in their words, few approaches were comparable and platform implementations varied, and they released their code and evaluation tools so that standardized benchmarks could follow [65]. Such a benchmark is the precondition for the cross-study comparisons that RQ2b and RQ2c could not make (Sections 3.2.2 and 3.2.3).

3.5.2. RQ5b. What reproducibility artifacts are publicly released by primary FL studies for resource-constrained IoT and TinyML devices?

Few studies release what would let others rerun their experiments (Fig. 20). Public source code was found for 18 of the 93 studies and executable scripts for 17, whereas the dataset was obtainable for 78, the hardware specification complete for 50 and the configuration complete for 27. The 50 follow the quality-assessment criterion, under which every training client is identified by model, with its processor and memory stated or given by the model's datasheet; the extraction coding of Section 3.1.1, which asks whether the report itself identifies the devices and their configuration, counts 14. Of the other items, the measurement procedure was complete, or not needed, for 10, and random seeds were reported by 5. Code counts as public only when a link printed in the report was opened and holds the FL training code the paper describes, and scripts only when such a link holds them; a third-party repository [45], a release only planned [44] or promised [48, 51], and scripts retained without a printed link [55] therefore do not count. The assessment logged 115 printed links in 58 studies: 43 were opened, 38 on code hosts and 5 on dataset pages; 35, most of them dataset pages or DOIs (22), were blocked by the network policy of the assessment environment; 4 could not be fetched in the later assessment of 15 studies; and 33 were not opened because they pointed to third-party software, vendor or dataset pages, cited papers or other web

pages, or misprinted a link that was opened. No link blocked by the network policy pointed to a study's own code, so that blocking affected only dataset judgments, for which a named public benchmark counts as obtainable without its link being opened; one study's own repository could not be fetched in the later assessment, so its code does not count as public [49].

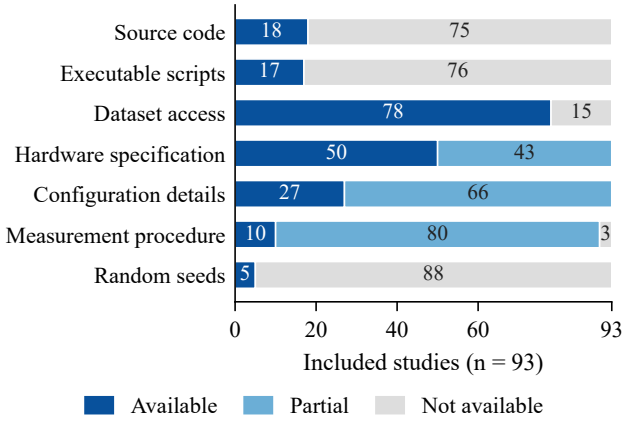


Figure 20: Publicly available reproducibility artifacts and reporting items across the 93 included studies, classified as available, partial or not available by the quality-assessment criteria (Section 2.8); code and scripts count as available only when the printed link was opened and holds them.

The reproducibility levels are cumulative: R1 reports the configuration or the hardware, at least in part; R2 adds the core configuration (aggregation rule and model architecture), a complete hardware specification and an obtainable dataset; R3 adds public FL training code and the complete configuration; and R4 adds a complete measurement procedure and random seeds. R0, which reports neither the configuration nor the hardware, cannot occur among included studies; 50 studies reached R1, 40 R2 and 3 R3 [12, 39, 65], and none R4; reproducibility (Q5) was judged Yes in 3 studies, Partial in 50 and No in 40.

Each level is held back by one or two missing items. Every R1 study lacked a complete hardware specification (43) or an obtainable dataset (15), 8 both; of the 40 at R2, 35 released no FL training code and the 5 that did, among them TDMiL [63], lacked the complete configuration; and all three R3 studies lacked a complete measurement procedure, two also the random seeds. Public code alone was not enough: 15 of the 18 studies with public code stayed below R3.

Discussion of RQ5b Comparability and reproducibility rarely coincide: of the 14 studies that share a dataset and a named architecture with another study (Section 3.5.1), 3 release their code, 15 studies release code without sharing a dataset and an architecture with any other, and 64 have neither. Release practice in the included studies thus provides neither the common experiments needed to compare results nor the artifacts needed to rerun them.

For FL on constrained devices, a reproducibility package needs more than code: the complete hardware specification of every client board, the full FL configuration (clients, rounds, partition, learning rate and batch size), the measurement procedure behind each resource figure and the random seeds. The 90 studies below R3 lack public FL training code or a complete hardware specification, dataset or configuration, and the three at R3 lack a complete measurement procedure.

3.6. Quality of the synthesized evidence

Table 20 reads the principal quantitative statement of RQ2b, RQ2c, RQ3a and RQ4b against the quality assessment of the studies behind it (Section 2.8). Q2 was rated Yes for most contributing studies, but Q4, which requires every outcome behind the claim to be measured on the hardware that ran it with a stated procedure or computed exactly (Table 6), was rated Yes for 9 of the 80 studies behind the RQ2b statement, 8 of the 66 behind RQ2c, none of the 10 behind RQ3a and 1 of the 8 behind RQ4b. A score at the top of the range does not guarantee a Yes on Q4: one R3 study that trained on Thunderboard Sense 2 boards and released its code and data takes its memory footprints from a stated toolchain procedure but states only where, not how, its communication cost was counted [65]. Restricted to the 26 studies scoring at least 3.5, the RQ2b statement holds, whereas the MCU-class cases with little headroom are 2 of the subset's 4, no longer fewer than half. The RQ2c contrast keeps its direction but narrows, from 25.0% against 7.9% to 16.7% against 7.4%, on 18 and 54 changes. The RQ4b statement does not hold: 5 of the subset's 7 overhead observations come from the one study in the subset that measured its overheads [99]. The other study with an overhead measured on physical hardware, on RP2040 boards, scores below the threshold [51]. Quality therefore qualifies rather than overturns these findings: the direction of the reported changes holds among the higher-scoring studies, whereas the share of MCU-class cases with thin memory headroom rests on four higher-scoring cases, and the size of the cross-tier contrast and the unstated basis of security overheads rest mainly on lower-scoring evidence.

Table 20
Quality of the evidence behind the principal quantitative statement of RQ2b, RQ2c, RQ3a and RQ4b, and the sensitivity analysis on the studies scoring at least 3.5 of 5.

		Contributing studies				Result		
RQ	Statement	All	Q2 Yes	Q4 Yes	≥3.5	All studies	Studies scoring ≥3.5	Holds
RQ2b	Resource changes favorable to the technique	80	60	9	25	194 of 230 (84.3%)	63 of 72 (87.5%)	Yes
RQ2c	Unfavorable resource changes, physical or hybrid T1 vs T2	66	60	8	25	15 of 60 (25.0%) vs 10 of 126 (7.9%)	3 of 18 (16.7%) vs 4 of 54 (7.4%)	Direction only
RQ3a	MCU-class cases with a fit ratio of at most 1.60 (Table 15)	10	9	0	4	4 of 10	2 of 4	No
RQ4b	Overhead observations without a stated basis	8	5	1	2	16 of 24 (66.7%)	2 of 7 (28.6%)	No

Contributing studies: All: studies behind the statement; Q2 Yes and Q4 Yes: studies rated Yes on hardware and evaluation validity and on measurement and outcome reporting (Table 6); ≥3.5: studies scoring at least 3.5 of 5 (Appendix Table A.4). Result: the statement recomputed on all contributing studies and on those scoring at least 3.5; units are resource changes (RQ2b, RQ2c), MCU-class cases (RQ3a) and overhead observations (RQ4b).

4. Open challenges and future research directions

Read together, the answers to RQ1–RQ5 share one pattern: the evidence thins as the device tightens. FL training ran on MCU-class hardware in a minority of the included studies (Section 3.1.1), but on these devices the binding resources were the least measured (Section 3.2.3), only one fully reported security mechanism study was evaluated on them (Section 3.4.2), the MCU-side software stack was assembled study by study (Section 3.3.2), and only two pairs of these studies share a public dataset (Section 3.5.1). FL has thus been shown to run on microcontrollers, but rarely at scale, seldom with its memory, energy and communication measured, with security costed in a single study, and not comparably across studies. The directions below address these gaps (Table 21), and Table 22 proposes a minimal benchmark and reproducibility package.

4.1. Demonstration on the targeted device tier, at scale

Claims of MCU-class suitability still outrun MCU-class evidence (Table 21): some MCU-targeting studies did not train on an MCU, and eligible excluded reports that target MCUs were evaluated on MCU-class hardware less often than the included studies (Section 3.1.2). Physical MCU-class deployments were small and seldom fully specified, and an unidentified deployment target was the most frequent reason for exclusion at full text (Section 3.1.1). A few boards show that training runs, not how a federation behaves as clients, data and links multiply. Studies should name the targeted device, train on it and scope each claim to what the evaluation platform supports; move from three-board demonstrations to tens or hundreds of MCU clients, which deployments of 50 nRF52840 boards [54] and 30 STM32F746 boards [67] show to be practicable; and specify each client's board, memory, radio and firmware.

4.2. Controlled cross-tier evaluation of optimization techniques

Whether savings shown on SBCs or in simulation carry over to MCUs remains open (Section 3.2.3). Compression, the most common primary strategy, was rarely demonstrated by training on physical MCU-class devices (Section 3.2.1). Unfavorable resource changes were more frequent on MCU-class than on SBC-class hardware, a contrast that narrows among the higher-scoring studies (Section 3.6), yet few studies combined an MCU with an SBC (Section 3.2.3), and the primary-configuration changes synthesized here cannot show whether a technique keeps its benefit across tiers. The baseline also shapes the verdict: task performance mostly improved against baselines that include another FL algorithm but declined in 11 of the 25 studies compared only with other references (Section 3.2.2). What is needed is a controlled design: one technique, workload, partition and number of rounds, run on an MCU, an SBC and a simulation host, compared with both FedAvg and a centralized or unoptimized reference, with each resource measured on each tier and the trade-offs reported beside the savings.

4.3. Measurement of the resources that bind MCUs

The resources that limit MCU clients are measured least on them (Section 3.2.3; Table 21). Fit was more often argued from model or update size than from peak training memory, whose components were almost never reported, although several of the MCU cases that read training memory against the device left little headroom (Section 3.3.1). Communication, the most frequent target, was rarely measured (Section 3.2.1), and more than a quarter of the energy changes were estimates (Section 3.2.2). Three practices would close these gaps: training memory measured on the device and separated into weights, gradients, activations, optimizer state and runtime overhead, read against SRAM and flash; energy per round measured with a stated instrument, separating computation from radio and training from inference, and reported as energy rather than power; and communication counted as bytes on the actual link, per

Table 21
Research directions derived from RQ1–RQ5, showing the evidence gap, the action that would close it and why the change matters for deployment.

Direction	Gap in the included evidence	What would close it	Why it matters for deployment
Demonstration on the targeted tier, at scale (RQ1)	9 of 35 MCU-targeting studies did not train on an MCU; median of 3.5 boards in physical MCU deployments; hardware fully specified in 6 of 26	Training on the named target board, with tens to hundreds of clients and a complete specification of each	Choosing a method for a device class requires evidence that it trains on that class
Controlled cross-tier evaluation (RQ2)	Unfavorable resource changes: 15 of 60 on MCUs, 10 of 126 on SBCs; 11 studies combined an MCU with an SBC	One technique and workload on an MCU, an SBC and a simulation host, against FedAvg and a centralized reference	Shows whether savings measured on SBCs or in simulation transfer to MCUs
Measurement of binding resources (RQ2, RQ3a)	On physical MCU deployments, memory changes measured in 4 studies, energy changes in 4, communication changes in 3; optimizer state reported by 1, activation memory by none	Training memory by component against SRAM and flash; measured energy per round, computation and radio separated; bytes on the link	With headroom of at most 1.60 times in 4 of 10 MCU cases, an unmeasured component can decide fit
MCU-native training stack (RQ3b)	TFLM ran only inference in 4 of 6 MCU uses; 23 of 26 studies wrote their own FL client	Open runtime with backpropagation, portable kernels and an FL client layer; runtime, version and modifications reported	Without it, each deployment is rebuilt and difficult to repeat
Costed security on MCU clients (RQ4)	8 of 29 mechanism studies fully reported, one evaluated on an MCU; 16 of 24 overheads without a stated basis	Differential privacy and secure aggregation run on MCUs and costed with and without them; poisoning and tampering evaluated	Protection that exceeds the client's budget cannot be deployed
Benchmark and reproducibility (RQ5)	8 comparable pairs, all on SBCs; a public dataset shared by only two pairs of the 26 MCU-trained studies; 3 studies at R3, none at R4	Minimal suite and graded package (Table 22)	Allows results to be compared across studies and rerun

Counts are studies unless stated; the sections cited in the text give their denominators and sources.

round and to a target accuracy. Memory is also open to method design: recomputing activations from checkpoints and updating weights in place during the backward pass cut peak training RAM from a theoretical 412 kB to 124 kB on an STM32F746 [67], and sparse updates with compile-time differentiation enabled CNN training within 256 kB of SRAM outside FL [9]; whether sparse updates of this kind also work in a federated client, and reduce what it uploads, remains to be measured.

4.4. An MCU-native training and federation stack

MCU-native FL rests on software assembled study by study (Section 3.3.2). TFLM mostly ran inference rather than training on the MCUs where it was used, one study wrote its own training code because TFLM does not implement automatic differentiation [70], most studies that trained on MCUs implemented the FL client themselves and several did not name what ran training, and one study had to modify its training library (Genann) so that the same code ran on Arduino, Raspberry and Espressif microcontrollers [62]. An open MCU training runtime is therefore a research direction in its own right: backpropagation for the layer types FL models use, memory planning for the backward pass, kernels portable across MCU families, and a client layer that exchanges updates over Wi-Fi and MQTT, the link and protocol most often reported across the included studies (Section 3.5.1), and over low-bandwidth links such as LoRa. It would be validated by running one FL workload unchanged on boards from two or more MCU families; meanwhile, studies should name the runtime, its version and role, and any modification.

4.5. Costed and threat-modeled security on MCU clients

Security has rarely been demonstrated where it is hardest to afford (Sections 3.4.1 and 3.4.2). Few of the studies reporting a mechanism reported it fully, with its property, threat model and client-side cost (Table 21); one of these was evaluated on an MCU, and the other studies that trained on MCUs reported no cost for their mechanisms. More than half of the reported overheads carry no stated basis, a share that falls among the higher-scoring studies (Section 3.6); measurements on physical hardware came from two studies: one on Rock Pi 4 boards with a TrustZone trusted execution environment, whose port to MCU-class devices is left to future work [99], and one on RP2040 microcontrollers, where the energy of a cryptographic identity layer was measured [51]. The threat most often modeled was inference from updates, followed by poisoning and tampering (Section 3.4.1). Three directions follow: running differential privacy and secure aggregation on MCU clients and measuring the memory, computation, latency and energy they add against the same FL without them; evaluating protection against poisoning and tampering, empirically or formally; and porting trusted-execution designs to the memory protection of microcontrollers. Because the reported cost of differential privacy lay in task performance, latency and energy rather than in

Table 22
Proposed minimal benchmark and reproducibility package for FL on constrained devices, with the basis for each specification in the included studies.

Component	Proposal	Basis in the included studies
Tasks and datasets	Keyword spotting (Google Speech Commands), image classification (CIFAR-10) and human activity recognition (UCI HAR)	MLPerf Tiny's keyword-spotting and image-classification benchmarks use Google Speech Commands and CIFAR-10 [15]; CIFAR-10 used by 22 studies, UCI HAR by 7; three MCU studies by one group used overlapping sets of the group's own keyword recordings
Model scale	MCU scale: a reference model whose measured peak training memory fits in 256 kB of SRAM; SBC scale: ResNet-18 or MobileNetV2	In the 10 MCU cases of Table 15, peak training memory from 4.8 kB to 320 kB against 80 kB to 520 kB of SRAM; CNN training within 256 kB shown outside FL [9]; ResNet-18 used by 5 studies, MobileNetV2 by 4
Device targets	One board from each of the ESP32, Arduino and STM32 families, with SRAM, flash, clock and radio stated; a Raspberry Pi 4 or Jetson Nano as the SBC reference	ESP32-family modules in 11, Arduino boards in 6 and STM32 microcontrollers in 4 of the 26 studies that trained on MCUs; Raspberry Pi 4 in 22 studies, Jetson Nano in 16
FL configuration and baselines	FedAvg and a centralized reference; a fixed client population; an IID and a named non-IID partition; rounds, local epochs, batch size and learning rate; repeated runs with stated seeds	FedAvg named among the baselines in 38 studies; the direction of performance changes differed by type of baseline (Section 3.2.2); 58 of 89 studies evaluated non-IID data
Communication	Wi-Fi with MQTT, plus one low-bandwidth link such as LoRa; bytes counted on the radio link rather than over a serial connection to a host	Wi-Fi in 21 studies, MQTT in 12, LoRa or LoRaWAN in 6, serial or UART in 6
Required metrics	Accuracy and F1 on a stated test set; bytes per round and in total to a target accuracy; peak training memory by component and flash use; time per round on the device; measured energy per round, computation and radio separated	Communication per round in 28 studies, in total in 9; peak training memory in 26, peak inference memory in 11, both in 3; computation as time in 31 of 43 studies, as operation counts in 5
Reproducibility package (R1–R4)	Public FL training code and executable scripts; an obtainable dataset and partition; a complete specification of every client board; the complete configuration; the measurement procedure behind each resource figure; random seeds; and, beyond the R levels, the firmware toolchain with runtime version and modifications (Section 3.3.2)	Of the 93 studies: public code 18, scripts 17, obtainable dataset 78, complete hardware specification 50 (quality-assessment criterion), complete configuration 27, measurement procedure complete or not needed 10, seeds 5

Bases from Sections 3.1.1, 3.2.2, 3.3.1 and 3.5 and Table 15. The R levels are those of Section 3.5.2.

transmitted bytes (Section 3.4.2), its privacy budget and accuracy loss should be reported together.

4.6. Shared benchmarks and reproducibility packages

Results can rarely be compared across studies, and few can be rerun (Section 3.5). Very few pairs of studies share a dataset, a named architecture, the evaluation setting, a baseline and a metric, none of them on MCU-class hardware, and resource metrics share names but not scopes (Section 3.5.1). Public FL training code was rare, and no study released a complete reproducibility package (Section 3.5.2; Table 21). MLPerf Tiny shows what a shared benchmark can fix: it comprises keyword spotting, visual wake words, image classification and anomaly detection, and measures the accuracy, latency and energy of inference [15], whereas FL adds on-device training, communication and aggregation. Table 22 extends this approach to federated training with a minimal suite built from choices the included studies share where they exist (CIFAR-10, UCI HAR, FedAvg, Wi-Fi with MQTT) and completed from MLPerf Tiny, and a reproducibility package graded by the R levels of Section 3.5.2. The suite would enable the cross-study comparisons that RQ2b and RQ2c could not make and, run on both tiers, cross-tier ones; the package would let them be checked.

5. Limitations of this review

The review has four main limitations. First, the search was limited to peer-reviewed studies in English published from 2019 onward (Section 2.4), and the planned Google Scholar and citation searches were not run, because Scopus and the six publisher libraries were expected to cover the field broadly and further sources to add mainly duplicates. Second, full texts were obtained from open-access sources, through institutional access or as arXiv preprints (Section 2.2), but 267 of the 1,631 reports sought, most of them subscription articles beyond the institution's access, could not be obtained by any of these routes; three included studies were assessed from their preprints. Third, the synthesis relies on figures as reported by the primary studies, which may be affected by publication bias toward favorable results. Fourth, screening, data extraction and quality assessment were LLM-assisted, with the first author approving the results (Sections 2.2, 2.7 and 2.8); the validation samples reported in Section 2.2 estimate how often these decisions were wrong, but cannot rule out isolated errors in records that no author read.

6. Conclusion

This review asked how far federated learning has been demonstrated on the resource-constrained IoT and TinyML devices it is proposed for, and on what evidence. Across the 93 included studies, the answer depends on the device tier. FL has been shown to train on microcontrollers: 26 studies did so, and the evaluated hardware matched the targeted tier in 76 of the 93. The evidence on MCU-class devices is nevertheless narrow. It rests mostly on deployments of a few boards that are seldom fully specified; the resources that decide whether training fits a microcontroller (peak training memory with its optimizer and activation components, energy and transmitted bytes) were rarely measured on the devices themselves; the MCU-side software stack was assembled study by study; one of the eight studies with a fully reported security or privacy mechanism was evaluated on an MCU; and only two pairs of MCU studies share a public dataset.

Most reported optimization gains are therefore demonstrations on SBC-class devices or in simulation, measured against baselines and scopes that differ between studies. They establish that techniques such as compression and communication-efficient training reduce resource use where they were evaluated, but not by how much on the constrained devices the field targets, nor whether savings measured on more capable hardware carry over to them.

Closing this gap calls for stronger evidence more than for further methods: training on the named target board at realistic scale, peak memory and energy measured by component on the device, security mechanisms costed on MCU clients, and shared tasks, datasets and reproducibility packages that make results comparable and repeatable. The research directions and the minimal benchmark of Section 4 are offered as a starting point. Within the limits set out in Section 5 (the search window and sources, the reports that could not be retrieved and the reliance on figures as reported), these conclusions describe the published evidence rather than the capabilities of FL itself, and only the best-documented part of that evidence: the eligible reports excluded for insufficient data trained on MCU-class hardware far less often than the included studies (Section 3.1.1), so the field as a whole is likely weaker on these measures.

Declaration of competing interest

The authors have no relevant financial or non-financial interests to disclose.

Data availability

The review protocol (version 1.4) with its amendment and deviation log, and the review's supporting materials (search strings, screening, extraction and quality-assessment prompts, screening decisions, extraction data, quality-assessment records and the validation samples), are available on Zenodo: https://zenodo.org/records/23192548?preview=1&token=eyJhbGciOiJIUzUxMiJ9.eyJpZCI6ImRjYTc5OGI4LWZhNDItNDIyNi1hMDY5LTA5Y2ZhYTNlZTYyNyIsImRhdGEiOnt9LCJyYW5kb20iOiIwNThkMzYyN2U4NDkyNDlhMjFlYjMzOTJiODNlNDI5NCJ9.aicLuaJdUfJ6YuV3Qfa2t-_BINuIKVtZCUyd5udwDxK2G-PXelEsWzfzb78SDIu_YEA3ao7LKEHeUKxFAQh8GQ.

A. Search strings

Table A.1
Complete search strings per source. Rows of field-based forms were combined with AND.

Source	Fields searched	Search string	Records
Scopus	Title, abstract, keywords	TITLE-ABS-KEY (("federated learning" OR "federated machine learning" OR "federated deep learning" OR "federated transfer learning" OR "federated edge learning" OR "federated meta-learning" OR "embedded federated learning" OR "on-device federated learning" OR TinyFL OR "tiny federated learning" OR "swarm learning" OR "gossip learning") AND ((microcontroller* OR MCU OR TinyML OR "tiny machine learning" OR "sensor node*" OR "wireless sensor network*" OR WSN OR ESP32 OR STM32 OR "Cortex-M*" OR nRF52* OR nRF53*) OR ((IoT OR "Internet of Things" OR "edge device*" OR "edge computing" OR "edge AI" OR "edge intelligence" OR "embedded system*" OR "embedded device*" OR "embedded AI" OR "single-board computer*" OR "Raspberry Pi" OR Jetson OR "on-device learning" OR "on-device machine learning") AND ("resource-constrained" OR "resource constrained" OR "resource-limited" OR "resource limited" OR "low-resource" OR "low resource" OR "low-power" OR "low power" OR "ultra-low-power" OR "ultra low power" OR "memory-constrained" OR "memory constrained" OR "energy-constrained" OR "energy constrained" OR "compute-constrained" OR "computation-constrained" OR "embedded machine learning" OR "on-device training")))) AND PUBYEAR > 2018 AND (LIMIT-TO (DOCTYPE , "ar") OR LIMIT-TO (DOCTYPE , "cp")) AND (LIMIT-TO (LANGUAGE , "English")) AND (EXCLUDE (SRCTYPE , "d"))	2,000*
IEEE Xplore	All Metadata (prefixed to every term)	("federated learning" OR "federated machine learning" OR "federated deep learning" OR "federated transfer learning" OR "federated edge learning" OR "federated meta-learning" OR "embedded federated learning" OR "on-device federated learning" OR TinyFL OR "tiny federated learning" OR "swarm learning" OR "gossip learning") AND ((microcontroller* OR MCU OR TinyML OR "tiny machine learning" OR "sensor node*" OR "wireless sensor network*" OR WSN OR ESP32 OR STM32 OR Cortex OR nRF52* OR nRF53*) OR ((IoT OR "Internet of Things" OR "edge device*" OR "edge computing" OR "edge AI" OR "edge intelligence" OR "embedded system*" OR "embedded device*" OR "embedded AI" OR "single-board computer*" OR "Raspberry Pi" OR Jetson OR "on-device learning" OR "on-device machine learning") AND ("resource-constrained" OR "resource-limited" OR "low-resource" OR "low-power" OR "ultra-low-power" OR "memory-constrained" OR "energy-constrained" OR "compute-constrained" OR "computation-constrained" OR "embedded machine learning" OR "on-device training")))	2,467
ACM Digital Library	Abstract (row 1); Anywhere (rows 2–3)	Abstract: "federated learning" OR "swarm learning" OR "gossip learning" OR "federated edge learning" OR TinyFL Anywhere: microcontroller OR MCU OR TinyML OR "sensor node" OR ESP32 OR STM32 OR "Raspberry Pi" OR Jetson OR "edge device" OR "embedded system" OR IoT Anywhere: "resource-constrained" OR "low-power" OR "memory-constrained" OR "on-device training" OR "energy-constrained"	296
ScienceDirect	Not field-restricted; five queries	1: ("federated learning" OR "swarm learning" OR "gossip learning") AND (microcontroller OR MCU OR TinyML) 2: ("federated learning" OR "swarm learning" OR "gossip learning") AND (ESP32 OR STM32 OR "sensor node") 3: ("federated learning" OR "swarm learning" OR "gossip learning") AND ("Raspberry Pi" OR Jetson OR "edge device") 4: ("federated learning" OR "swarm learning" OR "gossip learning") AND ("resource-constrained" OR "low-power" OR "on-device training") 5: ("federated learning" OR "swarm learning" OR "gossip learning") AND ("embedded system" OR "memory-constrained" OR "energy-constrained")	569
SpringerLink	Not field-restricted	("federated learning" OR "swarm learning" OR "gossip learning") AND (microcontroller OR TinyML OR ESP32 OR STM32 OR "sensor node" OR "edge device")	1,000*
Wiley Online Library	Abstract (row 1); Anywhere (rows 2–3)	Abstract: "federated learning" OR "swarm learning" OR "gossip learning" OR "federated edge learning" OR "on-device federated learning" Anywhere: microcontroller OR MCU OR TinyML OR "sensor node" OR ESP32 OR STM32 OR "Raspberry Pi" OR Jetson OR "edge device" OR "embedded system" OR IoT Anywhere: "resource constrained" OR "resource-constrained" OR "low power" OR "low-power" OR "memory constrained" OR "on-device training"	169
Taylor & Francis Online	Anywhere (rows 1–3)	Anywhere: "federated learning" OR "swarm learning" OR "gossip learning" OR "federated edge learning" Anywhere: microcontroller OR MCU OR TinyML OR "sensor node" OR ESP32 OR STM32 OR "Raspberry Pi" OR Jetson OR "edge device" OR "embedded system" Anywhere: "resource-constrained" OR "low-power" OR "memory-constrained" OR "on-device training" OR "energy-constrained"	81
Total			**6,582**

B. Screening validation

Table A.2
Title and abstract screening: agreement between the LLM decision and the authors' blind decision on 200 records drawn at random from the LLM-screened records (150 of the tool's exclusions and 50 of the records it forwarded), screened by two authors, 100 each, without seeing the LLM decisions. Forwarded: include or maybe. Agreement: Clopper–Pearson 95% CI. κ: Cohen's kappa, with a 95% CI from 2,000 bootstrap resamples drawn within the two sampling pools. Weighted: each record weighted by its pool's sampling fraction, giving the agreement expected over all LLM-screened records.

	Human decision		
LLM decision	Forwarded	Excluded	Total
Forwarded	42	8	50
Excluded	3	147	150
Total	45	155	200

Agreement 94.5% (95% CI 90.4–97.2); $\kappa = 0.85$ (95% CI 0.76–0.93). Weighted to all LLM-screened records: agreement 92.5%; $\kappa = 0.84$ (95% CI 0.74–0.92).

Table A.3
Full-text screening: agreement between the LLM decision and the decision that the eligibility rules derive from the authors' blind form answers, on 40 reports drawn at random from the included studies and every exclusion category and checked by two authors, 25 each, 10 in common. The unit is the assessment (checker × report). (a) Whether a deployment target was identified, the judgment behind most exclusions, in the 40 assessments that reached the target question. (b) Inclusion, in the 45 assessments (37 reports) that reached a decision; the resource-fit criterion is read, as the protocol allows, from the reported model size against the named device's capacity, and included means that every eligibility criterion was met. Five assessments in which the retrieved file was a preprint version of a journal article, so that the form stopped at the publication-type question, are left out. Agreement: Clopper–Pearson 95% CI. κ: Cohen's kappa, with a 95% CI from 2,000 bootstrap resamples over assessments.

(a) Deployment target identified

	Human decision		
LLM decision	Identified	Not identified	Total
Identified	32	0	32
Not identified	3	5	8
Total	35	5	40

(b) Inclusion

	Human decision		
LLM decision	Included	Not included	Total
Included	12	5	17
Not included	1	27	28
Total	13	32	45

(a) Agreement 92.5% (95% CI 79.6–98.4); $\kappa = 0.73$ (95% CI 0.38–1.00). (b) Agreement 86.7% (95% CI 73.2–94.9); $\kappa = 0.70$ (95% CI 0.46–0.90).

C. Quality assessment of the included studies

Table A.4
Quality assessment of the 93 included studies: judgment on each domain of Table 6, score (0–5), overall risk of bias and reproducibility level, sorted by overall risk of bias and then by score.

Study	Q1	Q2	Q3	Q4	Q5	Score	Risk of bias	R level	Study	Q1	Q2	Q3	Q4	Q5	Score	Risk of bias	R level
[103]	Y	Y	Y	Y	P	4.5	Low	R2	[51]	Y	Y	P	P	N	3	Some concerns	R1
[39]	P	Y	Y	Y	Y	4.5	Low	R3	[72]	N	Y	P	P	P	2.5	Some concerns	R2
[102]	Y	Y	Y	Y	P	4.5	Low	R2	[104]	P	Y	P	P	N	2.5	Some concerns	R1
[105]	Y	Y	Y	Y	P	4.5	Low	R2	[106]	P	P	P	P	P	2.5	Some concerns	R2
[99]	P	Y	Y	Y	P	4	Low	R2	[53]	N	Y	P	P	P	2.5	Some concerns	R2
[40]	N	Y	Y	Y	P	3.5	Low	R2	[107]	N	Y	P	P	P	2.5	Some concerns	R2
[12]	Y	Y	Y	P	Y	4.5	Some concerns	R3	[108]	P	Y	P	P	N	2.5	Some concerns	R1
[65]	Y	Y	Y	P	Y	4.5	Some concerns	R3	[82]	P	Y	P	P	N	2.5	Some concerns	R1
[50]	Y	Y	Y	P	P	4	Some concerns	R2	[61]	P	Y	P	P	N	2.5	Some concerns	R1
[73]	Y	Y	Y	P	P	4	Some concerns	R2	[74]	P	Y	P	P	N	2.5	Some concerns	R1
[88]	Y	Y	Y	P	P	4	Some concerns	R1	[70]	P	P	P	P	P	2.5	Some concerns	R1
[109]	Y	Y	Y	P	P	4	Some concerns	R1	[110]	P	Y	P	P	N	2.5	Some concerns	R1
[47]	Y	Y	Y	P	P	4	Some concerns	R2	[67]	N	Y	P	P	P	2.5	Some concerns	R2
[98]	Y	Y	Y	P	N	3.5	Some concerns	R1	[54]	N	Y	P	P	P	2.5	Some concerns	R2
[111]	Y	Y	P	P	P	3.5	Some concerns	R2	[81]	N	Y	P	P	P	2.5	Some concerns	R2
[112]	P	Y	Y	P	P	3.5	Some concerns	R2	[113]	N	Y	P	P	P	2.5	Some concerns	R1
[94]	P	Y	Y	P	P	3.5	Some concerns	R1	[84]	N	P	Y	P	P	2.5	Some concerns	R2
[90]	P	Y	Y	P	P	3.5	Some concerns	R2	[66]	P	Y	P	P	N	2.5	Some concerns	R1
[114]	P	Y	Y	P	P	3.5	Some concerns	R2	[115]	N	Y	Y	P	N	2.5	Some concerns	R1
[116]	Y	Y	P	P	P	3.5	Some concerns	R2	[97]	N	Y	Y	P	N	2.5	Some concerns	R1
[100]	P	Y	Y	P	P	3.5	Some concerns	R2	[59]	N	Y	P	P	P	2.5	Some concerns	R2
[93]	P	Y	Y	P	P	3.5	Some concerns	R2	[55]	N	Y	Y	P	N	2.5	Some concerns	R1
[117]	Y	Y	P	P	P	3.5	Some concerns	R2	[44]	P	P	Y	P	N	2.5	Some concerns	R1
[118]	P	Y	Y	P	P	3.5	Some concerns	R2	[56]	P	Y	P	P	N	2.5	Some concerns	R1
[71]	Y	Y	P	P	P	3.5	Some concerns	R1	[57]	N	Y	Y	P	N	2.5	Some concerns	R1
[58]	Y	Y	P	P	P	3.5	Some concerns	R2	[119]	P	P	P	P	N	2	Some concerns	R1
[75]	P	Y	P	Y	N	3	Some concerns	R1	[120]	P	P	P	P	N	2	Some concerns	R1
[121]	N	Y	Y	P	P	3	Some concerns	R2	[122]	N	Y	P	P	N	2	Some concerns	R1
[89]	Y	Y	P	P	N	3	Some concerns	R1	[123]	P	P	P	P	N	2	Some concerns	R1
[124]	Y	Y	P	P	N	3	Some concerns	R1	[78]	P	P	P	P	N	2	Some concerns	R1
[125]	Y	P	P	P	P	3	Some concerns	R2	[91]	N	Y	P	P	N	2	Some concerns	R1
[101]	P	Y	P	P	P	3	Some concerns	R2	[48]	N	P	P	P	P	2	Some concerns	R2
[126]	Y	P	Y	P	N	3	Some concerns	R1	[69]	N	Y	P	P	N	2	Some concerns	R1
[60]	N	Y	Y	P	P	3	Some concerns	R2	[87]	N	P	P	P	N	1.5	Some concerns	R1
[52]	Y	Y	P	P	N	3	Some concerns	R1	[76]	N	P	P	P	N	1.5	Some concerns	R1
[127]	N	Y	Y	P	P	3	Some concerns	R2	[128]	Y	Y	N	P	N	2.5	High	R1
[96]	P	Y	P	P	P	3	Some concerns	R2	[80]	Y	N	P	P	N	2	High	R1
[92]	P	Y	P	P	P	3	Some concerns	R2	[77]	P	P	P	N	P	2	High	R1
[129]	N	P	Y	Y	P	3	Some concerns	R1	[79]	N	N	Y	Y	N	2	High	R1
[62]	P	Y	P	P	P	3	Some concerns	R2	[64]	N	Y	N	P	P	2	High	R2
[86]	P	P	Y	P	P	3	Some concerns	R1	[130]	N	Y	N	P	P	2	High	R1
[85]	Y	P	Y	P	N	3	Some concerns	R1	[41]	P	N	P	P	N	1.5	High	R1
[83]	Y	P	Y	P	N	3	Some concerns	R1	[95]	P	P	P	N	N	1.5	High	R1
[131]	N	Y	Y	P	P	3	Some concerns	R2	[132]	N	P	P	N	N	1	High	R1
[46]	P	Y	Y	P	N	3	Some concerns	R1	[68]	N	P	N	P	N	1	High	R1
[49]	P	P	Y	P	P	3	Some concerns	R2	[45]	N	P	N	P	N	1	High	R1
[63]	N	Y	Y	P	P	3	Some concerns	R2									

Y: Yes (1); P: Partial (0.5); N: No (0). Risk of bias: from Q2–Q4 on the RoB 2 judgment levels (Section 2.8). R1–R3: reproducibility levels (Section 3.5.2). Studies scoring at least 3.5 form the sensitivity subset.

References

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, B. Agüera y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS), volume 54 of *Proceedings of Machine Learning Research*, PMLR, 2017, pp. 1273–1282.

[2] D. C. Nguyen, M. Ding, P. N. Pathirana, A. Seneviratne, J. Li, H. V. Poor, Federated learning for Internet of Things: A comprehensive survey, IEEE Communications Surveys & Tutorials 23 (2021) 1622–1658.

[3] A. Imteaj, U. Thakker, S. Wang, J. Li, M. H. Amini, A survey on federated learning for resource-constrained IoT devices, IEEE Internet of Things Journal 9 (2022) 1–24.

[4] P. P. Ray, A review on TinyML: State-of-the-art and prospects, Journal of King Saud University – Computer and Information Sciences 34 (2022) 1595–1623.

[5] J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, S. Han, Tiny machine learning: Progress and futures, IEEE Circuits and Systems Magazine 23 (2023) 8–34.

[6] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, S. Han, MCUNet: Tiny deep learning on IoT devices, in: Advances in Neural Information Processing Systems (NeurIPS), volume 33, 2020, pp. 11711–11722.

[7] S. S. Saha, S. S. Sandha, M. Srivastava, Machine learning for microcontroller-class hardware: A review, IEEE Sensors Journal 22 (2022) 21362–21390.

[8] H. Ren, D. Anicic, T. A. Runkler, TinyOL: TinyML with online-learning on microcontrollers, in: 2021 International Joint Conference on Neural Networks (IJCNN), IEEE, 2021, pp. 1–8. doi:10.1109/IJCNN52387.2021.9533927.

[9] J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, C. Gan, S. Han, On-device training under 256KB memory, in: Advances in Neural Information Processing Systems (NeurIPS), volume 35, 2022, pp. 22941–22954.

[10] Y. Gao, M. Kim, S. Abuadbba, Y. Kim, C. Thapa, K. Kim, S. A. Camtepe, H. Kim, S. Nepal, End-to-end evaluation of federated learning and split learning for Internet of Things, in: 2020 International Symposium on Reliable Distributed Systems (SRDS), IEEE, 2020, pp. 91–100. doi:10.1109/SRDS51746.2020.00017.

[11] C. Yang, Q. Wang, M. Xu, Z. Chen, K. Bian, Y. Liu, X. Liu, Characterizing impacts of heterogeneity in federated learning upon large-scale smartphone data, in: Proceedings of the Web Conference 2021 (WWW '21), ACM, 2021, pp. 935–946. doi:10.1145/3442381.3449851.

[12] N. Llisterri Giménez, M. Monfort Grau, R. Pueyo Centelles, F. Freitag, On-device training of machine learning models on microcontrollers with federated learning, Electronics 11 (2022) 573.

[13] L. Zhu, Z. Liu, S. Han, Deep leakage from gradients, in: Advances in Neural Information Processing Systems (NeurIPS), volume 32, Curran Associates, Inc., 2019. URL: https://proceedings.neurips.cc/paper/2019/hash/60a6c4002cc7b29142def8871531281a-Abstract.html.

[14] F. Mo, H. Haddadi, K. Katevas, E. Marin, D. Perino, N. Kourtellis, PPFL: Privacy-preserving federated learning with trusted execution environments, in: Proceedings of the 19th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys '21), ACM, 2021, pp. 94–108. doi:10.1145/3458864.3466628.

[15] C. Banbury, V. Janapa Reddi, P. Torelli, N. Jeffries, C. Kiraly, J. Holleman, P. Montino, D. Kanter, et al., MLPerf Tiny benchmark, in: Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks, volume 1, 2021. URL: https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/da4fb5c6e93e74d3df8527599fa62642-Abstract-round1.html.

[16] O. E. Gundersen, S. Kjensmo, State of the art: Reproducibility in artificial intelligence, Proceedings of the AAAI Conference on Artificial Intelligence 32 (2018).

[17] M. N. A. Ramadan, M. A. H. Ali, S. Y. Khoo, M. Alkhedher, Federated learning and TinyML on IoT edge devices: Challenges, advances, and future directions, ICT Express 11 (2025) 754–768.

[18] C. N. da Silva, C. V. S. Prazeres, Tiny federated learning for constrained sensors: A systematic literature review, IEEE Sensors Reviews 2 (2025) 17–31.

[19] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, L. Shamseer, J. M. Tetzlaff, E. A. Akl, S. E. Brennan, et al., The PRISMA 2020 statement: An updated guideline for reporting systematic reviews, BMJ 372 (2021) n71.

[20] E. Fragkou, D. Katsaros, A joint survey in decentralized federated learning and TinyML: A brief introduction to swarm learning, Future Internet 16 (2024) 413.

[21] L. Schuhmacher, J. Fernandez Landivar, I. Gryech, H. Sallouha, M. Rossi, S. Pollin, Machine learning on the edge for sustainable IoT networks: A systematic literature review, Internet of Things 36 (2026) 101846.

[22] E. Moore, A. Imteaj, S. Rezapour, M. H. Amini, A survey on secure and private federated learning using blockchain: Theory and application in resource-constrained computing, IEEE Internet of Things Journal 10 (2023) 21942–21958.

[23] E. Dritsas, M. Trigka, Federated learning for IoT: A survey of techniques, challenges, and applications, Journal of Sensor and Actuator Networks 14 (2025) 9.

[24] M. Aggarwal, V. Khullar, N. Goyal, FSL-TM: Review on the integration of federated split learning with TinyML in the Internet of Vehicles, Computers, Materials & Continua 86 (2026) 1–31.

[25] M. Aggarwal, V. Khullar, S. Rani, T. A. Prola, S. B. Bhattacharjee, S. M. Shawon, N. Goyal, Federated learning on Internet of Things: Extensive and systematic review, Computers, Materials & Continua 79 (2024) 1795–1834.

[26] M. Adam, U. Baroudi, Federated learning for IoT: Applications, trends, taxonomy, challenges, current solutions, and future directions, IEEE Open Journal of the Communications Society 5 (2024) 7842–7877.

[27] J. Wang, Z. Liu, X. Yang, M. Li, Z. Lyu, The Internet of Things under federated learning: A review of the latest advances and applications, Computers, Materials & Continua 82 (2025) 1–39.

[28] P. Boobalan, S. P. Ramu, Q.-V. Pham, K. Dev, S. Pandya, P. K. R. Maddikunta, T. R. Gadekallu, T. Huynh-The, Fusion of federated learning and Industrial Internet of Things: A survey, Computer Networks 212 (2022) 109048.

[29] W. Khalaifat, W. Elmedany, H. Alryalat, Privacy and security of federated learning in resource-constrained Internet of Things environment: Systematic literature review, Internet of Things 33 (2025) 101679.

[30] M. Ouzzani, H. Hammady, Z. Fedorowicz, A. Elmagarmid, Rayyan—a web and mobile app for systematic reviews, Systematic Reviews 5 (2016) 210.

[31] M. Monfort Grau, R. Pueyo Centelles, F. Freitag, On-device training of machine learning models on microcontrollers with a look at federated learning, in: Proceedings of the Conference on Information Technology for Social Good (GoodIT '21), ACM, 2021, pp. 198–203. doi:10.1145/3462203.3475896.

[32] B. Sudharsan, S. Salerno, D.-D. Nguyen, M. Yahya, A. Wahid, P. Yadav, J. G. Breslin, M. I. Ali, TinyML benchmark: Executing fully connected neural networks on commodity microcontrollers, in: 2021 IEEE 7th World Forum on Internet of Things (WF-IoT), IEEE, 2021, pp. 883–884. doi:10.1109/WF-IoT51360.2021.9595024.

[33] N. Schizas, A. Karras, C. Karras, S. Sioutas, TinyML for ultra-low power AI and large scale IoT deployments: A systematic review, Future Internet 14 (2022) 363.

[34] R. Immonen, T. Hämäläinen, Tiny machine learning for resource-constrained microcontrollers, Journal of Sensors 2022 (2022) 1–11.

[35] M. Canpolat Şahin, A. Kolukısa Tarhan, Evaluation and selection of hardware and AI models for edge applications: A method and a case study on UAVs, Applied Sciences 15 (2025) 1026.

[36] S. P. Baller, A. Jindal, M. Chadha, M. Gerndt, DeepEdgeBench: Benchmarking deep neural networks on edge devices, in: 2021 IEEE International Conference on Cloud Engineering (IC2E), IEEE, 2021, pp. 20–30. doi:10.1109/IC2E52221.2021.00016.

[37] A. Garcia-Perez, R. Miñón, A. I. Torre-Bastida, E. Zulueta-Guerrero, Analysing edge computing devices for the deployment of embedded AI, Sensors 23 (2023) 9495.

[38] K. Memon, N. Yahya, M. Z. Yusoff, R. Remli, A.-W. M. M. Mustapha, H. Hashim, S. S. A. Ali, S. Siddiqui, Edge computing for AI-based brain MRI applications: A critical evaluation of real-time classification and segmentation, Sensors 24 (2024) 7091.

[39] N. Llisterri Giménez, J. Lee, F. Freitag, H. Vandierendonck, The Effects of Weight Quantization on Online Federated Learning for the IoT: A Case Study, IEEE Access 12 (2024) 5490–5502.

[40] Z. Yang, Q. Sun, Energy-efficient Personalized Federated Search with Graph for Edge Computing, ACM Transactions on Embedded Computing Systems 22 (2023) 1–24.

[41] C. S. Amadi, S. O. Ajakwe, C. A. Nnadiekwe, J. Taesoo, Lightweight data security system for resource-constrained IIoT devices, Franklin Open 15 (2026) 100585.

[42] J. A. C. Sterne, J. Savović, M. J. Page, R. G. Elbers, N. S. Blencowe, I. Boutron, C. J. Cates, H.-Y. Cheng, M. S. Corbett, S. M. Eldridge, J. R. Emberson, M. A. Hernán, S. Hopewell, A. Hróbjartsson, D. R. Junqueira, P. Jüni, J. J. Kirkham, T. Lasserson, T. Li, A. McAleenan, B. C. Reeves, S. Shepperd, I. Shrier, L. A. Stewart, K. Tilling, I. R. White, P. F. Whiting, J. P. T. Higgins, RoB 2: a revised tool for assessing risk of bias in randomised trials, BMJ 366 (2019) l4898.

[43] P. Jüni, A. Witschi, R. Bloch, M. Egger, The hazards of scoring the quality of clinical trials for meta-analysis, JAMA 282 (1999) 1054–1060.

[44] L. Radrigan, A. S. Morales, P. Toledo, S. E. Godoy, E. Guerra-Vallejos, An Energy-Efficient Hierarchical Federated Learning Protocol with Downward Feature Transfer: A Simulation-Based Feasibility Study for Low-Power Edge Nodes, Electronics 15 (2026) 3448.

[45] J. Ha, A. El Azzaoui, J. H. Park, FL-TENB4: A Federated-Learning-Enhanced Tiny EfficientNetB4-Lite Approach for Deepfake Detection in CCTV Environments, Sensors 25 (2025) 788.

[46] G. Wadhwa, Y. Natarajan, K. R. Sri Preethaa, M. P. Gopinath, M. S. Devi, Federated learning for sustainable IoT appliance load monitoring at the edge devices, Discover Sustainability 6 (2025) 654.

[47] M. Bibi, Q. W. Khan, S. A. Yazdan, R. Ahmad, D.-H. Kim, DAQSyn: A decentralized adaptive quantization-aware synchronous framework in heterogeneous on-device AI networks, Neural Networks 205 (2027) 109474.

[48] M. J. C. S. Reis, Scalable Intrusion Detection in IoT Networks Via Property Testing and Federated Edge AI, IEEE Access 13 (2025) 153244–153262.

[49] R. Anandhi, G. U. Srikanth, Federated sparse-learning framework for detecting low-rate stealth attacks in resource-constrained IoT devices, Scientific Reports 16 (2026) 21298.

[50] A. Trigkas, D. Piromalis, P. Papageorgas, A Federated Approach for Adaptive Urban Sound Classification on TinyML Edge Devices, Sensors 26 (2026) 2854.

[51] T.-D. Tran, P.-D. Bui, V.-H. Pham, EdgeTrust-Shard: Hierarchical blockchain architecture for federated learning in cross-chain IoT ecosystems, Journal of Systems Architecture 173 (2026) 103701.

[52] L. Wulfert, C. Wiede, A. Grabmaier, TinyFL: On-Device Training, Communication and Aggregation on a Microcontroller For Federated Learning, in: 2023 21st IEEE Interregional NEWCAS Conference (NEWCAS), 2023, pp. 1–5. doi:10.1109/newcas57931.2023.10198040.

[53] D. Qu, Y. Ma, Edge–Mesh–Ledger: Federated AI and Blockchain Framework for Scalable Global Sustainability Solutions, in: 2025 International Conference on Artificial Intelligence for Sustainable Innovation (AI-SI), 2025, pp. 1–6. doi:10.1109/ai-si66213.2025.11341516.

[54] S. Adhikary, S. Dutta, FedTinyWolf—A Memory Efficient Federated Embedded Learning Mechanism, IEEE Embedded Systems Letters 16 (2024) 513–516.

[55] J. Pei, M. Feng, P. Muthukumar, P. V. Krishna, M. Dai, Generative AI-Native Edge Sensors: Collaborative Data Reconstruction via Lightweight Federated Learning, IEEE Journal of Selected Areas in Sensors 3 (2026) 180–190.

[56] N. Nedungadi, S. Sankaran, K. Achuthan, FedX: Privacy-Preserving Explainable Federated Ensemble Intrusion Detection System for Edge-Enabled Internet of Vehicles, Big Data and Cognitive Computing 10 (2026) 160.

[57] P. Fusco, F. Palmieri, M. Ficco, Combining epsilon-greedy reinforcement learning based gradient sparsification and siamese neural networks for few-shot federated tinyML intrusion detection in IoT, Internet of Things 34 (2025) 101820.

[58] A. Tanović, I. Mezei, Lightweight Anomaly Detection in Digit Recognition Using Federated Learning, Future Internet 17 (2025) 343.

[59] T. T. Huong, T. P. Bac, K. N. Ha, N. V. Hoang, N. X. Hoang, N. T. Hung, K. P. Tran, Federated Learning-Based Explainable Anomaly Detection for Industrial Control Systems, IEEE Access 10 (2022) 53854–53872.

[60] N. Llisterri Giménez, J. Miquel Solé, F. Freitag, Embedded federated learning over a LoRa mesh network, Pervasive and Mobile Computing 93 (2023) 101819.

[61] H. Ren, D. Anicic, T. A. Runkler, TinyReptile: TinyML with Federated Meta-Learning, in: 2023 International Joint Conference on Neural Networks (IJCNN), 2023, pp. 1–9. doi:10.1109/ijcnn54540.2023.10191845.

[62] M. Ficco, A. Guerriero, E. Milite, F. Palmieri, R. Pietrantuono, S. Russo, Federated learning for IoT devices: Enhancing TinyML with on-board training, Information Fusion 104 (2024) 102189.

[63] M. Gulati, K. Zandberg, Z. Huang, G. Wunder, C. Adjih, E. Baccelli, TDMiL: Tiny Distributed Machine Learning for Microcontroller-Based Interconnected Devices, IEEE Access 12 (2024) 167810–167826.

[64] V. Nikić, D. Bortnik, M. Lukić, D. Vukobratović, I. Mezei, Lightweight Digit Recognition in Smart Metering System Using Narrowband Internet of Things and Federated Learning, Future Internet 16 (2024) 402.

[65] B. Djamaa, H. Yekhlef, M. A. Kouda, A. Bradai, FedCoRE: Effective Federated Learning for constrained RESTful environments in the Artificial Intelligence of Things, Journal of Network and Computer Applications 244 (2025) 104357.

[66] M. Ficco, G. P. Rimoli, A. Guerriero, F. Palmieri, Comparing Model Compression Techniques for MCU-Based Federated TinyML, in: 2025 14th International Conference on Networks, Communication and Computing (ICNCC), 2025, pp. 173–178. doi:10.1109/icncc68912.2025.00034.

[67] S. Li, C. Xu, H. Wang, μ-Fed: Memory-Efficient Federated Learning on Microcontrollers for Multi-Modal Cardiovascular Monitoring, in: 2025 IEEE 31st International Conference on Parallel and Distributed Systems (ICPADS), 2025, pp. 1–8. doi:10.1109/icpads67057.2025.11322967.

[68] C. B. Ram, N. M B, S. P. Gari, Federated TinyML on ESP32: A Resource-Constrained Framework for IoT Health Monitoring, in: 2025 2nd International Conference on Software, Systems and Information Technology (SSITCON), 2025, pp. 1–6. doi:10.1109/ssitcon66133.2025.11342233.

[69] H. Ren, X. Li, D. Anicic, T. A. Runkler, TinyMetaFed: Efficient Federated Meta-learning for TinyML, in: Machine Learning and Principles and Practice of Knowledge Discovery in Databases, Communications in Computer and Information Science, Springer Nature Switzerland, 2025, pp. 126–137. doi:10.1007/978-3-031-74640-6_10.

[70] Y. Dai, R. Tan, A Continuous-Time On-Device Federated Learning Network, IEEE Transactions on Networking 34 (2026) 2791–2804.

[71] W. Guo, T. Yu, Q. Luo, X. Hu, Y. Wu, Z. Pan, An Analytical Federated Learning Method for Scalable Nonintrusive Load Monitoring, IEEE Transactions on Instrumentation and Measurement 75 (2026) 2513218–2513218.

[72] K. Ibrar, P. Fusco, G. P. Rimoli, F. Palmieri, M. Ficco, Cross-Modal Federated TinyML for MCU-based Internet of Medical Things, IEEE Journal of Biomedical and Health Informatics (2026) 1–14.

[73] Y. Tao, J. Jia, T. Deng, TinySLFL: A Flash-Endurance-Aware Federated Edge Learning Framework with Layer-Wise Delayed Aggregation for Resource-Constrained Microcontrollers, Electronics 15 (2026) 2084.

[74] O. Torres Sanchez, A. Rodrigues, G. Borges, D. Raposo, F. Boavida, J. S. Silva, A Framework for Tiny Federated Learning in Resource-Constrained IIoT Environments: A Case Study, IEEE Internet of Things Journal 13 (2026) 19076–19089.

[75] A. Zunic, P. Vasiljevic, M. Popovic, Anomaly detection system at the edge of the internet based on the MPT-FLA, in: 2026 IEEE Zooming Innovation in Consumer Technologies International Conference (ZINC), 2026, pp. 93–98. doi:10.1109/zinc69910.2026.11655795.

[76] R. Al-Zaidi, M. Al-Khalidi, O. Alani, H. Takruri, D. Jose, Lightweight Federated Intrusion Detection System for Resource Constrained IoT Networks, in: Proceedings of the 9th International Conference on Future Networks and Distributed Systems, 2025, pp. 82–87. doi:10.1145/3789692.3789703.

[77] A. Alharbi, Predictive fault management in smart sensor networks using a dynamic quantum-AI architecture (DynaQuAI), Scientific Reports 16 (2026) 21551.

[78] O. Torres Sanchez, G. Borges, D. Raposo, A. Rodrigues, F. Boavida, J. S. Silva, Enhancing the performance of Industrial IoT LoRaWAN-enabled federated learning frameworks: A case study, Internet of Things 32 (2025) 101632.

[79] M. Nkoom, D. Commey, Y. Alsenani, S. G. Hounsinou, G. V. Crosby, Federated DDoS Detection with Clustered Quantization-Aware Training Models for IoRT, in: 2026 IEEE 23rd Consumer Communications & Networking Conference (CCNC), 2026, pp. 1–6. doi:10.1109/ccnc65079.2026.11366492.

[80] G. Özmen, D. Yiltas-Kaplan, Robust and Lightweight Federated Learning for NB-IoT Security: A Blockchain-Verified CNN-RNN Approach, Sensors 26 (2026) 3578.

[81] R. Chen, Q. Wan, X. Zhang, X. Qin, Y. Hou, D. Wang, X. Fu, M. Pan, EEFL: High-Speed Wireless Communications Inspired Energy Efficient Federated Learning over Mobile Devices, in: Proceedings of the 21st Annual International Conference on Mobile Systems, Applications and Services, 2023, pp. 544–556. doi:10.1145/3581791.3596865.

[82] S. Ye, L. Zeng, Q. Wu, K. Luo, Q. Fang, X. Chen, Eco-FL: Adaptive Federated Learning with Efficient Edge Collaborative Pipeline Training, in: Proceedings of the 51st International Conference on Parallel Processing, 2022, pp. 1–11. doi:10.1145/3545008.3545015.

[83] D.-J. Han, D.-Y. Kim, M. Choi, D. Nickel, J. Moon, M. Chiang, C. G. Brinton, Federated Split Learning With Joint Personalization-Generalization for Inference-Stage Optimization in Wireless Edge Networks, IEEE Transactions on Mobile Computing 23 (2024) 7048–7065.

[84] G. Wilkins, S. Di, J. C. Calhoun, Z. Li, K. Kim, R. Underwood, R. Mortier, F. Cappello, FedSZ: Leveraging Error-Bounded Lossy Compression for Federated Learning Communications, in: 2024 IEEE 44th International Conference on Distributed Computing Systems (ICDCS), 2024, pp. 577–588. doi:10.1109/icdcs60910.2024.00060.

[85] R. S. A, T. R. A, A. K. Samy, FedEnviSense: A Federated Dual-Domain Acoustic Intelligence Framework for Urban and Environmental Event Detection, in: 2025 International Conference on Electrical, Communication, and Computing Technologies (iCONECCT), 2025, pp. 1–7. doi:10.1109/iconecct67014.2025.11470309.

[86] C. Guo, H. Fan, S. Dong, M. Yin, G. Qi, Y. Ma, C. Jing, H. Liu, N. Song, Y. Kang, ST-CDF: A Generative AI Framework for Physics-Consistent Imputation and Simulation in Precision Agriculture, Applied Sciences 16 (2026) 6250.

[87] T. Zhang, D. Xu, J. Ma, A. K. Bashir, M. M. Al Dabel, H. Feng, Deep Federated Fractional Scattering Network for Heterogeneous Edge Internet of Vehicles Fingerprinting: Theory and Implementation, IEEE Internet of Things Journal 12 (2025) 4935–4952.

[88] S. Alawadi, A. Ait-Mlouk, S. Toor, A. Hellander, Toward efficient resource utilization at edge nodes in federated learning, Progress in Artificial Intelligence 13 (2024) 101–117.

[89] H. Patel, S. K. Amalapuram, S. Kumar, B. R. Tamma, IoT-FedMalDetect: Federated Learning Based Malware Detection for IoT Edge Devices, in: 2025 IEEE Conference on Communications and Network Security (CNS), 2025, pp. 1–9. doi:10.1109/cns66487.2025.11194927.

[90] L. Ibraimi, M. Selimi, F. Freitag, BePOCH: Improving Federated Learning Performance in Resource-Constrained Computing Devices, in: 2021 IEEE Global Communications Conference (GLOBECOM), 2021, pp. 1–6. doi:10.1109/globecom46510.2021.9685095.

[91] S. Shukla, S. Rafatirad, H. Homayoun, S. M. P. Dinakarrao, Federated Learning with Heterogeneous Models for On-device Malware Detection in IoT Networks, in: 2023 Design, Automation & Test in Europe Conference & Exhibition (DATE), 2023, pp. 1–6. doi:10.23919/date56975.2023.10137288.

[92] R. Akinie, N. K. Gyimah, M. Bhavsar, J. Kelly, Fine-Tuning Federated Learning-Based Intrusion Detection Systems for Transportation IoT, in: SoutheastCon 2025, 2025, pp. 1155–1161. doi:10.1109/southeastcon56624.2025.10971473.

[93] S. Pal, M. Umair, W.-H. Tan, Y.-L. Foo, Practical Evaluation of Federated Learning in Edge AI for IoT, JOIV : International Journal on Informatics Visualization 7 (2023) 2115–2125.

[94] S. Pal, W.-H. Tan, Y.-L. Foo, Privacy-Focused Edge AI-on-IoT with Federated Learning, in: 2023 IEEE 8th International Conference on Recent Advances and Innovations in Engineering (ICRAIE), 2023, pp. 1–6. doi:10.1109/icraie59459.2023.10468159.

[95] M. Rahmati, A. Pagano, Federated Learning-Driven Cybersecurity Framework for IoT Networks with Privacy Preserving and Real-Time Threat Detection Capabilities, Informatics 12 (2025) 62.

[96] S. Xu, A Lightweight Federated Learning Intrusion Detection Framework for the Internet of Things, in: 2026 International Conference on Information Communication, IoT Technology, and Smart Cities (IITS), 2026, pp. 67–70. doi:10.1109/iits69561.2026.11648442.

[97] X. Zhou, W. Bi, Y. Wang, F. Khan, R. A. Aldmour, Privacy-Preserving Edge AI Security Architecture for Consumer Electronics in Cultural Heritage Environments, IEEE Transactions on Consumer Electronics 72 (2026) 4837–4844.

[98] S. Ma, J. Han, H. Zheng, Fed-YOLOv10 framework for sustainable privacy-preserving real-time classroom behavior analysis, Cognitive Robotics 6 (2026) 196–206.

[99] H. Dang, CHRONOS: A Hardware-Assisted Phase-Decoupled Framework for Secure Federated Learning in IoT, Concurrency and Computation: Practice and Experience 38 (2026) e70891.

[100] Y. Xu, Y. Liao, H. Xu, Z. Ma, L. Wang, J. Liu, Adaptive Control of Local Updating and Model Compression for Efficient Federated Learning, IEEE Transactions on Mobile Computing 22 (2023) 5675–5689.

[101] Y. Liao, Y. Xu, H. Xu, L. Wang, Z. Yao, C. Qiao, MergeSFL: Split Federated Learning with Feature Merging and Batch Size Regulation, in: 2024 IEEE 40th International Conference on Data Engineering (ICDE), 2024, pp. 2054–2067. doi:10.1109/icde60146.2024.00164.

[102] Y. Deng, W. Chen, J. Ren, F. Lyu, Y. Liu, Y. Liu, Y. Zhang, TailorFL: Dual-Personalized Federated Learning under System and Data Heterogeneity, in: Proceedings of the 20th ACM Conference on Embedded Networked Sensor Systems, 2022, pp. 592–606. doi:10.1145/3560905.3568503.

[103] B. Wu, Z. Ding, J. Huang, RELIEF: Turning Missing Modalities into Training Acceleration for Federated Learning on Heterogeneous IoT Edge, IEEE Internet of Things Journal (2026) 1–1.

[104] Y. Zhao, F. Ullah, K. Mahmood, A. Ahad, Z. Rahman, L. Mostarda, D. Cacciagrano, Federated Meta-Learning-Driven Self-Organizing Maps for Edge-based Privacy-Preserving Intrusion Detection in

Industrial IoT, IEEE Internet of Things Journal (2026) 1–1.

[105] Y. Cai, Z. Zhang, D. Li, Y. Guo, X. Chen, Moss: Proxy Model-based Full-Weight Aggregation in Federated Learning with Heterogeneous Models, Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies 9 (2025) 1–31.

[106] T.-A. Nguyen, V.-P. Bui, S. R. Pandey, K. Hue Ta, N. H. Tran, P. Popovski, Microsecond Federated SVD on Grassmann Manifold for Real-time IoT Intrusion Detection, in: ICC 2026 - IEEE International Conference on Communications, 2026, pp. 1–6. doi:10.1109/icc59461.2026.11587063.

[107] K. Swetha, K. Shreyas, V. K. Balaji, K. Manivannan, FEEL-ECG: Federated Edge Learning for Personalized and Explainable ECG Monitoring with Adaptive Compression and Preprocessing, in: 2025 IEEE 6th India Council International Subsections Conference (INDISCON), 2025, pp. 1–7. doi:10.1109/indiscon66021.2025.11254407.

[108] F. Akbarian, R. Vlaeminck, J. Verheijen, A. Aminifar, A. Aminifar, Federated Learning for Obstacle Detection to Assist the Visually-Impaired Using Augmented Reality, in: 2025 28th Euromicro Conference on Digital System Design (DSD), 2025, pp. 486–490. doi:10.1109/dsd67783.2025.00073.

[109] S. Razaghi, A. Aral, CE-FedAvg: A Communication-Efficient Federated Learning Framework for LoRaWAN-Based Edge AI, in: ICC 2026 - IEEE International Conference on Communications, 2026, pp. 1–6. doi:10.1109/icc59461.2026.11587288.

[110] A. Gupta, Ritwik, V. K. Chaurasiya, Adaptive Low-Latency Split Federated Learning With Dynamic Model Partitioning in Resource-Constrained Healthcare IoT, IEEE Transactions on Green Communications and Networking 10 (2026) 1308–1321.

[111] D. S, M. G. Shankar, E. Daniel, B. G. V. R, Enhancing security in IoMT using federated TinyGAN for lightweight and accurate malware detection, Scientific Reports 16 (2026) 7116.

[112] Y. Xu, Y. Liao, H. Xu, Z. Yao, J. Cheng, C. Qiao, Enhancing Split Federated Learning With Worker Clustering and Feature Compression, IEEE Journal on Selected Areas in Communications 43 (2025) 4091–4106.

[113] S. Savazzi, V. Rampa, S. Kianoush, M. Bennis, An Energy and Carbon Footprint Analysis of Distributed and Federated Learning, IEEE Transactions on Green Communications and Networking 7 (2023) 248–264.

[114] R. Shan, J. C. Calhoun, Layer Adaptive Error-Bounded Compression for Federated Learning on Edge Devices, in: 2026 2nd International Conference on Artificial Intelligence Systems (AIS), 2026, pp. 312–319. doi:10.1109/ais69919.2026.11621065.

[115] C. Tong, L. Zhang, Y. Ding, D. Yue, A Heterogeneity-Aware Adaptive Federated Learning Framework for Short-Term Forecasting in Electric IoT Systems, IEEE Internet of Things Journal 12 (2025) 15388–15403.

[116] A. Gupta, S. Misra, N. Pathak, D. Das, FedCare: Federated Learning for Resource-Constrained Healthcare Devices in IoMT System, IEEE Transactions on Computational Social Systems 10 (2023) 1587–1596.

[117] S. M. Mudavat, A. Mitra, S. P. Mohanty, E. Kougianos, FLitViT: Vision Transformers-Based Federated Learning Approach for Plant Disease Classification, in: 2025 OITS International Conference on Information Technology (OCIT), 2025, pp. 297–302. doi:10.1109/ocit66168.2025.11400427.

[118] M. Bibi, Q. W. Khan, S. A. Yazdan, R. Ahmad, D.-H. Kim, LHFL: Logical hierarchical federated learning mechanism in distributed physical AI networks, Internet of Things 39 (2026) 102055.

[119] S. Vashisht, S. Rani, FSSL-TCN-Per: A Federated Semi-Supervised Personalized Model for Next-Generation Secure Bio-Medical and Healthcare Solutions, IEEE Transactions on Consumer Electronics 71 (2025) 11387–11394.

[120] Y. Tian, R. Chandrasekaran, K. Ergun, X. Yu, T. Rosing, Federated Hyperdimensional Computing: Comprehensive Analysis and Robust Communication, ACM Transactions on Internet of Things 6 (2025) 1–30.

[121] P. Lai, X. Jiang, H. Phan, C. Borcea, K. Tran, A. Chen, V. D. Mayyuri, R. Jin, FedX: Adaptive Model Decomposition and Quantization for IoT Federated Learning, in: 2025 21st International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT), 2025, pp. 212–220. doi:10.1109/dcoss-iot65416.2025.00035.

[122] F. La Rosa, M. T. Reggio, P. Dell'Acqua, R. Marino, M. Villari, From open-vocabulary detection to structural assessment: An agentic federated approach for guardrail monitoring, Telematics and Informatics Reports 22 (2026) 100334.

[123] T. Ma, Z. Ye, S. Zhang, C. Ma, Research on an Ophthalmic Disease Auxiliary Diagnosis Strategy Based on Heterogeneous Federated Learning, in: Proceedings of the 2025 6th International Symposium on Artificial Intelligence for Medical Sciences, 2025, pp. 223–229. doi:10.1145/3777577.3777613.

[124] D. Chaudhary, S. Rajasegarar, S. R. Pokhrel, L. Pan, R. D, In-network Attack Detection with Federated Deep Learning in IoT Networks: Real Implementation and Analysis, in: 2025 IEEE Conference on Engineering Informatics (ICEI), 2025, pp. 1–9. doi:10.1109/icei68356.2025.11603755.

[125] S. Zhan, Y. Wu, C. Tian, Y. Zhao, L. Li, Heterogeneity-Aware Coordination for Federated Learning via Stitching Pre-trained blocks, in: 2024 IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS), 2024, pp. 1–10. doi:10.1109/iwqos61813.2024.10682959.

[126] P. Li, Y. Zhong, C. Zhang, Y. Wu, R. Yu, FedRelay: Federated Relay Learning for 6G Mobile Edge Intelligence, IEEE Transactions on Vehicular Technology 72 (2023) 5125–5138.

[127] S. Baghersalimi, T. Teijeiro, D. Atienza, A. Aminifar, Personalized Real-Time Federated Learning for Epileptic Seizure Detection, IEEE Journal of Biomedical and Health Informatics 26 (2022) 898–909.

[128] S. Douch, M. R. Abid, K. Zine-Dine, D. Bouzidi, S. Bourhnane, D. Benhaddou, Early Exists Federated Learning (EEFL): Brining Training to the Edge, in: 2024 Sixth International Conference on Intelligent Computing in Data Sciences (ICDS), 2024, pp. 1–8. doi:10.1109/icds62089.2024.10756466.

[129] A. Agiollo, P. Bellavista, M. Mendula, A. Omicini, EneA-FL: Energy-aware orchestration for serverless federated learning, Future Generation Computer Systems 154 (2024) 219–234.

[130] M. M. Maheri, D. Herasymuk, H. Haddadi, Client Clustering Meets Knowledge Sharing: Enhancing Privacy and Robustness in Personalized Peer-to-Peer Learning, in: 2025 IEEE Annual Congress on Artificial Intelligence of Things (AIoT), 2025, pp. 223–230. doi:10.1109/aiot66900.2025.00042.

[131] K. Wang, Z. Zhou, Z. Li, LATTE: Layer Algorithm-aware Training Time Estimation for Heterogeneous Federated Learning, in: Proceedings of the 30th Annual International Conference on Mobile Computing and Networking, 2024, pp. 1470–1484. doi:10.1145/3636534.3690705.

[132] B. R. A, S. B. Basheeruddin, S. Manohar, S. M. Asif, P. Thanuja, Implementation of Federated Transformer-based Edge-Level Intrusion Detection Systems, in: 2026 6th International Conference on Trends in Material Science and Inventive Materials (ICTMIM), 2026, pp. 1549–1554. doi:10.1109/ictmim68190.2026.11506625.